

M É M O I R E

Présenté en vue de l'obtention du
MASTÈRE DE RECHERCHE

Mention : Robotique, Informatique et Systèmes de Communication
Parcours : Data Science & Smart Services

par
Houwaida Hidri

Étude et implementation d'algorithmes intelligents de navigation autonome d'un robot mobile

Soutenu le 24/12/2024 devant le Jury d'examen composé de :

<i>Présidente :</i> Mme. Zohra KARDOUS	Maître de conférences	ENIT
<i>Rapporteur :</i> M. Zied TMAR	Maître Assistant	ISTIC
<i>Encadrante :</i> Mme. Wissal BEY	Maître Assistant	ISTIC

Année Universitaire : 2023 – 2024

Remerciements

Tout d'abord, j'exprime ma profonde gratitude à mon encadrante, Mme **Wissal Bey**. Ses commentaires précieux, ses encouragements, sa guidance et ses suggestions constructives ont fourni une base solide pour ce mémoire de recherche. J'apprécie grandement nos discussions enrichissantes et je tiens à lui témoigner ma reconnaissance de m'avoir consacré généreusement de son temps.

Je tiens à exprimer ma reconnaissance à Mme **Zohra Kardous**, la présidente du jury, pour son attention et son expertise lors de l'évaluation de mon travail.

Un immense merci à M **Zied Tmar**, rapporteur, pour sa lecture attentive et ses remarques pertinentes.

Je remercie les membres du jury d'avoir pris le temps d'examiner mon travail et j'espère qu'ils y trouveront la qualité et la clarté attendues.

Ma gratitude s'adresse à tous les enseignants de l'Institut Supérieur des Technologies de l'Informatique et de la Communication qui ont contribué à ma formation.

Ce travail n'aurait pas été possible sans le soutien de ma famille. Je tiens à remercier tout particulièrement ma mère **Amel**, mon père **Mounir**, et ma sœur **Safa** pour leur aide et leurs encouragements. Un merci tout spécial à mes amies **Maya**, **Maryem** et **Feriel** pour leur soutien et leur amitié.

Je tiens à remercier M **Gilles Sabas** fondateur et directeur de A7 Emailing et la directrice administrative Mme **Arbia Boujiza**, pour leur soutien et leur accompagnement au sein de l'entreprise, ainsi que toute l'équipe de l'entreprise.

Dédicace

Ce travail est dédié à mon petit frère **Youssef**, décédé le 16/08/2023. Je m'engage à continuer d'avancer et à réussir en son honneur. Chaque pas que je fais, chaque défi que je relève, sera une manière de lui rendre hommage.

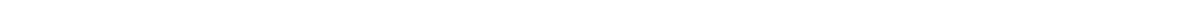


Table des matières

Introduction Générale	1
1 Généralités sur les robots autonomes	3
1.1 Introduction	4
1.2 Les robots mobiles	4
1.2.1 Architecture matérielle	4
1.2.2 Frameworks et middlewares pour la robotique	11
1.3 Navigation autonome	16
1.3.1 Perception de l'environnement	18
1.3.2 Localisation Avancée	20
1.3.3 Algorithmes de planification de trajectoire	21
1.4 Sécurité et fiabilité	21
1.5 Principes éthiques	22
1.6 Tendances Émergentes	22
1.6.1 Perception Bio-inspirée	22
1.6.2 Edge Computing pour la Perception	23
1.7 Conclusion	23
2 Algorithmes intelligents pour la mobilité autonome	25
2.1 Introduction	26
2.2 Apprentissage automatique	26
2.2.1 Apprentissage supervisé	27
2.2.2 Apprentissage non supervisé	29
2.2.3 Apprentissage par renforcement	29
2.2.4 Apprentissage profond	31
2.2.5 Systèmes experts	32
2.2.6 Logique Floue	32
2.2.7 Algorithmes génétique	33
2.2.8 Traitement du langage naturel (NLP)	33
2.3 État de l'art	35
2.3.1 Q-learning et navigation autonome	35
2.3.2 Intégration de l'IoT et des techniques de détection d'objets	35
2.3.3 Environnements d'apprentissage et défis	35
2.3.4 Gestion de la mémoire et traitement en temps réel	36
2.4 Apprentissage machine pour la perception de l'environnement	36
2.5 Avantages de l'intelligence artificielle et de la robotique	37
2.6 Conclusion	37
3 Environnements matériels et logiciels	38
3.1 Introduction	39
3.2 Robot MiniLab	39
3.2.1 Définition	39
3.2.2 Architecture de MiniLab	39
3.3 Environnement de travail	41

3.3.1	VMware Workstation	41
3.3.2	Ubuntu	42
3.3.3	Gazebo	42
3.3.4	Rviz	43
3.4	Conception de l'environnement sur Gazebo	43
3.4.1	Structure de l'interface d'accueil de Gazebo	43
3.4.2	Construction d'un modèle sur Gazebo	44
3.5	Placement du modèle du robot dans l'environnement de simulation	48
3.5.1	Mise en place de l'environnement	48
3.6	Simulation du robot avec le modèle réalisé	51
3.7	Architecture et fonctionnement des nœuds ROS	53
3.7.1	Nœuds actifs	53
3.7.2	Topics actifs	54
3.8	Conclusion	55
4	Étude et implémentation d'algorithmes de planification de trajectoire pour la navigation autonome	56
4.1	Introduction	57
4.2	Acquisition et traitement des données à partir des topics ROS	57
4.3	Analyse de l'algorithme de navigation existant	58
4.3.1	Principes fondamentaux de l'algorithme de planification de la fenêtre dynamique	59
4.3.2	Fonctionnement de l'algorithme de la fenêtre dynamique (DWA)	59
4.3.3	Avantages et limites de l'algorithme de la fenêtre dynamique	61
4.3.4	Applications du DWA	61
4.4	Implémentation de l'algorithme Q Learning	62
4.4.1	Introduction au Q Learning	62
4.4.2	Représentation de l'environnement	62
4.4.3	Fonction goal_reached	64
4.4.4	Fonction Q	65
4.4.5	Algorithme d'apprentissage	66
4.5	Comparaison	67
4.5.1	Méthodologie	67
4.5.2	Réactivité	67
4.5.3	Planification à Long Terme	68
4.5.4	Gestion des Objets Dynamiques	68
4.5.5	Complexité et Implémentation	68
4.5.6	Robustesse	68
4.6	Validation expérimentale	68
4.7	Conclusion	70
	Conclusion Générale et Perspectives	72
	Bibliographie	74

Table des figures

1.1	Les capteurs clés des robots autonomes	5
1.2	Moteur à courant continu	7
1.3	Vérin électrique	7
1.4	Exemples des robots utilisant l'architecture Intel Core	8
1.5	Exemples des robots utilisant l'architecture MCU	8
1.6	Exemples des robots utilisant l'architecture ARM	8
1.7	Exemples des robots utilisant l'architecture SoC	9
1.8	Exemples des robots utilisant l'architecture FPGA	9
1.9	Exemples des robots utilisant l'architecture DSP	9
1.10	Exemples des robots utilisant l'architecture AI Processors	10
1.11	Logo de ROS	11
1.12	Les versions du ROS	13
1.13	Principe de fonctionnement du ROS	15
1.14	Robot Humanoïde Kismet	16
1.15	Boucle de navigation autonome	17
2.1	Méthodes d'intelligence artificielle	26
2.2	Régression Linéaire	27
2.3	Les types de classification	28
2.4	Clustering	29
2.5	Mécanisme de l'apprentissage par renforcement	30
3.1	Robot MiniLab	39
3.2	Les dimensions du robot MiniLab	40
3.3	Les performances de robot MiniLab	41
3.4	Logo de VMware Workstation	41
3.5	Logo de Ubuntu	42
3.6	Logo de Gazebo	43
3.7	Logo de Rviz	43
3.8	Interface d'accueil de Gazebo	44
3.9	Accès à l'outil Building Editor dans Gazebo	45
3.10	Interface principale du Building Editor dans Gazebo	45
3.11	Utilisation du menu « Create Walls » dans le Building Editor de Gazebo	46
3.12	Modification des propriétés du mur dans le modèle	46
3.13	Modèle final avec obstacles dans Gazebo	47
3.14	Structure du fichier model.config pour la configuration de modèle	47
3.15	Éléments du fichier model.sdf pour la description du modèle	48
3.16	Contenu du dossier MiniLab	48
3.17	Processus de configuration d'un Catkin workspace	49
3.18	Résultat de l'exécution de la commande catkin_make	50
3.19	Résultat de l'exécution de la commande catkin_make (suite)	51
3.20	Lancement du fichier de simulation avec roslaunch	52
3.21	Résultat de la simulation	53
3.22	Liste des nœuds actifs	54

3.23	Liste des Topics	55
4.1	Architecture du script SensorDataLogger pour l'acquisition des données	57
4.2	Algorithme de la fenêtre dynamique (DWA)	60
4.3	Extraction des données de l'odomètre pour la représentation de l'état du robot .	63
4.4	Récupération de la distance entre le robot et l'obstacle	63
4.5	Fonctions pour la définition de l'objectif et la vérification de l'atteinte de l'objectif par le robot	64
4.6	Contrôle des vitesses du robot	65
4.7	Initialisation de la fonction Q et des paramètres du Q-learning pour la navigation du robot	65
4.8	Définition de la fonction take_action	66
4.9	Mis à jour de la Q-table	66
4.10	Résultat de l'exécution	69
4.11	Graphe de la valeur de Q	69
4.12	Graphe de la somme des récompenses	70
4.13	Graphe de la distance au but après chaque action	70

Glossaire des acronymes et définitions

ARM (Acorn RISC Machine) une architecture de processeur RISC (Reduced Instruction Set Computing) développée par ARM Limited.

CPU (Central Processing Unit) responsable du traitement des données et l'exécution des algorithmes nécessaires à la navigation, à la perception et au contrôle.

CNN (Convolutional Neural Network) un type d'architecture de deep learning utilisé principalement pour l'analyse d'images et la reconnaissance visuelle.

DWA (Dynamic Window Approach) un algorithme de planification de trajectoire pour les robots mobiles.

DSP (Digital Signal Processor) un type de processeur spécialement conçu pour traiter des signaux numériques en temps réel, permettant des opérations telles que le filtrage, l'analyse de fréquence et la compression de données.

FPGA (Field-Programmable Gate Array) un circuit intégré reconfigurable qui permet aux utilisateurs de programmer et de personnaliser le matériel après sa fabrication, offrant ainsi une flexibilité pour développer des systèmes numériques spécifiques.

GPS (Global Positioning System) un système de navigation par satellite qui permet de déterminer la position exacte d'un objet sur la surface de la Terre.

IA (Intelligence Artificielle) la capacité des machines et des systèmes informatiques à effectuer des tâches qui nécessitent normalement l'intelligence humaine, comme la perception, la prise de décision, l'apprentissage et la résolution de problèmes.

KNN (K-Nearest Neighbors) un algorithme d'apprentissage supervisé utilisé pour la classification et la régression.

LisP (List Processing) un langage de programmation de haut niveau, principalement utilisé en intelligence artificielle et en traitement symbolique.

OSRF (Open Source Robotics Foundation) une organisation à but non lucratif qui soutient le développement de logiciels de robotique open source.

OGRE (Open Graphics Rendering Engine) un moteur de rendu graphique open source conçu pour faciliter le développement d'applications 3D.

PG (Policy Gradient) une classe d'algorithmes d'apprentissage par renforcement qui optimisent directement la politique d'un agent.

PR2 (Personal Robot 2) un robot mobile développé par Willow Garage, conçu pour la recherche en robotique et l'interaction avec les humains.

ROS (Robot Operating System) un ensemble d'outils logiciels et de bibliothèques conçu pour faciliter le développement de robots.

RTC (Robotic Technology Component) Un composant de technologie robotique désignant les éléments matériels et logiciels qui constituent un système robotique.

RNN (Recurrent Neural Network) un type d'architecture de deep learning conçu pour traiter des séquences de données.

RNA (Réseau de Neurones Artificiels) un modèle computationnel inspiré du fonctionnement des neurones biologiques.

SAIL (Stanford Artificial Intelligence Laboratory) un langage de programmation développé à Stanford dans les années 1970, spécifiquement conçu pour la recherche en intelligence artificielle.

STAIR (Stanford AI Robot) un projet de robotique développé par l'Université de Stanford, visant à créer un robot capable d'effectuer des tâches d'assistance dans des environnements domestiques et de bureau.

SVM (Support Vector Machine) un algorithme d'apprentissage supervisé utilisé pour la classification et la régression.

Introduction Générale

Dans un contexte où l'automatisation et l'intelligence artificielle révolutionnent de nombreux secteurs, la robotique mobile autonome occupe une place de plus en plus prépondérante. Les robots mobiles capables de naviguer de manière autonome dans des environnements complexes et dynamiques trouvent des applications dans des domaines variés tels que l'industrie, la logistique, l'exploration ou l'assistance aux personnes. Cette évolution rapide soulève de nombreux défis technologiques, notamment en ce qui concerne les algorithmes de navigation et de prise de décision.

Ce mémoire, intitulé "Étude et implémentation d'algorithmes intelligents de navigation autonome d'un robot mobile", s'inscrit dans cette dynamique d'innovation. Notre travail se concentre spécifiquement sur le robot MiniLab et vise à explorer, analyser et mettre en œuvre des solutions algorithmiques avancées pour permettre à ce robot de se déplacer de manière autonome et efficace dans son environnement.

L'objectif principal de ce projet est :

1. Étudier des solutions intelligentes de navigation autonome.
2. Implémenter ces solutions sous l'environnement Robot Operating System (ROS) pour le robot MiniLab.

Pour atteindre ces objectifs, notre travail s'articule autour de trois axes principaux :

- Une étude bibliographique approfondie des algorithmes intelligents de navigation autonome.
- Une analyse détaillée de l'architecture du robot mobile MiniLab.
- Le développement et l'implémentation d'algorithmes intelligents de navigation autonome basés sur des techniques d'intelligence artificielle, notamment l'apprentissage par renforcement

La navigation autonome d'un robot mobile comme le MiniLab implique plusieurs aspects cruciaux : la perception de l'environnement, la cartographie 3D, la planification de trajectoire, et la prise de décision en temps réel. Chacun de ces aspects repose sur des algorithmes spécifiques qui doivent être non seulement performants individuellement, mais aussi capables de fonctionner de manière intégrée pour assurer une navigation fluide et sûre.

Notre étude se concentre sur l'étude et l'implémentation de différentes approches d'intelligence artificielle, notamment l'apprentissage automatique (supervisé, non supervisé, et l'apprentissage par renforcement) et l'apprentissage profond. Nous explorerons comment ces techniques peuvent être appliquées à la perception de l'environnement et à la prise de décision en robotique mobile.

La méthodologie adoptée combine recherche théorique, modélisation, implémentation logicielle et expérimentation. Nous utiliserons des outils de simulation avancés tels que Gazebo et Rviz, intégrés dans l'environnement ROS, pour tester nos algorithmes avant de les déployer sur le robot MiniLab physique.

L'importance de ce travail réside dans son potentiel à améliorer l'autonomie, l'efficacité et la sécurité des robots mobiles, en particulier du MiniLab. Les résultats de cette étude pourront contribuer à l'avancement des connaissances dans le domaine de la robotique mobile et potentiellement trouver des applications dans divers secteurs.

Ce mémoire est structuré en quatre parties principales :

1. Généralités sur les robots autonomes : Cette partie établit les bases théoriques en abordant l'architecture matérielle et logicielle des robots mobiles, les principes de navigation autonome, ainsi que les aspects de sécurité, de fiabilité et d'éthique.
2. Algorithmes intelligents pour la mobilité autonome : Cette section se concentre sur les différentes approches d'apprentissage automatique, notamment l'apprentissage profond et l'apprentissage par renforcement, ainsi que leur application à la perception de l'environnement en robotique.
3. Plateforme expérimentale et environnement de travail : Cette section présente en détail le robot MiniLab, l'environnement de travail (VMware, Ubuntu, Gazebo, Rviz), la conception de l'environnement de simulation, et l'architecture des nœuds ROS.
4. Algorithmes intelligents pour la mobilité autonome : Cette partie se concentre sur l'explication de l'algorithme de la fenêtre dynamique (Dynamic Window Approach), et l'implémentation d'un algorithme basé sur l'apprentissage par renforcement (Q Learning).

En conclusion, nous discuterons les résultats obtenus, des implications de notre travail et proposerons des perspectives pour de futures recherches dans ce domaine en constante évolution.

Généralités sur les robots autonomes

Sommaire

1.1	Introduction	4
1.2	Les robots mobiles	4
1.2.1	Architecture matérielle	4
1.2.2	Frameworks et middlewares pour la robotique	11
1.3	Navigation autonome	16
1.3.1	Perception de l'environnement	18
1.3.2	Localisation Avancée	20
1.3.3	Algorithmes de planification de trajectoire	21
1.4	Sécurité et fiabilité	21
1.5	Principes éthiques	22
1.6	Tendances Émergentes	22
1.6.1	Perception Bio-inspirée	22
1.6.2	Edge Computing pour la Perception	23
1.7	Conclusion	23

1.1 Introduction

Un robot est un dispositif composé d'un ensemble de capteurs, d'actionneurs et de systèmes de contrôle qui lui permettent de percevoir son environnement, de prendre des décisions et d'exécuter des tâches de manière plus efficace et plus précise que l'homme, ou des tâches qui mettraient en danger la vie humaine. Dans ce chapitre, nous nous intéressons aux robots mobiles, en nous concentrant sur leur navigation autonome, leur sécurité et leur fiabilité, ainsi que les principes éthiques qui les soutiennent.

1.2 Les robots mobiles

Les robots mobiles sont des systèmes mécaniques, électroniques et informatiques, capables de se déplacer de manière autonome ou semi-autonome dans un environnement physique. Ils sont largement utilisés dans divers domaines, parmi lesquels :

- **Dans l'industrie** : Les robots mobiles servent notamment pour le transport, l'inspection, la maintenance et la fabrication flexible personnalisée.
- **En robotique de service** : Les robots mobiles sont utilisés comme assistants pour les personnes âgées ou handicapées.
- **Pour l'exploration** : Les robots mobiles sont employés pour l'exploration d'espaces difficiles d'accès, comme les milieux sous-marins, où ils peuvent cartographier et explorer ces environnements sans mettre en danger d'équipes humaines.
- **Sécurité et surveillance** : Les robots sont employés pour surveiller des zones sensibles, ces robots peuvent détecter des intrusions ou des comportements suspects grâce à des capteurs avancés.
- **Sauvetage et intervention d'urgence** : Les robots sont utilisés lors de catastrophes naturelles pour rechercher des survivants dans des décombres ou des zones dangereuses, minimisant ainsi le risque pour les équipes humaines.
- **Tourisme et hôtellerie** : Dans le secteur du tourisme, certains hôtels utilisent des robots pour accueillir les clients, fournir des informations ou même livrer des services dans les chambres.

1.2.1 Architecture matérielle

Les robots mobiles partagent une structure commune, comprenant quatre composantes essentielles : un ensemble de capteurs pour percevoir l'environnement, des unités de traitement pour analyser les données et prendre des décisions, des actionneurs pour exécuter les mouvements et actions et des sources d'énergie pour assurer un fonctionnement autonome.

1.2.1.1 Capteurs

Les capteurs sont des composants électroniques qui transforment des grandeurs physiques en une autre grandeur de nature différente. Ils mesurent des paramètres comme la distance, la lumière ou la température. Utilisant divers principes physiques, ils génèrent des données analogiques ou numériques. Ces informations sont traitées par le système de contrôle du robot, permettant une perception en temps réel de l'environnement pour la navigation et l'interaction. La Figure 1.1 montre l'ensemble des capteurs qui sont nécessaire à la robotique.

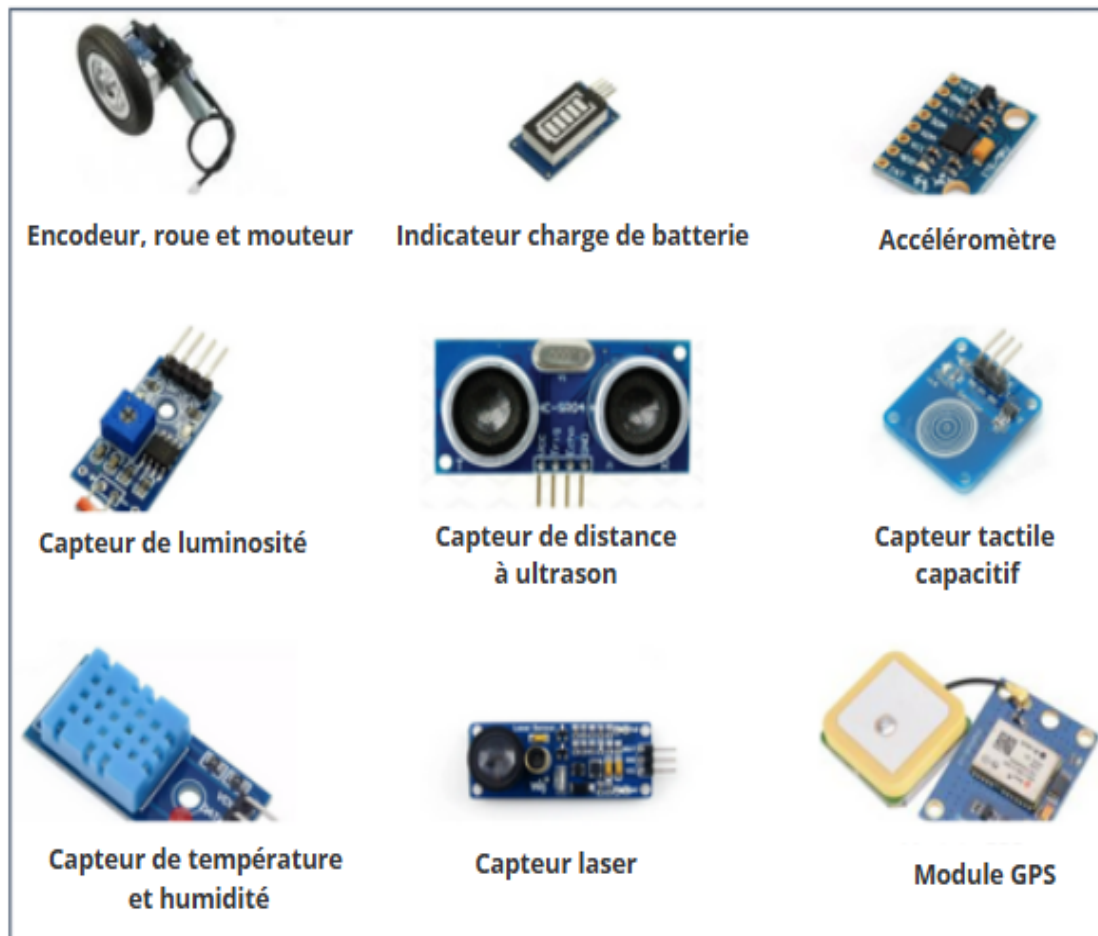


FIGURE 1.1 – Les capteurs clés des robots autonomes

Il y a deux grands types d'informations dont les robots mobiles ont besoin pour accomplir leurs tâches correctement :

- **Informations proprioceptives** : Les informations proprioceptives sont l'ensemble des données sur l'état interne du robot, par exemple la position et le mouvement.
 - **Les encodeurs de roue** : Les encodeurs de roue permettent de mesurer le déplacement, la vitesse et la direction de rotation de chaque roue en utilisant les impulsions mesurées par tour.
 - **Les capteurs de niveau de batterie** : Les capteurs de niveau de batterie existent en différents types pour mesurer l'état de la batterie. Les capteurs de tension mesurent la tension aux bornes de la batterie du robot, ce qui permet d'estimer le niveau d'énergie restant. D'autre part, les capteurs d'intensité de courant mesurent le courant entrant et sortant de la batterie, permettant ainsi de calculer la consommation d'énergie du robot en temps réel.
 - **Les accéléromètres et gyroscopes** : Les accéléromètres et les gyroscopes sont utilisés pour calculer la position, la vitesse, l'orientation et l'accélération angulaire et linéaire des robots mobiles.

- **L'odomètre** : L'odomètre est un capteur qui fournit des données précises sur la localisation et le déplacement du robot dans son environnement. Plus spécifiquement, l'odomètre permet de mesurer la position cartésienne du robot dans l'espace 3D (x, y, z), sa vitesse linéaire selon ses trois axes de déplacement (v_x, v_y, v_z), sa vitesse angulaire autour de ses trois axes de rotation (w_x, w_y, w_z), ainsi que son orientation complète exprimée sous la forme d'un quaternion (x, y, z, w).
- **Informations extéroceptives** : Les capteurs extéroceptifs permettent au robot d'acquérir des données sur l'environnement externe et les éléments qui l'entourent.
 - **Les capteurs de luminosité** : Les capteurs de luminosité permettent de détecter les changements d'éclairage, ce qui aide à contrôler l'éclairage du robot. Celui-ci peut allumer ou éteindre des leds en fonction de la luminosité de l'environnement.
 - **Les capteurs ultrasons** : Les capteurs à ultrasons permettent de détecter la présence d'objets à une distance d'au moins 2 cm à 4 mètres pour des portées faibles, et jusqu'à 10 cm pour les capteurs à grande portée, en utilisant des ondes sonores.
 - **Les capteurs tactiles** : Les capteurs tactiles permettent une interaction physique entre un robot et les humains. Ils donnent la capacité aux robots de sentir la pression qu'une personne applique sur eux et de réagir en conséquence.
 - **Les capteurs de température** : Les capteurs de température permettent de mesurer la température de l'air, la température de surface et la température d'immersion. Dans le contexte des robots, ils sont utilisés pour détecter d'éventuels dysfonctionnements de leurs composants ou pour identifier la présence d'une source de chaleur inhabituelle, comme un incendie, dans l'environnement où le robot évolue.
 - **Les capteurs laser et lidar** : Les capteurs LiDAR utilisent un laser pour mesurer des distances. Ils balayent l'environnement avec un faisceau laser et analysent le signal réfléchi pour créer une carte 3D détaillée de la scène. Les capteurs Laser permettent une détection fiable sur de grandes distances, même pour des petits objets en divers matériaux. Ces deux types de capteurs assurent une détection précise sur de longues portées, ce qui favorise une navigation autonome sûre.
 - **Le système de positionnement global (Global Positioning System (GPS))** : Le GPS, utilise un réseau de satellites pour fournir des informations de localisation précises aux robots mobiles.

1.2.1.2 Actionneurs

Les actionneurs sont des éléments de la partie opérative, qui lui permettent d'exécuter les actions que lui ordonne la partie commande [?]. Ils sont choisis en fonction des actions physiques qu'ils doivent effectuer.

- **Les moteurs électriques** : Les moteurs électriques sont essentiels à la mobilité des robots. Typiquement, chaque roue est équipée d'un moteur à courant continu, comme illustré dans la Figure 1.2 dessous. Ces moteurs sont choisis pour leur compacité, leur efficacité et leur facilité de contrôle.



FIGURE 1.2 – Moteur à courant continu

- **Les vérins électriques** : Les vérins électriques sont des dispositifs essentiels en robotique, convertissant le mouvement rotatif d'un moteur électrique en mouvement linéaire. Utilisant généralement un système de vis sans fin, de vis à billes ou de courroie, ils permettent de contrôler avec précision les mouvements répétitifs des différentes parties du robot, comme les bras ou les pinces.



FIGURE 1.3 – Vérin électrique

1.2.1.3 L'unité centrale de traitement (CPU)

L'unité centrale de traitement (Central Processing Unit, CPU) est responsable de l'exécution des instructions du programme ainsi que de la gestion des interactions entre les capteurs et les actionneurs. Parmi les CPU utilisés dans les robots mobiles, on trouve les modèles Intel Core i3, i5, i7, Celeron, ainsi que des architectures comme MCU, ARM, SoC, FPGA et DSP...

Ces processeurs traitent les données collectées par les capteurs et les stockent dans la mémoire vive.

- **Intel Core** : Les processeurs Intel Core offrent une excellente capacité de traitement parallèle grâce à leur architecture multi-cœurs, permettant aux robots de gérer simultanément plusieurs tâches comme le traitement d'images, la planification de trajectoires et le contrôle des actionneurs. Les technologies intégrées comme Intel Quick Sync Video et les instructions AVX (Advanced Vector Extensions) permettent un traitement rapide et efficace des flux vidéo et des calculs matriciels, essentiels pour la vision par ordinateur et les algorithmes d'apprentissage automatique dans les applications robotiques.



FIGURE 1.4 – Exemples des robots utilisant l’architecture Intel Core

- **MCU (STM32) :** Les MCU STM32 excellent dans le contrôle en temps réel des moteurs et des capteurs grâce à leurs périphériques dédiés comme les timers avancés et les convertisseurs analogique-numérique rapides, ce qui en fait un choix privilégié pour le contrôle précis des actionneurs robotiques. Il offre une consommation d’énergie très optimisée avec différents modes de fonctionnement basse consommation, tout en maintenant des performances suffisantes pour les tâches robotiques de base.



FIGURE 1.5 – Exemples des robots utilisant l’architecture MCU

- **ARM :** La gamme de processeurs ARM, allant des Cortex-M pour les systèmes embarqués légers aux Cortex-A pour les applications plus complexes, offre une extensibilité remarquable, tandis que leurs fonctionnalités de sécurité intégrées et leur support pour différents systèmes d’exploitation en temps réel en font une solution robuste pour le développement de robots modernes.



FIGURE 1.6 – Exemples des robots utilisant l’architecture ARM

- **SoC :** ces systèmes permettent de traiter simultanément les données des capteurs, d’exécuter les algorithmes de contrôle et de gérer les communications en temps réel, tout en minimisant la latence et la consommation énergétique, ce qui réduit non seulement

l'encombrement et le poids des robots, mais améliore aussi leur autonomie grâce à une gestion optimisée de l'énergie.



Robot Misty II



Robot Pepper



Robot Spot Mini

FIGURE 1.7 – Exemples des robots utilisant l'architecture SoC

- **FPGA** : L'architecture FPGA permet un traitement parallèle réel qui garantit une exécution simultanée et synchronisée des opérations, tandis que sa latence minimale assure une réactivité système optimale. Sa capacité d'adaptation matérielle permet une reconfiguration flexible selon les besoins spécifiques, complétée par une efficacité énergétique remarquable.



Robotnaut 2



Robot AIBO



Robot HexaPod

FIGURE 1.8 – Exemples des robots utilisant l'architecture FPGA

- **DSP** : Les DSP sont optimisés pour effectuer des calculs mathématiques complexes, tels que la transformation de Fourier, le filtrage et la modélisation des systèmes, avec une faible latence. Dans le domaine de la robotique, cette efficacité permet un traitement rapide des données des capteurs, ce qui est crucial pour des tâches telles que la reconnaissance d'objets, la navigation autonome et le contrôle des mouvements.



Robot NAO



Robot iCub



Robot Asimo

FIGURE 1.9 – Exemples des robots utilisant l'architecture DSP

- **Processeurs d'IA** : Les processeurs d'intelligence artificielle (IA) sont souvent basés sur des architectures de réseaux de neurones, ce qui leur permet de traiter rapidement et efficacement de vastes volumes de données. Cependant, il existe également d'autres types de

processeurs qui ne reposent pas sur ces réseaux, tels que les processeurs à base de règles et ceux basés sur des arbres décisionnels. Dans le contexte de la robotique, ces processeurs améliorent la capacité des robots à apprendre de leur environnement, à reconnaître des images et à prendre des décisions en temps réel. Grâce à leur puissance de calcul, ils facilitent des tâches complexes telles que la navigation autonome, la manipulation d'objets et l'interaction humaine, rendant ainsi les robots plus intelligents et adaptables à diverses situations. En intégrant des approches variées, y compris la logique floue et les algorithmes basés sur des règles, ces processeurs permettent aux robots d'exploiter une gamme plus large de techniques pour optimiser leurs performances.

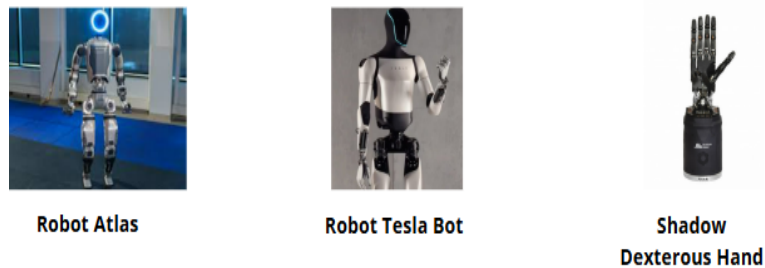


FIGURE 1.10 – Exemples des robots utilisant l'architecture AI Processors

1.2.1.4 Les sources d'énergie

Les robots mobiles ont besoin d'une source d'énergie embarquée pour fonctionner de manière indépendante.

- **Batteries électriques** : Les batteries électriques sont essentielles au fonctionnement des robots, avec plusieurs types adaptés aux besoins spécifiques de chaque application. tels que les batteries lithium-ion (Li-ion), les batteries nickel-hydrure métallique (NiMH) et les batteries modulaires.
- **Énergie solaire** : Les batteries à énergie solaire sont essentielles pour garantir une autonomie durable et écologique dans diverses applications robotiques. Par exemple, le chargeur solaire Husqvarna pour les tondeuses Automower permet une utilisation prolongée sans dépendre du réseau électrique, même par faible ensoleillement. De plus, des entreprises comme Naïo Technologies et Varta développent des solutions pour fournir une alimentation solaire aux robots agricoles, optimisant ainsi leur fonctionnement dans des environnements isolés.
- **Générateurs électriques** : Les robots mobiles qui ont besoin de plus d'énergie utilisent des générateurs électriques couplés à des moteurs à essence.
- **Les piles à combustible** : fonctionnent par le biais de réactions chimiques qui convertissent l'énergie chimique de combustibles, comme l'hydrogène, en énergie électrique, chaleur et eau. Elles sont utilisées dans la construction de robots autonomes, permettant à ces derniers de fonctionner sans émissions polluantes tout en bénéficiant d'une autonomie prolongée. Des entreprises comme Stäubli et Naïo Technologies intègrent des systèmes de piles à hydrogène dans leurs robots pour des applications variées, allant de l'agriculture à l'industrie. Ces technologies permettent non seulement d'améliorer l'efficacité énergétique des robots, mais aussi de répondre aux exigences environnementales croissantes, rendant les robots plus adaptés aux opérations dans des environnements sensibles.
- **Supercondensateurs** : Les supercondensateurs fournissent une source d'énergie efficace

pour des applications nécessitant des cycles de charge et de décharge rapides. Grâce à leur capacité à délivrer des pics de puissance élevés, ils sont idéaux pour alimenter des systèmes robotiques automatisés, améliorant ainsi la productivité et la sécurité au travail. Par exemple, dans les robots industriels, les supercondensateurs permettent de récupérer l'énergie lors des phases de freinage et de la restituer rapidement lors des mouvements, optimisant ainsi l'efficacité énergétique.

1.2.2 Frameworks et middlewares pour la robotique

Les logiciels robotiques constituent le "cerveau" des robots mobiles orchestrant leurs fonctions et leur comportement. Ces logiciels englobent une large gamme d'outils et de systèmes, allant des systèmes d'exploitation spécialisés aux applications de contrôle et d'intelligence artificielle (IA). Ils assurent des fonctions essentielles telles que le traitement des données des capteurs, la planification de mouvements, la prise de décision, et la coordination des différents composants du robot. Parmi les logiciels les plus couramment utilisés, on trouve Arduino pour la programmation de microcontrôleurs, Scratch pour l'initiation à la programmation robotique, List Processing (LisP) pour le développement d'IA, et Robot Operating System (ROS) comme framework complet pour le développement de robots.

1.2.2.1 Robot Operating System (ROS)

- **Définition :** ROS est un ensemble de bibliothèques et de technologies conçu pour faciliter le développement d'applications robotiques. Entièrement open source, ROS peut être installé sur plusieurs systèmes d'exploitation, bien que les performances optimales soient généralement observées sous Ubuntu et Debian. Il est compatible avec les langages de programmation C++ et Python.



FIGURE 1.11 – Logo de ROS

- **Historique :** ROS a été développé en 2007, sous le nom de switchyard par le Stanford Artificial Intelligence Laboratory (SAIL) dans le cadre du projet Stanford AI Robot (STAIR). Le développement de ROS a été motivé par les difficultés et défis rencontrés par les chercheurs, notamment Eric Berger, lors de la construction de robot de manipulation mobile (Personal Robot).

Le projet STAIR avait pour but de construire un robot capable de chercher et de récupérer des objets en réponse à une demande d'une personne. Pour cela, chaque équipe devait se concentrer sur l'une des tâches suivantes : reconnaissance vocale, navigation, manipulation et vision.

Les équipes devaient ensuite collaborer pour s'assurer que leurs composants fonctionnaient correctement sur le robot. Ils avaient besoin de développer des interfaces et des protocoles de communication pour que les composants puissent partager les données entre eux, ce qui permettrait au robot de prendre des décisions précises et rapides. Cependant, les chercheurs ont rencontré des difficultés lors de l'intégration de ces fonctionnalités, car chacun d'entre eux connaissait une seule fonctionnalité. Par exemple, "un excellent développeur de logiciels pourrait ne pas avoir les connaissances matérielles requises, tandis que quelqu'un développant une planification de trajectoire de pointe pourrait ne pas savoir comment faire la vision par ordinateur requise".

Selon les mots d'Eric Berger, ils cherchaient "quelque chose qui ne soit pas mauvais, dans toutes ces différentes dimensions" [9]. Cette nécessité de faciliter la collaboration et la réutilisation des composants a motivé la création d'un logiciel open source, facile à utiliser par tous, sans nécessiter de connaissances spécialisées. Le développement de ROS s'est poursuivi chez Willow Garage de 2008 à 2013, un laboratoire de recherche en robotique fondé fin 2006 par Scott Hassan, connu pour son robot Personal Robot 2 (PR2). Plusieurs versions internes de ROS ont été créées et testées sur ce robot. En 2013, Willow Garage a tenté de s'implanter sur le marché des robots de service commerciaux, mais ROS n'a pas réussi à s'imposer dans ce segment. Par conséquent, Willow Garage s'est scindée en plusieurs jeunes entreprises indépendantes. La gestion et le développement de ROS ont ensuite été transférés à la fondation Open Source Robotics Foundation (OSRF), qui a évolué en OpenRobotics en 2018 avant d'être intégrée en 2023 par Intrusion (filiale d'Alphabet, la maison mère de Google), tout en préservant son autonomie et son aspect open source. Aujourd'hui, ROS est largement adopté dans la communauté robotique pour plusieurs raisons importantes. Aujourd'hui, ROS est largement adopté dans la communauté robotique, et ce pour plusieurs raisons importantes :

Les avantages de ROS sont nombreux et significatifs pour la communauté robotique. Premièrement, sa nature open-source favorise la collaboration et le partage de connaissances entre chercheurs et développeurs du monde entier. De plus, son architecture modulaire permet une grande flexibilité, facilitant l'intégration de différents composants et technologies. D'autre part, ROS offre une vaste bibliothèque de packages préexistants, accélérant ainsi le développement de nouvelles applications robotiques.

1.2. Les robots mobiles

Enfin, sa compatibilité avec de nombreux langages de programmation et systèmes d'exploitation le rend accessible à un large éventail d'utilisateurs, des débutants aux experts. Ces caractéristiques font de ROS un outil puissant et polyvalent, essentiel pour l'avancement de la robotique moderne.

La Figure 1.7 résume par ordre chronologique les différentes versions du ROS

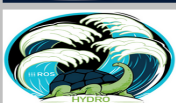
Distro	Release date	Poster	Turtle, turtle in tutorial	EOL date
ROS Noetic Ninjemys (Recommended)	May 23rd, 2020			May, 2025 (Focal EOL)
ROS Melodic Morenia	May 23rd, 2018			May, 2023 (Bionic EOL)
ROS Lunar Loggerhead	May 23rd, 2017			May, 2019
ROS Kinetic Kame	May 23rd, 2016			April, 2021 (Xenial EOL)
ROS Jade Turtle	May 23rd, 2015		Jade Turtle	May, 2017
ROS Indigo Igloo	July 22nd, 2014			April, 2019 (Trusty EOL)
ROS Hydro Medusa	September 4th, 2013			May, 2015
ROS Groovy Galapagos	December 31, 2012			July, 2014
ROS Fuerte Turtle	April 23, 2012			--
ROS Electric Emys	August 30, 2011			--
ROS Diamondback	March 2, 2011			--
ROS C Turtle	August 2, 2010			--
ROS Box Turtle	March 2, 2010			--

FIGURE 1.12 – Les versions du ROS

— Architecture

Les nœuds sont des unités fondamentales d'exécution dans l'architecture du ROS. Chaque nœud effectue un traitement spécifique dans l'architecture de ROS.

— **Le nœud maître** : joue un rôle central dans un système ROS. Il est responsable de gérer les communications entre les différents nœuds ou processus au sein de ce système. Son rôle consiste à donner à chaque nœud un nom unique comme un identifiant, également appelés "esclaves", et à assurer leur gestion. Le nœud maître coordonne l'ensemble de ces nœuds et permet leur interaction en publiant et en s'abonnant aux rubriques de messages appropriées. C'est la commande `roscore` qui est utilisée pour exécuter le nœud maître [21]. Les nœuds communiquent entre eux à travers le protocole XML-Remote Procedure Call (XMLRPC), qui est un protocole basé sur HTTP.

— **Nœud publisher** : Un nœud Publisher génère et publie des informations dans des topics, telles que celles qui peuvent provenir des capteurs ou des calculs.

— **Nœud subscriber** : Un nœud Subscriber s'abonne à des topics pour recevoir des informations publiées par les autres nœuds.

— **Nœud service** : Un nœud Service offre des services spécifiques pour les autres nœuds tel qu'un calcul, une requête de données....

— **Topic** : un topic est une voie de communication par laquelle les nœuds échangent des messages. Les nœuds publishers envoient des messages sur des topics, et les nœuds subscribers reçoivent des messages de ces topics.

— **Principe de fonctionnement** : ROS est un framework modulaire qui permet de construire des systèmes robotiques complexes en interconnectant des processus indépendants (nœuds) qui communiquent via des messages, des services et des paramètres partagés.

Le fonctionnement de ROS s'effectue de la manière suivante :

1- Lancement du nœud master en exécutant la commande `roscore` dans le terminal pour que ce nœud commence à écouter les connexions des autres nœuds.

2- Exécution de la commande `roslaunch my-package ultrason-publisher`, ROS lance le nœud publisher en exécutant le fichier de code source associé au nœud publisher dans le package « my-package »

3- Le nœud publisher se déclare auprès du nœud master, en envoyant son nom et ses informations de déclaration.

4- Le nœud master vérifie si le nom déclaré par le nœud publisher est déjà utilisé par un autre nœud. Si c'est le cas, il ajoute un suffix numérique au nom pour le rendre unique.

5- Le nœud master enregistre le nœud publisher avec son nom unique dans son registre des nœuds actifs.

6- Définition du topic : dans le fichier de code du nœud publisher, on définit le topic où les mesures seront publiées.

7- Le nœud subscriber se déclare auprès du nœud master en spécifiant le nom de la topic à laquelle il souhaite s'abonner.

8- Dans le code du nœud subscriber, il doit déclarer l'abonnement à la topic en util-

sant la méthode "subscribe" du nœud

9- Une fois que le nœud subscriber est abonné à la topic, il peut lire les messages publiés par le nœud publisher en utilisant la méthode "read " du nœud subscriber.

10- Le nœud master enregistre la communication entre le noeud publisher et le nœud subscriber, pour qu'ils puissent communiquer directement à l'avenir sans passer par le master.

La Figure 1.8 illustre et résume le fonctionnement du ROS



FIGURE 1.13 – Principe de fonctionnement du ROS

1.2.2.2 Yet Another Robot Platform (YARP)

YARP est une couche logicielle intermédiaire (middleware) développée par les italiens pour faciliter la communication entre les différents systèmes d'un robot humanoïde en utilisant des protocoles standard comme TCP/IP ou UDP. La première version de YARP (YARP Core) a été mise en place en 2000 sur le robot humanoïde Kismet illustré par la figure 1.9 [17]. Un système robotique typique comprend plusieurs composants tels qu'un système de vision, un système de mouvement et un système de contrôle. YARP agit comme un intermédiaire entre ces différentes parties, permettant le passage d'informations de l'une à l'autre. Bien qu'utilisable pour divers types de robots, YARP est particulièrement adapté aux robots mobiles.

L'objectif principal des développeurs de YARP est d'augmenter la durée de vie des projets logiciels robotiques en évitant d'avoir à les réécrire ou les mettre à jour fréquemment, permettant ainsi aux développeurs de se concentrer sur d'autres aspects de leurs projets robotiques tout en minimisant les coûts de maintenance.

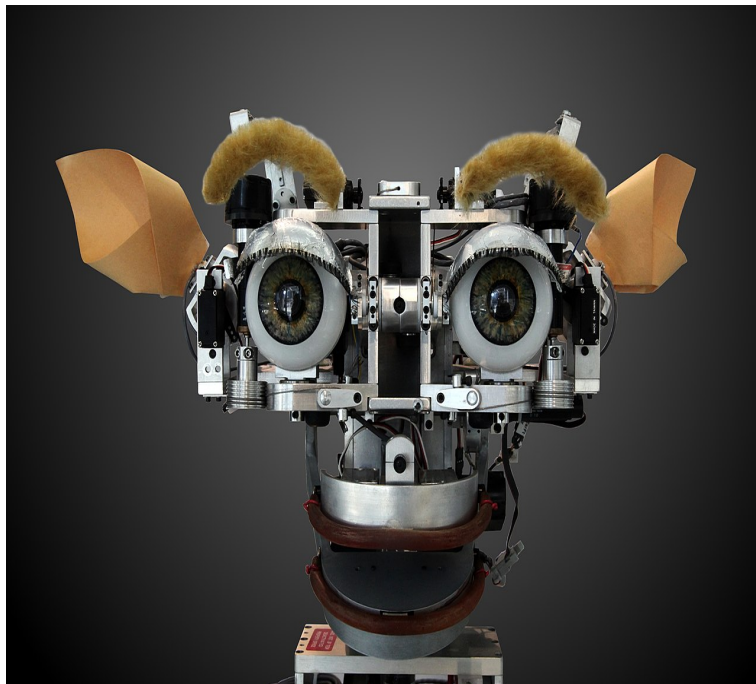


FIGURE 1.14 – Robot Humanoïde Kismet

1.2.2.3 MoveIt

MoveIt est un cadre de planification de mouvements largement utilisé pour la manipulation robotique. Il est intégré à ROS (Robot Operating System) et permet de gérer des tâches complexes telles que la planification de trajectoires, la détection de collisions, et l'interaction avec des capteurs. MoveIt prend en charge des robots à haute mobilité et permet aux utilisateurs de générer des trajectoires optimisées en évitant les obstacles, tout en s'occupant de la cinématique et de la dynamique des robots. Il est également conçu pour être facilement extensible grâce à son interface avec diverses bibliothèques de planification de mouvements, notamment OMPL (Open Motion Planning Library).

1.3 Navigation autonome

Le processus de navigation pour un robot mobile autonome consiste à identifier un parcours réalisable au sein de son environnement opérationnel, en tenant compte des contraintes spatiales imposées par les obstacles à proximité. Cet environnement, qualifié d'espace de configuration libre, regroupe l'ensemble des positions et orientations que peut adopter le robot sans entrer en collision. La tâche de navigation implique donc la détermination de la séquence de déplacements permettant au robot de se rendre d'une configuration de départ donnée vers une configuration d'arrivée souhaitée, tout en demeurant dans les limites de l'espace de configuration accessible.

1.3. Navigation autonome

Pour naviguer de façon efficace, le robot doit effectuer les tâches suivantes illustrées par la Figure 1.10 [2] :

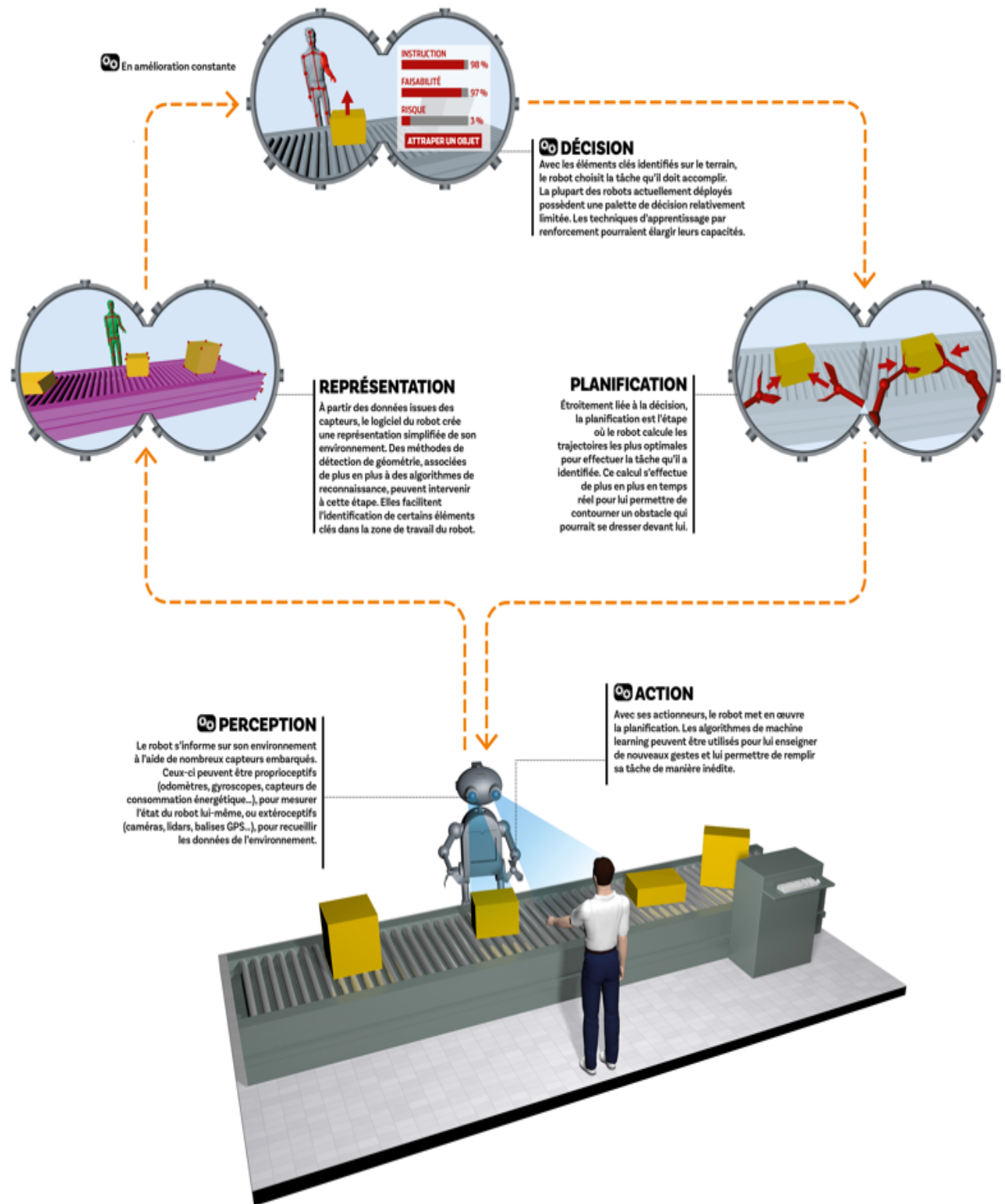


FIGURE 1.15 – Boucle de navigation autonome

1.3.1 Perception de l'environnement

Pour se déplacer efficacement dans son environnement, un robot mobile doit disposer de capacités de perception avancées. Cette compréhension de l'espace qui l'entoure repose principalement sur l'exploitation de données visuelles tridimensionnelles captées par ses caméras. Ces données relatives aux objets, zones dégagées et mouvements détectés permettent de modéliser numériquement l'environnement du robot. Si le GPS reste fréquemment utilisé pour la localisation, d'autres capteurs entrent en jeu.

On distingue les capteurs actifs, émettant des ondes analysées après réflexion (télémètres laser, radars, sonars), et les capteurs passifs fonctionnant par simple observation. L'approche vision, proche de la perception humaine, semble particulièrement prometteuse. Deux grands types de localisation sont possibles : qualitative avec des informations de position approximatives, ou quantitative fournissant des coordonnées précises. Différentes méthodes de localisation qualitative ont été explorées, comme celles de Morita et al. [7], tandis que le problème du "robot kidnappé" a notamment été traité par Thrun et al. dans [8].

1.3.1.1 Technologies de Perception Moderne

1. **Capteurs 3D Avancés [23]** : La technologie LiDAR (Light Detection and Ranging) connaît une évolution révolutionnaire dans le domaine de la robotique mobile autonome. Selon Zhang et Singh (2023), les récentes innovations en matière de systèmes LiDAR ont permis d'atteindre des performances sans précédent en termes de précision, de fiabilité et d'efficacité énergétique. Les auteurs soulignent particulièrement l'émergence des LiDAR solid-state qui, contrairement aux systèmes mécaniques traditionnels, utilisent une technologie de balayage électronique sans pièces mobiles. Cette avancée permet non seulement de réduire la taille des dispositifs de 40% en moyenne, mais aussi d'augmenter leur durée de vie jusqu'à 50 000 heures de fonctionnement continu, tout en diminuant leur consommation énergétique de 60% . Les systèmes LiDAR multi-faisceaux représentent une autre innovation majeure, offrant une couverture panoramique à 360 degrés avec une densité de points pouvant atteindre 2,6 millions de points par seconde et une résolution angulaire de 0,1°. Cette haute résolution, couplée à une fréquence de rafraîchissement de 20 Hz, permet une reconstruction tridimensionnelle en temps réel de l'environnement avec une précision sans précédent. Zhang et Singh démontrent également les capacités impressionnantes des LiDAR à longue portée, capables de détecter et mesurer des objets jusqu'à 200 mètres avec une précision de $\pm 2\text{mm}$ dans des conditions optimales. Ces performances sont obtenues grâce à l'utilisation de lasers de classe 1 (sécurité oculaire) opérant dans la bande spectrale 905-940nm, permettant une meilleure pénétration atmosphérique et une résistance accrue aux interférences lumineuses. Les auteurs soulignent également l'importance de la fusion des données LiDAR avec d'autres capteurs, notamment les caméras et les IMU, permettant d'atteindre une précision de localisation inférieure à 5cm dans des environnements urbains complexes. Cette synergie multi-capteurs, combinée aux algorithmes de traitement avancés basés sur l'apprentissage profond, permet une détection et une classification des objets en temps réel avec un taux de précision dépassant 95% pour les objets statiques et 85% pour les objets en mouvement.
2. **Systèmes de Vision** : L'évolution des systèmes RGB-D représente une avancée majeure dans le domaine de la perception robotique, notamment grâce à l'intégration d'architectures de réseaux neuronaux sophistiquées. Les travaux de Wang et al. (2024) mettent en lumière deux approches principales de traitement : les réseaux de fusion précoce et les architectures multi-flux. Les réseaux de fusion précoce, traitant simultanément les quatre

canaux (R,G,B,D) dès les premières couches, démontrent une amélioration de la précision de 23% tout en réduisant la consommation mémoire de 35%. Les architectures multi-flux, considérées comme l'état de l'art actuel, utilisent des branches parallèles pour le traitement RGB et profondeur, enrichies par des connexions résiduelles qui améliorent la robustesse de 45%. Cette architecture est complétée par trois méthodes de fusion innovantes : la fusion par attention spatiale, augmentant la précision de reconstruction 3D de 34% ; la fusion par apprentissage des caractéristiques, réduisant le bruit de mesure de 42% ; et la fusion temporelle, diminuant les artefacts de mouvement de 56%. Ces avancées sont rendues possibles grâce à des systèmes de calibration automatique atteignant une précision de 0,1 pixels et une synchronisation matérielle sub-milliseconde. Les applications pratiques de ces technologies en robotique mobile ont permis d'atteindre des performances remarquables, notamment une détection d'obstacles avec une précision de 99,3% et une estimation de pose d'objets sub-millimétrique. Toutefois, comme le soulignent les auteurs, des défis persistent, particulièrement en conditions de forte luminosité et en termes de consommation énergétique des processeurs neuromorphiques. Les directions futures de recherche se concentrent sur le développement de nouveaux émetteurs IR, l'optimisation des réseaux neuronaux pour le edge computing, et l'amélioration des techniques de fusion multi-capteurs pour une perception robotique encore plus robuste et précise.

3. **Capteurs Complémentaires** : Les capteurs complémentaires apportent des capacités sensorielles diversifiées et redondantes à la robotique. L'IMU (Inertial Measurement Unit), composé d'accéléromètres, de gyroscopes et de magnétomètres, travaille en synergie avec les systèmes de vision pour assurer une odométrie précise, même lors de perturbations visuelles temporaires. Cette fusion de données permet une navigation plus robuste et une meilleure estimation des mouvements du robot. Les capteurs tactiles, qu'ils soient de type force, capacitif ou simple bumper, enrichissent les capacités d'interaction du robot avec son environnement, permettant une navigation sécurisée dans des espaces encombrés et une détection fine des contacts physiques. Enfin, les radars millimétriques, opérant sur des fréquences entre 30 et 300 GHz, excellent dans des conditions météorologiques difficiles où d'autres capteurs montrent leurs limites. Leur capacité à mesurer directement la vitesse par effet Doppler et à détecter des obstacles jusqu'à 300 mètres les rend particulièrement précieux pour les applications extérieures et les véhicules autonomes. Cette combinaison de capteurs crée une redondance sensorielle essentielle, garantissant une navigation fiable et sûre dans des environnements variés et dynamiques.

1.3.1.2 Techniques de traitement des données

1. **Fusion Multi-Capteurs [4]** : Dans leur étude fondamentale publiée dans *Robotics and Autonomous Systems*, Chen et al. (2023) présentent une analyse approfondie des approches modernes de fusion multi-capteurs en robotique. Les auteurs mettent en lumière l'importance du filtrage de Kalman adaptatif, une technique sophistiquée qui permet d'optimiser dynamiquement les poids attribués aux différents capteurs en fonction de leur fiabilité et des conditions environnementales. Cette approche est complétée par l'utilisation innovante de réseaux de neurones spécialement conçus pour la fusion de données, capables d'apprendre et d'exploiter les corrélations complexes entre les différents capteurs. Un aspect particulièrement novateur de leur travail réside dans l'intégration de la dimension temporelle, où l'historique des mesures est pris en compte pour améliorer la précision et la robustesse des estimations. Les chercheurs démontrent comment cette approche tridimensionnelle - combinant filtrage adaptatif, réseaux de neurones et fusion temporelle - permet

d'obtenir des résultats significativement plus précis que les méthodes traditionnelles de fusion, particulièrement dans des environnements dynamiques et complexes où la fiabilité des capteurs peut varier considérablement au fil du temps

2. **Intelligence Artificielle pour la Perception [15]** : Les travaux de Liu et al. (2024), présentés à la Conference on Robot Learning (CoRL), marquent une avancée significative dans l'application de l'intelligence artificielle à la perception robotique. Leur contribution majeure, l'architecture PerceptionFormer, représente une innovation importante dans l'utilisation des transformers pour la fusion multi-modale en robotique. Cette architecture s'appuie sur des réseaux de neurones auto-attentifs sophistiqués qui permettent une compréhension contextuelle approfondie de l'environnement, en analysant simultanément les relations complexes entre différentes modalités de perception. Un aspect particulièrement remarquable de leur approche est l'intégration de mécanismes d'apprentissage par transfert, permettant aux systèmes robotiques de s'adapter rapidement à de nouveaux environnements sans nécessiter un réentraînement complet. Les auteurs démontrent comment leur architecture surpasse les approches traditionnelles en termes de précision et de rapidité d'adaptation, notamment dans des scénarios complexes où la compréhension contextuelle est cruciale. Leurs résultats expérimentaux montrent une amélioration significative de la robustesse et de la généralisation des capacités perceptives des robots, ouvrant ainsi la voie à des applications plus avancées dans des environnements dynamiques et non structurés.

1.3.2 Localisation Avancée

1.3.2.1 Méthodes Quantitatives [5]

Le SLAM (Simultaneous Localization and Mapping) a connu des avancées significatives ces dernières années, notamment dans trois directions principales. Tout d'abord, le SLAM visuel-inertiel représente une innovation majeure en fusionnant intelligemment les données des caméras avec celles des capteurs inertiels (IMU). Cette approche permet non seulement une localisation plus précise mais aussi une meilleure résistance aux conditions difficiles comme les mouvements brusques ou les environnements faiblement éclairés. En parallèle, le SLAM sémantique a introduit une dimension nouvelle en intégrant la compréhension de scène dans le processus de cartographie. Cette évolution permet aux systèmes de reconnaître et classer les objets et structures de l'environnement, créant ainsi des cartes enrichies qui ne se limitent plus à la simple géométrie mais incluent également le contexte et la fonction des éléments cartographiés. Enfin, le SLAM collaboratif a ouvert la voie à des applications multi-robots où plusieurs agents partagent leurs observations et construisent collectivement une représentation cohérente de l'environnement. Cette approche distribuée améliore non seulement la rapidité et l'efficacité de la cartographie mais permet aussi une meilleure gestion des zones complexes ou étendues. Ces trois axes de développement convergent vers un SLAM plus intelligent, plus robuste et mieux adapté aux exigences des applications robotiques modernes, que ce soit dans l'industrie, l'exploration ou les services.

1.3.2.2 Méthodes Qualitatives [20]

Les méthodes qualitatives de navigation robotique, particulièrement celles basées sur l'apprentissage, ont révolutionné la manière dont les robots interagissent avec leur environnement. La navigation par points de repère visuels constitue une approche fondamentale où le robot apprend à identifier et à utiliser des éléments distinctifs de l'environnement comme des portes, des panneaux ou des objets caractéristiques pour se repérer et planifier ses déplacements. Cette mé-

thode s'enrichit de la cartographie topologique adaptative, qui permet au robot de construire et de mettre à jour dynamiquement une représentation abstraite de l'espace sous forme de graphe, où les nœuds représentent des lieux significatifs et les arêtes les chemins possibles entre ces lieux. Cette représentation s'adapte continuellement aux changements de l'environnement et à l'expérience accumulée par le robot. La reconnaissance de lieux basée sur le deep learning vient compléter ces approches en permettant une identification robuste et contextuelle des espaces, même dans des conditions variables d'éclairage ou de point de vue. Les réseaux de neurones profonds sont entraînés sur de vastes ensembles de données pour reconnaître les caractéristiques distinctives des différents types d'espaces, permettant ainsi une navigation plus intuitive et plus proche de la compréhension humaine de l'environnement. L'intégration de ces trois aspects crée un système de navigation qualitative sophistiqué, capable d'apprendre et de s'adapter continuellement aux défis complexes de la navigation en environnement réel.

1.3.3 Algorithmes de planification de trajectoire

La planification de trajectoire [11] pour un robot mobile vise à déterminer un chemin sécurisé et optimal entre une position initiale P_i et une position finale P_f dans un environnement complexe, tout en évitant les collisions. Cela nécessite de prendre en compte les dynamiques du robot, comme sa vitesse maximale et son rayon de braquage, ainsi que les obstacles statiques et dynamiques qui l'entourent. Un aspect clé de cette planification est l'utilisation d'algorithmes adaptés, parmi lesquels l'approche de la fenêtre dynamique (ou "dynamic window approach" en anglais). Cet algorithme évalue les vitesses possibles du robot en fonction de ses contraintes dynamiques et des informations sur l'environnement, permettant ainsi une navigation fluide et réactive face aux changements. La planification peut être globale, utilisant des graphes et des techniques comme A^* ou Dijkstra, ou locale, se basant sur des données en temps réel pour ajuster l'itinéraire. Cependant, des défis subsistent, notamment la gestion des minimums locaux et l'optimisation des mouvements en présence d'obstacles mobiles. Les avancées récentes, telles que l'approche d'obstacles de vitesse, offrent des solutions innovantes pour contourner ces limitations.

1.4 Sécurité et fiabilité

La prise en compte des aspects de sécurité est primordiale lors de la conception et de l'exploitation des robots mobiles autonomes. Ces systèmes complexes, amenés à évoluer dans des environnements dynamiques et parfois imprévisibles, sont susceptibles de rencontrer diverses situations à risque pouvant entraîner des dysfonctionnements ou des dommages. Dès lors, il est indispensable d'intégrer, dès les premières étapes de développement, des mécanismes robustes visant à prévenir, détecter et gérer de manière anticipative les éventuelles défaillances. Ces dispositifs de sécurité doivent permettre d'identifier rapidement les anomalies, qu'elles soient d'origine mécanique, électronique ou logicielle, et de déclencher des actions correctives appropriées afin de préserver l'intégrité du robot et de son environnement. Une approche globale et rigoureuse en matière de sûreté de fonctionnement est ainsi essentielle pour assurer la fiabilité, la résilience et la sécurité opérationnelle des robots mobiles, quel que soit le contexte d'utilisation. Il existe des méthodes importantes pour assurer la fiabilité et la sécurité de robots mobiles autonomes :

- **Systèmes de sécurité et d'arrêt d'urgence** : Les robots mobiles autonomes intègrent des systèmes de sécurité essentiels pour prévenir les accidents. Ceux-ci comprennent des boutons d'arrêt d'urgence, des capteurs de collision, et des limites logicielles et matérielles pour restreindre les mouvements dangereux. Des protocoles automatiques sont implémentés pour

détecter et réagir aux anomalies, assurant ainsi une protection continue du robot et de son environnement. La redondance des systèmes critiques et la surveillance en temps réel renforcent la sécurité.

- **Test et validation approfondis :** La fiabilité des robots mobiles autonomes est assurée par des tests rigoureux. Ceux-ci incluent des tests unitaires pour chaque composant, des tests d'intégration, et des simulations virtuelles pour évaluer le comportement du robot dans divers scénarios. Des essais sur le terrain sont menés dans des environnements contrôlés puis réels. Un processus de validation continue tout au long du cycle de vie du robot permet des ajustements basés sur l'expérience opérationnelle.
- **Formation et sensibilisation des utilisateurs :** La sécurité des robots mobiles autonomes dépend aussi de la bonne formation des utilisateurs. Des manuels clairs et des programmes de formation complets sont élaborés, couvrant l'utilisation normale et les procédures d'urgence. Des sessions de sensibilisation aux risques et aux bonnes pratiques sont organisées. Un support technique est mis à disposition, et la formation est régulièrement mise à jour pour intégrer les évolutions du robot et les retours d'expérience.

1.5 Principes éthiques

L'éthique a une importance cruciale dans la conception et le développement des robots mobiles autonomes. Ces systèmes intelligents, amenés à interagir avec des humains et à évoluer dans des environnements complexes, doivent se conformer à un ensemble de valeurs et de normes déontologiques afin de garantir leur fonctionnement responsable et bienveillant.

Parmi les principes fondamentaux à respecter, on peut citer la redevabilité, qui implique que les robots soient en mesure de justifier leurs actions et leurs décisions, d'en assumer la responsabilité et d'ajuster leurs comportements en conséquence. La transparence est également essentielle, exigeant que les processus décisionnels et les raisonnements sous-jacents des robots soient explicables et compréhensibles pour les utilisateurs. Bien que dotés d'une certaine autonomie, ces systèmes ne doivent pas être totalement indépendants, mais rester sous le contrôle et la supervision humaine. Leur intégration harmonieuse au sein des environnements opérationnels est primordiale, nécessitant une capacité à interagir de manière cohérente avec les autres systèmes et entités présents. Des considérations de sécurité sont également indispensables, les robots doivent être conçus pour détecter et surmonter les menaces potentielles, tout en protégeant l'intégrité des utilisateurs et des infrastructures environnantes. Au-delà de ces aspects fonctionnels, une dimension éthique profonde doit guider les comportements des robots mobiles, les amenant à prendre des décisions conformes aux valeurs morales et sociétales. La flexibilité et l'adaptabilité sont aussi des qualités requises, permettant aux robots de moduler leurs actions en fonction des changements contextuels. Enfin, ces systèmes doivent disposer de capacités d'apprentissage leur permettant d'évoluer et de s'améliorer au fil de leurs expériences, tout en respectant le cadre éthique défini.

1.6 Tendances Émergentes

1.6.1 Perception Bio-inspirée

Les approches de perception bio-inspirée [22] en robotique mobile représentent une avancée significative dans le domaine de la navigation autonome, s'appuyant sur des mécanismes observés dans la nature. Les systèmes de vision événementielle, inspirés du fonctionnement

de la rétine humaine, offrent une nouvelle approche du traitement visuel en ne captant que les changements d'intensité lumineuse dans la scène, ce qui permet une réduction drastique de la quantité de données à traiter tout en maintenant une excellente réactivité aux mouvements. Cette technologie est complétée par la navigation basée sur le flux optique, qui s'inspire directement du comportement des insectes volants, en particulier les abeilles, qui utilisent le mouvement apparent des objets dans leur champ visuel pour estimer leur propre déplacement et éviter les obstacles. Cette méthode permet une navigation fluide et naturelle, particulièrement efficace dans des environnements dynamiques. L'apprentissage par imitation de comportements biologiques vient enrichir ces approches en permettant aux robots d'acquérir des stratégies de navigation complexes en observant et en reproduisant les comportements d'organismes vivants, qu'il s'agisse de la manière dont les fourmis retrouvent leur chemin ou dont les oiseaux naviguent sur de longues distances. Cette combinaison d'approches bio-inspirées permet de développer des systèmes de navigation plus robustes, plus efficaces énergétiquement et mieux adaptés aux défis des environnements réels, tout en ouvrant la voie à une nouvelle génération de robots capables d'interagir de manière plus naturelle avec leur environnement.

1.6.2 Edge Computing pour la Perception

L'intégration de l'Edge Computing dans les systèmes de perception robotique représente une évolution majeure pour l'autonomie et l'efficacité des robots mobiles. Les processeurs neuromorphiques, qui imitent l'architecture et le fonctionnement du cerveau biologique, constituent une avancée fondamentale dans ce domaine en permettant le traitement rapide et efficace des informations sensorielles directement sur le robot. Ces processeurs, organisés en réseaux de neurones artificiels matériels, excellent particulièrement dans le traitement parallèle des données de perception et la prise de décision en temps réel, tout en consommant significativement moins d'énergie que les architectures traditionnelles. Cette approche est renforcée par l'accélération matérielle des algorithmes de perception, qui permet l'optimisation des calculs complexes nécessaires à la navigation autonome directement au niveau du hardware. Les FPGA (Field-Programmable Gate Arrays) et les ASIC (Application-Specific Integrated Circuits) dédiés à la perception permettent d'exécuter efficacement les algorithmes de traitement d'images, de détection d'obstacles et de localisation, réduisant ainsi la latence et augmentant la réactivité du système. L'optimisation énergétique des systèmes de perception constitue le troisième pilier de cette approche, avec des stratégies innovantes de gestion de l'alimentation qui adaptent dynamiquement la consommation énergétique en fonction des besoins de perception du robot. Cette optimisation inclut des mécanismes de mise en veille sélective des capteurs, des stratégies de calcul adaptatif et des architectures matérielles à basse consommation, permettant d'augmenter significativement l'autonomie des robots tout en maintenant leurs capacités de perception et de navigation.

1.7 Conclusion

Les robots mobiles autonomes sont des systèmes très avancés qui demandent beaucoup de travail et d'attention. Leur création nécessite non seulement des progrès techniques importants, comme des composants performants, des capacités de déplacement intelligentes et des dispositifs de sécurité fiables, mais aussi le respect de règles de base essentielles. Au-delà des aspects purement fonctionnels, il est crucial que ces robots soient conçus pour agir de manière éthique et bienveillante envers l'humanité. Parmi ces aspects techniques essentiels, leurs capacités de déplacement intelligentes jouent un rôle central. Le chapitre suivant, « Algorithmes Intelligents

1.7. Conclusion

pour la mobilité autonome », se penchera sur ces algorithmes sophistiqués qui permettent aux robots de naviguer de manière autonome dans des environnements complexes, constituant ainsi le cœur de leur intelligence mobile.

Algorithmes intelligents pour la mobilité autonome

Sommaire

2.1	Introduction	26
2.2	Apprentissage automatique	26
2.2.1	Apprentissage supervisé	27
2.2.2	Apprentissage non supervisé	29
2.2.3	Apprentissage par renforcement	29
2.2.4	Apprentissage profond	31
2.2.5	Systèmes experts	32
2.2.6	Logique Floue	32
2.2.7	Algorithmes génétique	33
2.2.8	Traitement du langage naturel (NLP)	33
2.3	État de l'art	35
2.3.1	Q-learning et navigation autonome	35
2.3.2	Intégration de l'IoT et des techniques de détection d'objets	35
2.3.3	Environnements d'apprentissage et défis	35
2.3.4	Gestion de la mémoire et traitement en temps réel	36
2.4	Apprentissage machine pour la perception de l'environnement	36
2.5	Avantages de l'intelligence artificielle et de la robotique	37
2.6	Conclusion	37

2.1 Introduction

La navigation autonome des robots mobiles est un domaine de recherche dynamique et interdisciplinaire qui englobe des techniques variées telles que l'apprentissage automatique, la vision par ordinateur, et les systèmes de contrôle. Avec l'essor de l'Internet des objets (IoT), ces robots doivent non seulement naviguer dans des environnements physiques, mais aussi interagir avec des systèmes connectés pour optimiser leurs opérations. Des algorithmes intelligents, notamment le Q-learning et les réseaux de neurones convolutifs (CNN), sont fondamentaux pour améliorer les performances des robots, leur permettant de s'adapter aux variations de leur environnement, de prendre des décisions en temps réel, et de réaliser des tâches complexes de manière autonome. Ce chapitre explore en détail ces techniques au cœur de l'autonomie robotique. Nous débuterons par un aperçu des trois grandes familles d'algorithmes d'apprentissage : supervisé, non supervisé et par renforcement. L'accent sera ensuite mis sur l'apprentissage profond, particulièrement puissant pour la mobilité autonome. Nous examinerons le rôle clé de l'apprentissage machine dans la perception de l'environnement, avec des applications en vision artificielle et en localisation. Enfin, nous nous concentrons sur les avantages apportés par l'IA et la robotique. Ce chapitre établit les fondements algorithmiques qui rendent possible la mobilité robotique entièrement autonome, illustrant la progression des techniques d'IA.

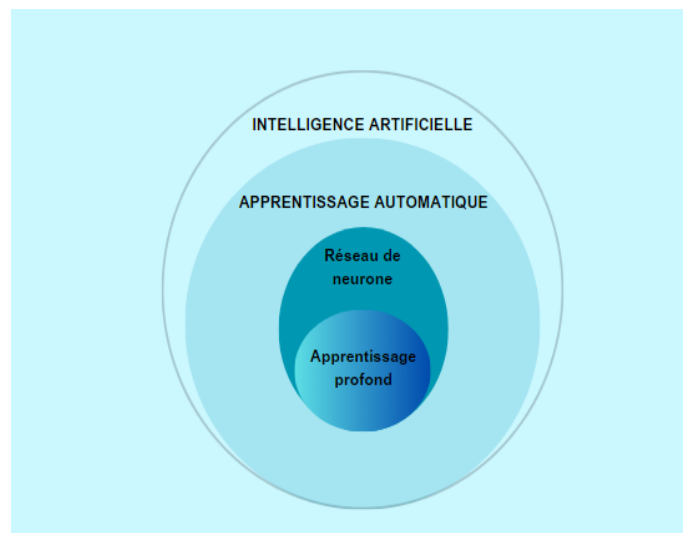


FIGURE 2.1 – Méthodes d'intelligence artificielle

2.2 Apprentissage automatique

Les systèmes d'IA utilisent des algorithmes pour prévoir et classer des données collectées à partir de bases de données informatiques. Il existe plusieurs types d'apprentissage, notamment :

- Apprentissage supervisé
- Apprentissage non supervisé
- Apprentissage par renforcement
- Apprentissage profond
- Systèmes experts
- Logique floue
- Algorithmes génétiques

— Traitement du langage naturel(NLP)

2.2.1 Apprentissage supervisé

L'apprentissage supervisé [14] est une approche fondamentale en IA où un système apprend à partir d'un ensemble de données structurées. Dans ce processus, on fournit à la machine un ensemble d'entrées (inputs) associées à leurs sorties (outputs) correspondantes, généralement étiquetées par un expert humain. L'objectif est de déterminer une fonction f , paramétrée par un ensemble de variables, qui établit une relation précise entre les entrées et les sorties. Le rôle de la machine est d'ajuster ces paramètres pour optimiser la fonction, minimisant ainsi l'écart entre les prédictions et les vraies valeurs. Cette approche se base principalement sur deux approches : la classification, où la sortie est une classe discrète, et la régression, où la sortie est une valeur continue. L'apprentissage supervisé réside dans sa capacité à généraliser à partir des exemples fournis, permettant au système de produire des prédictions pertinentes pour de nouvelles données jamais rencontrées auparavant, déduisant ainsi une règle générale applicable à un large éventail de situations.

— **Régression linéaire :**

L'objectif de la régression linéaire est d'entraîner un modèle prédictif capable d'inférer une cible, ou variable dépendante, à partir d'un ensemble de caractéristiques, ou variables indépendantes. Le processus de modélisation par régression linéaire vise à déterminer l'équation de la droite qui approche le mieux la tendance globale présente dans un ensemble de points de données (Figure 2.2). Le but est de transformer la masse de points en une ligne la plus probable.

$$y = mx + p \quad (2.1)$$

où y représente la variable à expliquer, x la variable explicative, et m et p les paramètres à estimer, à savoir respectivement la pente et l'ordonnée à l'origine.

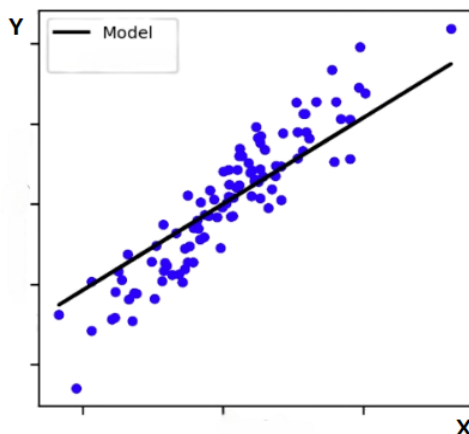


FIGURE 2.2 – Régression Linéaire

- **Régression linéaire simple :** une seule variable indépendante est utilisé pour estimer une variable dépendante.
- **Régression linéaire multiple :** plus qu'une variable indépendante est présente.
- **La classification :** La classification est une technique d'apprentissage automatique où le résultat obtenu est une valeur discrète. Dans ce type de modèle, la variable à prédire peut prendre une valeur parmi un ensemble fini de possibilités. L'objectif principal de la classification est de prédire la classe ou la catégorie à laquelle appartient une nouvelle donnée.

Il existe deux types de classification qui sont illustrés par la Figure 2.3 :

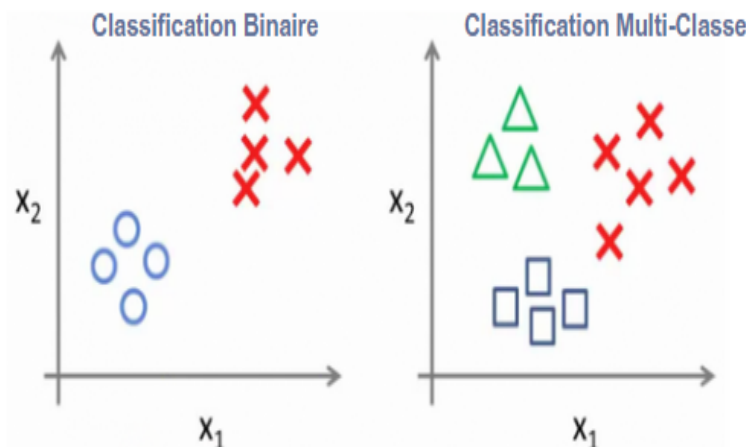


FIGURE 2.3 – Les types de classification

- **Classification binaire** : concerne deux classes possibles telles que la classification Machine à Vecteurs de Support (SVM), le perceptron et les réseaux de neurones.
- **Machine à Vecteurs de Support (SVM)** : Cet algorithme est considéré comme un classificateur linéaire. Il sépare les données en utilisant des hyperplans et maximise les distances entre les échantillons et les hyperplans. SVM est souvent utilisé dans des applications telles que la finance pour comparer les performances actuelles et futures et les retours sur investissement.
- **Perceptron** : Le perceptron est un algorithme de classification binaire inventé en 1957 par Frank Rosenblatt. Il compare la somme de plusieurs signaux d'entrée pour décider à quelle catégorie un nouvel exemple appartient. Il est utilisé dans des applications variées, comme la séparation de classes dans des données (par exemple, classer des emails comme "spam" ou "non-spam"), la reconnaissance de motifs en vision par ordinateur, et l'analyse de sentiment dans le traitement du langage naturel.
- **Classification multi-classes** : concerne plus de deux catégories, nous citons par exemple le k-plus proches voisins (kNN) et les Réseau de Neurones Artificiels (RNA).
- **Réseau de Neurones Artificiels (RNA)** : Les RNA sont des modèles qui imitent le fonctionnement des neurones biologiques pour apprendre à classer les données. Ils peuvent être utilisés pour des problèmes de classification et de régression. Pour la classification binaire, le RNA aura un seul neurone de sortie qui prendra une valeur binaire, par exemple 0 ou 1, oui ou non, vrai ou faux. Pour la classification multi-classe, le RNA aura plusieurs neurones de sortie, chacun représentant une classe différente. Le nombre de neurones de sortie correspondra au nombre de classes à prévoir. La classe prédite sera celle avec la valeur de sortie la plus élevée.
- **K Plus Proches Voisins (kNN)** : Le kNN est utilisé pour classer des données en fonction de la distance entre les données d'entrée et celles de l'ensemble d'entraînement. Lorsqu'une nouvelle donnée est à classer, l'algorithme identifie les k voisins les plus proches et détermine sa classe en fonction de ceux-ci. Cette méthode est populaire dans des domaines tels que la reconnaissance d'images, les systèmes de recommandation, la détection d'anomalies et l'analyse de texte, en raison de sa simplicité et de son efficacité.

2.2.2 Apprentissage non supervisé

L'apprentissage non supervisé [12] consiste à fournir à la machine un ensemble de données d'entrée non étiquetées, sans les sorties correspondantes. Son but est d'identifier les structures et les motifs (patterns) dans les données sans avoir de guide ou de référence (sans l'intervention d'un expert). Ce type d'apprentissage vise à organiser et à comprendre les données de manière autonome.

Il existe plusieurs algorithmes d'apprentissage non supervisé

- **Le clustering** : Le clustering est une méthode d'exploration de données non supervisée qui vise à rassembler des informations non catégorisées en se basant sur leurs ressemblances ou disparités. Cette technique permet d'organiser et de structurer des ensembles de données brutes en identifiant des motifs ou des caractéristiques communes, sans nécessiter d'étiquettes prédéfinies. Le clustering est utilisé dans divers domaines, comme le marketing pour la segmentation de la clientèle, afin de cibler des groupes spécifiques avec des campagnes publicitaires adaptées, et en biologie pour la classification de gènes ou d'espèces selon leurs caractéristiques, ce qui aide à étudier les relations évolutives.

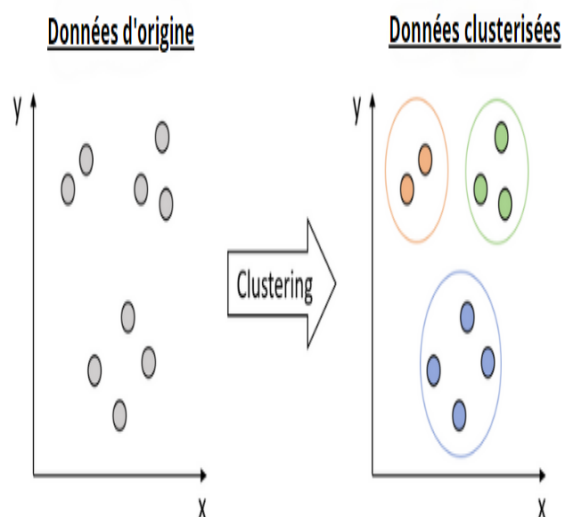


FIGURE 2.4 – Clustering

- **Réduction de la dimensionnalité** : La réduction de dimensionnalité est une technique qui compresse les données en réduisant le nombre de variables tout en préservant l'information essentielle. Elle est utile lorsque les jeux de données ont trop de caractéristiques, ce qui peut nuire aux performances des algorithmes d'apprentissage automatique. Par exemple, en vision par ordinateur, elle permet de réduire la taille des images tout en conservant les éléments importants pour la classification ou la détection d'objets. En finance, elle aide à l'analyse des risques et à la détection de fraudes en simplifiant de grands ensembles de données.

2.2.3 Apprentissage par renforcement

L'apprentissage par renforcement [1] est une méthode d'apprentissage automatique inspirée de l'apprentissage humain et animal où un agent apprend à prendre des décisions optimales par interaction avec son environnement. L'agent exécute des actions, observe les résultats, et reçoit des récompenses ou des pénalités Figure 2.5. Son objectif est de maximiser le cumul des

récompenses à long terme. Ce processus itératif d'essai-erreur permet à l'agent de développer une stratégie optimale, en équilibrant l'exploration de nouvelles options et l'exploitation des connaissances acquises.

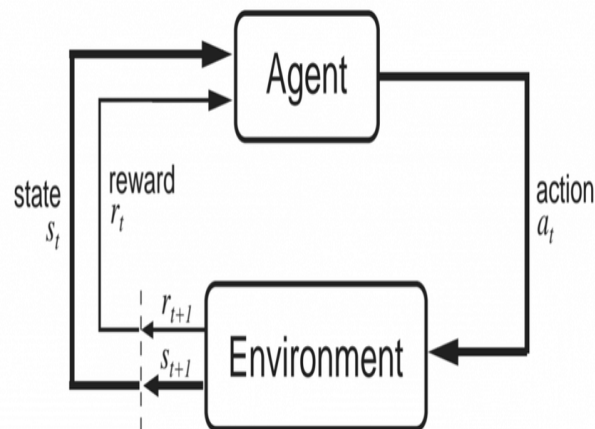


FIGURE 2.5 – Mécanisme de l'apprentissage par renforcement

- **Q-Learning (QL)** : Le QL est un algorithme d'apprentissage par renforcement qui permet de déterminer l'action optimale pour maximiser les récompenses futures. Il apprend à évaluer la qualité des actions dans différents États, en tenant compte d'un facteur d'actualisation qui détermine l'importance des récompenses futures. Un facteur proche de 0 privilégie l'état actuel, tandis qu'un facteur proche de 1 valorise les résultats futurs.

Le QL est utilisé dans divers domaines, comme les jeux vidéo pour former des agents à jouer, dans les robots autonomes pour naviguer efficacement, dans les systèmes de recommandation pour améliorer l'engagement des utilisateurs, et dans le secteur financier pour aider à prendre des décisions d'investissement.

- **Méthode des gradients de politique (Policy Gradient (PG))** : La PG est un algorithme d'apprentissage par renforcement qui ajuste directement la stratégie de décision d'un agent. Au lieu d'apprendre une fonction de valeur, il modifie les probabilités de sélection des actions dans différents états pour maximiser la récompense attendue. Cette approche est utilisée dans divers domaines, comme les véhicules autonomes pour naviguer efficacement dans la circulation, la robotique pour accomplir des tâches complexes, l'optimisation de la gestion des ressources dans les réseaux informatiques, la personnalisation des expériences d'apprentissage en éducation, et même dans la création artistique pour générer des œuvres interactives.
- **Algorithme acteur-critique (Soft Actor-Critic (SAC))** : Le SAC représente une approche novatrice dans le domaine de l'apprentissage par renforcement. Cette méthode se distingue par sa capacité à apprendre de manière dynamique, sans nécessiter une interaction directe avec l'environnement pendant la phase d'apprentissage. Sa particularité réside dans son double objectif : maximiser les récompenses tout en maintenant un niveau optimal d'aléatoire dans les décisions grâce à l'entropie de la politique.

L'originalité de SAC vient de sa structure qui combine deux éléments essentiels : une fonction évaluant la valeur des actions et une politique probabiliste guidant les choix de l'agent. Cette architecture permet d'explorer efficacement différentes possibilités d'actions tout en évitant les comportements trop rigides. Les réseaux de neurones, au cœur de cet algorithme, permettent d'adapter la stratégie aux environnements complexes et changeants.

Dans le monde réel, SAC brille particulièrement dans les domaines nécessitant une adaptation fine et continue. Dans la robotique : un bras robotique équipé de SAC peut apprendre progressivement à saisir des objets de formes variées, en ajustant constamment ses mouvements grâce à ce subtil équilibre entre exploitation des connaissances acquises et exploration de nouvelles solutions.

2.2.4 Apprentissage profond

L'apprentissage profond [19], un domaine clé de l'IA, qui utilise des réseaux de neurones pour modéliser des données complexes. Bien que le concept ait été popularisé par la science-fiction au milieu du 20e siècle, ses fondements scientifiques remontent à 1943, avec les travaux pionniers de Walter Pitts et Warren McCulloch. Ces chercheurs ont créé un modèle informatique basé sur les réseaux neuronaux du cerveau et conçu une machine imitant la fonction d'un neurone biologique. Leurs recherches, initiées à l'Université de Chicago puis poursuivies au Massachusetts Institute of Technology, ont conduit à la création de ce qui est considéré aujourd'hui comme le premier département de sciences cognitives. Ces travaux fondateurs ont posé les bases théoriques de l'apprentissage profond, ouvrant la voie à des décennies de recherche et d'innovation dans le domaine de l'IA.

- **Les réseaux de neurones récurrents (RNN) :** Les RNN sont conçus pour traiter des séquences de données en utilisant des connexions récurrentes qui leur permettent de mémoriser des informations des étapes précédentes. Cela leur permet de capturer des dépendances temporelles, ce qui est utile pour des tâches comme la traduction automatique, la reconnaissance vocale et l'analyse de sentiments.
- **Les réseaux de neurones convolutifs (CNN) :** Les CNN sont conçus pour traiter des données structurées en grille, comme les images. Leur fonctionnement repose sur des couches de convolution qui extraient automatiquement des caractéristiques importantes à différents niveaux de complexité, en utilisant des filtres pour détecter des motifs, des textures et des formes. Les CNN ont de nombreuses applications dans des domaines tels que la vision par ordinateur, la reconnaissance d'images et la classification d'objets. Grâce à leur capacité à apprendre des caractéristiques visuelles de manière hiérarchique, ils sont particulièrement performants pour des tâches comme la détection de visages, la lecture optique de caractères ou encore l'analyse d'imagerie médicale.
- **Apprentissage de bout-en-bout :** L'apprentissage de bout-en-bout est une approche d'apprentissage profond qui vise à traiter directement les données brutes d'entrée pour produire des résultats, sans nécessiter d'étapes intermédiaires de traitement. Dans le contexte de la navigation autonome, cette méthode utilise des réseaux de neurones pour recevoir des informations provenant de capteurs, comme des caméras ou des lidars, et générer directement des commandes de navigation, telles que des instructions de direction ou des vitesses. Le fonctionnement repose sur l'entraînement du modèle à partir d'un vaste ensemble de données d'exemples d'entrées et de sorties, permettant au réseau d'apprendre à associer des situations spécifiques à des actions appropriées.
En pratique, l'apprentissage de bout-en-bout est particulièrement utile pour les véhicules autonomes et les robots, car il simplifie le processus de navigation en éliminant la nécessité de modules distincts pour la perception, la planification et le contrôle.

Par exemple, un véhicule autonome peut être entraîné à conduire simplement en observant des données d'entraînement provenant de conducteurs humains, apprenant ainsi à réagir aux obstacles, à respecter les feux de circulation et à naviguer dans des environnements urbains. Cette approche permet une adaptation rapide à des situations variées et complexes, rendant les systèmes de navigation plus agiles et réactifs.

2.2.5 Systèmes experts

Un système expert est un programme informatique conçu pour résoudre des problèmes complexes en imitant le raisonnement d'un expert humain. Il repose sur une base de connaissances (faits et règles) et un moteur d'inférence, qui utilise ces informations pour prendre des décisions ou fournir des recommandations. Dans le domaine de la robotique, les systèmes experts sont utilisés pour diverses applications :

- **Diagnostic et maintenance** : Les systèmes experts peuvent diagnostiquer les pannes dans les robots en analysant les données de fonctionnement et en comparant ces informations à une base de connaissances sur les défaillances possibles. Cela permet une maintenance prédictive, réduisant ainsi les temps d'arrêt.
- **Planification et prise de décision** : Dans des environnements complexes, comme ceux rencontrés dans la robotique mobile ou industrielle, ces systèmes aident à planifier les mouvements et les actions des robots en fonction des conditions environnementales et des objectifs à atteindre. Par exemple, un robot peut utiliser un système expert pour déterminer le meilleur chemin à suivre dans un entrepôt.
- **Assistance médicale** : Les robots chirurgicaux intègrent souvent des systèmes experts pour assister les médecins lors d'interventions. Ces systèmes peuvent analyser les données du patient et recommander les meilleures pratiques chirurgicales, améliorant ainsi la précision et la sécurité des procédures.
- **Contrôle de qualité en production** : Dans le secteur industriel, les systèmes experts sont utilisés pour superviser le processus de fabrication, identifier les défauts potentiels et proposer des solutions en temps réel pour optimiser la qualité du produit.
- **Interaction homme-machine** : Les systèmes experts facilitent l'interaction entre les robots et les utilisateurs humains en traduisant les demandes en actions appropriées, rendant ainsi l'utilisation des robots plus intuitive et accessible

2.2.6 Logique Floue

La logique floue (ou fuzzy logic en anglais) est une approche de l'intelligence artificielle qui permet de traiter des valeurs de vérité intermédiaires, plutôt que de se limiter aux simples états binaires de vrai ou faux. Développée par le mathématicien Lotfi Zadeh dans les années 1960, cette méthode est particulièrement utile pour modéliser des systèmes complexes où les données ne peuvent pas être facilement classées dans des catégories strictes. La logique floue repose sur l'idée que, contrairement à la logique booléenne qui attribue une valeur de vérité de 0 (faux) ou 1 (vrai), il existe des degrés de vérité qui peuvent être représentés par des valeurs réelles comprises entre 0 et 1. Cela permet une représentation plus nuancée des informations, rendant cette approche particulièrement adaptée à des situations où les données sont imprécises ou vagues. Par exemple, au lieu de dire qu'une température est "chaude" ou "froide", la logique floue permet de dire qu'elle est "un peu chaude" ou "tiède" avec des valeurs spécifiques associées à ces descriptions. La logique floue trouve plusieurs applications dans le domaine de la robotique :

- **Contrôle des systèmes dynamiques** : Dans les robots autonomes, la logique floue est utilisée pour le contrôle moteur et la navigation. Par exemple, un robot peut ajuster sa vitesse et sa direction en fonction d'informations vagues sur la distance aux obstacles, permettant une navigation plus fluide et réactive.
- **Reconnaissance de formes** : Les systèmes de vision par ordinateur peuvent utiliser la logique floue pour interpréter des images et reconnaître des objets dans des environnements complexes. Cela est particulièrement utile lorsque les objets sont partiellement obscurcis ou lorsque les conditions d'éclairage varient.
- **Systèmes d'aide à la conduite** : Dans les véhicules autonomes, la logique floue aide à interpréter divers capteurs pour prendre des décisions sur le freinage, l'accélération et le changement de voie en fonction de conditions routières variables.
- **Interaction homme-machine** : La logique floue améliore l'interaction entre les humains et les robots en permettant aux robots de comprendre et d'interpréter les commandes humaines qui peuvent être vagues ou imprécises.
- **Diagnostic et maintenance prédictive** : En analysant les données opérationnelles d'un robot, un système basé sur la logique floue peut évaluer l'état du robot et prédire quand une maintenance sera nécessaire, réduisant ainsi les temps d'arrêt imprévus.

2.2.7 Algorithmes génétique

Un algorithme génétique fonctionne en simulant le processus d'évolution biologique.

1. **Initialisation** : Une population initiale de solutions candidates, souvent représentées sous forme de chaînes binaires (chromosomes), est générée aléatoirement.
2. **Évaluation** : Chaque solution est évaluée à l'aide d'une fonction de fitness, qui mesure sa performance par rapport au problème à résoudre.
3. **Sélection** : Les meilleures solutions sont sélectionnées pour se reproduire, favorisant celles qui ont obtenu les meilleures évaluations.
4. **Croisement et mutation** : Les solutions sélectionnées subissent des opérations de croisement (recombinaison de parties de deux solutions) et de mutation (modification aléatoire d'une partie d'une solution) pour créer une nouvelle génération de solutions.
5. **Itération** : Ce processus est répété sur plusieurs générations jusqu'à ce qu'une solution satisfaisante soit trouvée ou qu'un critère d'arrêt soit atteint.

2.2.8 Traitement du langage naturel (NLP)

Le NLP combine plusieurs disciplines, notamment la linguistique informatique, l'apprentissage automatique et l'intelligence artificielle. Son objectif est de permettre aux ordinateurs d'analyser et de traiter des données en langage naturel, qu'il s'agisse de texte écrit ou de discours. Les techniques de NLP incluent :

- **Analyse syntaxique** : Compréhension de la structure grammaticale des phrases.
- **Analyse sémantique** : Interprétation du sens des mots et des phrases dans leur contexte.
- **Apprentissage machine** : Utilisation d'algorithmes pour apprendre à partir de données et améliorer les performances au fil du temps.
- **Interaction homme-machine** : Le NLP permet une interaction plus naturelle entre les humains et les robots. Grâce à la compréhension du langage naturel, les robots peuvent engager des conversations fluides et adaptées au contexte, rendant leur utilisation plus intuitive et accessible.

- **Analyse des sentiments** : Dans certains cas, les robots peuvent analyser les émotions exprimées par les utilisateurs à travers leur langage. Cela peut être particulièrement utile dans des applications de service client où comprendre le sentiment d'un client peut aider à personnaliser la réponse ou l'assistance fournie.
- **Traduction automatique et communication multilingue** : Les robots déployés dans des environnements internationaux peuvent utiliser le NLP pour traduire instantanément le langage parlé ou écrit, facilitant ainsi la communication entre personnes de différentes langues.
- **Analyse des sentiments sur les réseaux sociaux** : Outils permettant d'évaluer les opinions publiques sur divers sujets en analysant les commentaires et publications.

2.2.8.1 Fonctionnement du traitement du langage naturel

Le processus de NLP implique plusieurs étapes :

1. **Prétraitement des données** : Nettoyage et préparation des données textuelles pour l'analyse.
2. **Extraction d'entités** : Identification des éléments clés dans le texte, comme les noms, les dates ou les lieux.
3. **Analyse des sentiments** : Évaluation des émotions ou opinions exprimées dans le texte.
4. **Génération de texte** : Création de réponses ou d'analyses basées sur les données traitées.

2.2.8.2 quelques applications clés du NLP dans la robotique

- **Assistants vocaux et robots d'assistance** : Les robots d'assistance, tels que ceux utilisés dans les soins aux personnes âgées ou dans les environnements domestiques, intègrent des systèmes NLP pour comprendre les commandes vocales des utilisateurs. Par exemple, un robot peut exécuter des tâches comme allumer des lumières ou fournir des informations sur la météo en répondant à des requêtes vocales.
- **Robots conversationnels (chatbots)** : De nombreux robots, notamment ceux utilisés dans le service client, sont équipés de chatbots basés sur le NLP. Ces systèmes peuvent interpréter les questions des clients et fournir des réponses pertinentes, améliorant ainsi l'expérience utilisateur. Par exemple, un chatbot peut aider à suivre une commande ou répondre à des questions fréquentes.
- **Interaction homme-machine** : Le NLP permet une interaction plus naturelle entre les humains et les robots. Grâce à la compréhension du langage naturel, les robots peuvent engager des conversations fluides et adaptées au contexte, rendant leur utilisation plus intuitive et accessible.
- **Analyse des sentiments** : Dans certains cas, les robots peuvent analyser les émotions exprimées par les utilisateurs à travers leur langage. Cela peut être particulièrement utile dans des applications de service client où comprendre le sentiment d'un client peut aider à personnaliser la réponse ou l'assistance fournie.
- **Traduction automatique et communication multilingue** : Les robots déployés dans des environnements internationaux peuvent utiliser le NLP pour traduire instantanément le langage parlé ou écrit, facilitant ainsi la communication entre personnes de différentes langues.
- **Automatisation des tâches** : Le NLP aide également à automatiser certaines tâches liées à l'analyse de texte, comme la classification de documents ou l'extraction d'informations pertinentes à partir de grandes quantités de données textuelles.

2.3 État de l'art

2.3.1 Q-learning et navigation autonome

L'article de Strauss et Sahin (2008) [6] se concentre sur l'utilisation du Q-learning, un algorithme d'apprentissage par renforcement, pour la navigation autonome. Ce modèle permet au robot d'apprendre à éviter des obstacles en évaluant les actions à entreprendre en fonction de l'état de l'environnement, ce qui est essentiel dans des scénarios où les conditions peuvent changer rapidement.

2.3.1.1 Processus d'apprentissage :

Le robot commence sans connaissance préalable de son environnement et utilise un tableau Q qui associe des états aux valeurs d'action. Au fur et à mesure que le robot interagit avec son environnement, il reçoit des récompenses ou des pénalités, ce qui lui permet d'ajuster ses actions pour maximiser les récompenses à long terme.

2.3.1.2 Combinaison simulation et monde réel :

L'article souligne l'importance de l'apprentissage à la fois en simulation et dans des environnements réels. Les simulations permettent d'accélérer le processus d'apprentissage en fournissant un cadre sans risque, tandis que les expériences réelles affinent les décisions du robot en tenant compte des imprévus.

2.3.2 Intégration de l'IoT et des techniques de détection d'objets

Dans l'article de Saravanan et al. [10], les auteurs mettent en avant un robot mobile autonome qui intègre des techniques de détection d'objets via des CNN, illustrant l'application de l'IA dans des scénarios pratiques.

2.3.2.1 Fonctionnalités IoT

Le robot est équipé de capteurs IoT qui lui permettent de surveiller des paramètres cruciaux comme l'humidité du sol et la température, essentiels pour des applications comme la gestion des plantes en intérieur. Ces données sont collectées et analysées en temps réel, permettant au robot d'ajuster ses actions en conséquence.

2.3.2.2 Transfert d'intelligence

L'architecture proposée inclut un transfert d'intelligence depuis le cloud vers un nœud de Fog Computing. Cela permet de réduire la latence et de minimiser les besoins en bande passante, tout en garantissant que le robot puisse fonctionner même en cas de connexion intermittente.

2.3.3 Environnements d'apprentissage et défis

Les défis auxquels sont confrontés les robots dans les trois articles se concentrent sur la nécessité d'apprendre dans des environnements dynamiques et inconnus.

2.3.3.1 Obstacles statiques et dynamiques

Les robots doivent naviguer non seulement autour d'obstacles fixes, mais aussi s'adapter à des objets en mouvement, ce qui complique encore la navigation. Cela nécessite des algorithmes robustes capables de gérer l'incertitude et la variabilité.

2.3.4 Gestion de la mémoire et traitement en temps réel

L'utilisation de techniques d'optimisation de la mémoire est cruciale pour exécuter des modèles d'apprentissage automatique sur des plateformes avec des ressources limitées. De plus, le traitement des données en temps réel est essentiel pour garantir des décisions rapides et appropriées.

2.4 Apprentissage machine pour la perception de l'environnement

L'intégration de l'apprentissage machine dans la perception de l'environnement a propulsé la robotique mobile autonome vers de nouveaux horizons. Cette synergie entre intelligence artificielle et capteurs avancés a transformé la manière dont les robots interprètent leur environnement, notamment en matière de vision et de localisation.

Dans le domaine de la vision, les architectures neuronales profondes ont révolutionné le traitement d'images. Ces modèles, s'inspirant du fonctionnement du cortex visuel, excellent dans la reconnaissance de formes complexes et la segmentation de scènes en temps réel. Les dernières innovations permettent aux robots de distinguer finement les objets, les personnes et leurs interactions dynamiques, même dans des environnements encombrés ou changeants.

L'estimation de profondeur à partir d'images monoculaires a connu des avancées significatives grâce à des techniques d'apprentissage non supervisé. Ces approches enrichissent considérablement la perception spatiale des robots sans nécessiter de capteurs additionnels onéreux, ouvrant la voie à des systèmes de navigation plus compacts et économiques. En matière de localisation, l'incorporation de l'apprentissage profond dans les algorithmes de cartographie et de localisation simultanées (**Simultaneous Localization And Mapping (SLAM)**) [3] : est une technique utilisée pour créer une carte d'un environnement en suivant la position du robot. Les capteurs comme les LIDAR, les caméras et les odométries sont utilisés pour le localiser. La cartographie est la création d'une représentation 2D ou 3D de l'environnement. À chaque instant, le robot doit estimer sa position et la carte de son environnement. a nettement amélioré la robustesse des systèmes dans des environnements non structurés.

Ces nouvelles approches permettent une meilleure extraction de caractéristiques visuelles stables et une prédiction plus précise des mouvements, renforçant ainsi la fiabilité de l'odométrie visuelle. L'émergence de modèles basés sur l'attention, adaptés au traitement d'images séquentielles, a introduit une dimension temporelle cruciale dans l'analyse de l'environnement. Cette évolution permet une compréhension plus nuancée du contexte, essentielle pour anticiper les changements dynamiques de l'espace environnant. La fusion des données provenant de multiples capteurs bénéficie également des avancées en apprentissage machine. Des techniques sophistiquées optimisent la combinaison d'informations issues de diverses sources sensorielles, améliorant ainsi la précision et la robustesse de la perception globale. Les méthodes d'adaptation rapide, telles que l'apprentissage par transfert, gagnent en importance. Elles permettent aux systèmes robotiques d'ajuster efficacement leurs modèles perceptuels à de nouveaux environnements, avec un mi-

nimum de données d'entraînement supplémentaires. L'intégration de stratégies d'apprentissage actif dans les systèmes de perception permet aux robots d'interagir de manière plus proactive avec leur environnement, optimisant ainsi leur processus d'acquisition de connaissances, particulièrement dans des scénarios d'exploration de milieux inconnus.

Ces avancées convergent vers le développement de robots dotés d'une compréhension contextuelle approfondie, capables de s'adapter en temps réel et d'interagir naturellement avec leur environnement. Cette évolution ouvre la voie à des applications innovantes dans des domaines variés, de l'assistance personnelle à l'exploration de terrains complexes, en passant par l'optimisation des processus industriels dans des environnements dynamiques.

2.5 Avantages de l'intelligence artificielle et de la robotique

L'IA et la robotique, bien que distinctes, sont étroitement liées et offrent des avantages synergiques considérables. L'IA apporte aux robots une capacité de perception, d'apprentissage et de prise de décision autonome, tandis que la robotique fournit à l'IA une plateforme physique pour interagir avec le monde réel. Cette symbiose permet la création de systèmes hautement adaptables et performants. Dans l'industrie, des robots dotés d'IA peuvent non seulement effectuer des tâches répétitives avec précision, mais aussi s'adapter à des situations imprévues, optimisant ainsi la production. Dans le domaine médical, des bras robotiques guidés par l'IA réalisent des interventions chirurgicales complexes avec une précision inégalée. En exploration, des robots autonomes équipés d'IA peuvent naviguer dans des environnements dangereux ou inaccessibles, collectant et analysant des données en temps réel. Cette combinaison stimule également l'innovation dans des domaines comme les véhicules autonomes, où l'IA traite les informations de l'environnement pour guider le comportement du véhicule. Ensemble, l'IA et la robotique repoussent les limites de l'automatisation intelligente, ouvrant la voie à des applications révolutionnaires qui améliorent l'efficacité, la sécurité et la qualité de vie dans de nombreux secteurs.

2.6 Conclusion

Ce chapitre a présenté les principales approches d'IA appliquées à la mobilité autonome, couvrant l'apprentissage automatique et l'apprentissage profond. Nous avons exploré leur utilisation pour la perception de l'environnement et souligné les avantages qu'elles apportent à la robotique. Ces algorithmes intelligents ouvrent de nouvelles perspectives pour relever les défis de la mobilité autonome, tout en nécessitant une compréhension approfondie pour une mise en œuvre efficace. Les chapitres suivants examineront leur application pratique dans le développement de solutions autonomes robustes.

Environnements matériels et logiciels

Sommaire

3.1	Introduction	39
3.2	Robot MiniLab	39
3.2.1	Définition	39
3.2.2	Architecture de MiniLab	39
3.3	Environnement de travail	41
3.3.1	VMware Workstation	41
3.3.2	Ubuntu	42
3.3.3	Gazebo	42
3.3.4	Rviz	43
3.4	Conception de l'environnement sur Gazebo	43
3.4.1	Structure de l'interface d'accueil de Gazebo	43
3.4.2	Construction d'un modèle sur Gazebo	44
3.5	Placement du modèle du robot dans l'environnement de simulation	48
3.5.1	Mise en place de l'environnement	48
3.6	Simulation du robot avec le modèle réalisé	51
3.7	Architecture et fonctionnement des nœuds ROS	53
3.7.1	Nœuds actifs	53
3.7.2	Topics actifs	54
3.8	Conclusion	55

3.1 Introduction

Ce chapitre présente l'ensemble des outils matériels et logiciels utilisés dans le cadre de notre projet. Nous commencerons par présenter le robot MiniLab, sujet de l'étude réalisée dans ce mémoire, en détaillant ses caractéristiques et son architecture. Ensuite, nous explorerons l'environnement de travail mis en place, notamment la configuration virtuelle sous VMware Workstation et l'utilisation du système d'exploitation Ubuntu. Un accent particulier sera mis sur les outils de simulation Gazebo et Rviz, essentiels pour le développement et les tests. Nous décrirons également en détail les tâches réalisées, de la conception de l'environnement virtuel à la simulation du robot, avant de conclure sur l'architecture ROS mise en œuvre dans notre projet.

3.2 Robot MiniLab

3.2.1 Définition

Le MiniLab est un robot dédié à la recherche, conçu et développé par l'entreprise Enova Robotics . Ce robot de service polyvalent est spécifiquement destiné à l'expérimentation et à l'implémentation d'algorithmes d'IA. Il se prête aussi bien à la programmation de bas niveau qu'aux applications de haut niveau en environnement intérieur, telles que la planification de trajectoires, le suivi, la surveillance et la supervision. Équipé de nombreux capteurs, le MiniLab offre aux chercheurs un banc d'essai idéal pour mettre en œuvre et tester divers algorithmes, en exploitant les données riches et variées fournies par ses capteurs. Sa flexibilité et sa polyvalence en font un outil précieux pour la recherche en robotique autonome et en IA appliquée.



FIGURE 3.1 – Robot MiniLab

3.2.2 Architecture de MiniLab

L'architecture de MiniLab [18] représente un équilibre minutieux entre puissance de calcul, flexibilité opérationnelle et intégration sensorielle, formant ainsi une infrastructure technologique capable de supporter une vaste gamme d'algorithmes d'IA pour la navigation et la prise de décision autonome.

3.2. Robot MiniLab

3.2.2.1 Architecture Matérielle

Le robot MiniLab est équipé d'un ensemble de capteurs et actionneurs :

- Cinq capteurs ultrason
- Cinq capteurs infrarouge
- Caméra de profondeur : Asus Xtion Live Pro
- Encodeurs avec résolution de 16 bits
- Led indicateurs
- Boutons
- Deux moteurs à courant continu
- Processeur Atom N2800 (1.86Hz)
- Capteurs de bas niveau qui s'interfacent avec Arduino pour le traitement des données des capteurs
- Batterie : 12V

3.2.2.2 Les caractéristiques du robot MiniLab

— **Dimension :**

Les dimensions du robot MiniLab sont illustrés par la Figure 3.2

- Largeur : 0,409 m
- Longueur : 0,364 m
- Hauteur : 0,231 m

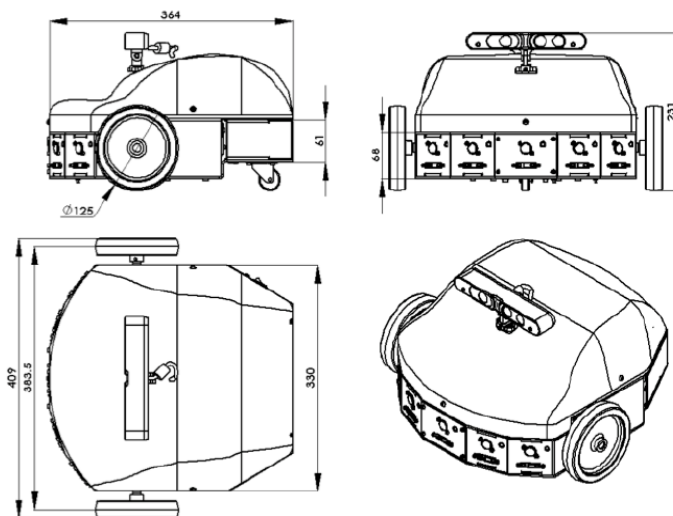


FIGURE 3.2 – Les dimensions du robot MiniLab

- **Poids :** 11,5 kg
- **Capacité de charge :** 3 kg
- **Vitesse maximale de mouvement vers l'avant et vers l'arrière :** 1.5 m/s
- **Angle de pente maximum :** 10°

3.2.2.3 Performance et spécification techniques

- Le kit dispose d'un châssis en aluminium solide propulsé par des moteurs à couple élevé, indiquant un design robuste et durable adapté à diverses applications,
- La connectivité WIFI (IEEE 802.11b/g/n-) permet la communication et le contrôle à distance du robot,

3.3. Environnement de travail

- L'autonomie : 4 heures,
- L'abstraction matérielle avec l'architecture ROS facilite l'intégration et la programmation des composants du robot.

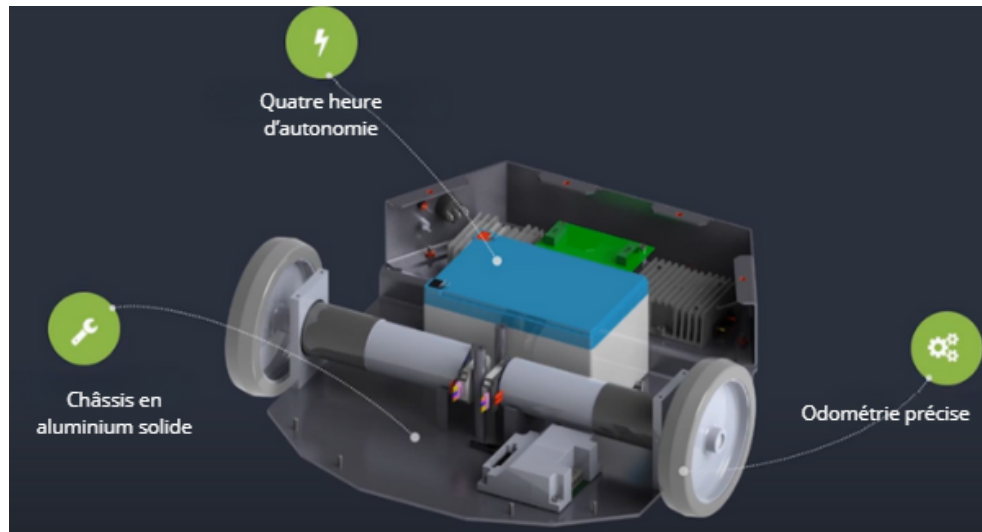


FIGURE 3.3 – Les performances de robot MiniLab

3.3 Environnement de travail

3.3.1 VMware Workstation

VMware Workstation est un outil de virtualisation de poste de travail créé par la société VMware. Il peut être utilisé pour mettre en place un environnement de test pour développer de nouveaux logiciels, ou pour tester l'architecture complexe d'un système d'exploitation avant de l'installer réellement sur une machine physique.



FIGURE 3.4 – Logo de VMware Workstation

3.3.2 Ubuntu

Ubuntu est un système d'exploitation open source de classe entreprise, basé sur la distribution Linux Debian. Développé et maintenu par Canonical Ltd. Ubuntu se distingue par son cycle de publication biannuel, offrant des versions à support à long terme (LTS) tous les deux ans. Ce système est conçu pour fonctionner sur une large gamme de plateformes, des serveurs aux ordinateurs personnels, en passant par les systèmes embarqués. Dans le domaine de la robotique et de l'IA, Ubuntu est utilisé pour sa stabilité, sa sécurité et sa compatibilité avec les principaux outils de développement et d'expérimentation. Son noyau Linux optimisé et sa capacité à fonctionner en temps réel avec des patches appropriés en font une plateforme idéale pour le contrôle de systèmes robotiques tels que MiniLab.

La version d'Ubuntu utilisée dans ce projet est Ubuntu 20.04.06 LTS



FIGURE 3.5 – Logo de Ubuntu

3.3.3 Gazebo

Gazebo est un simulateur 3D haute fidélité pour systèmes robotiques, conçu pour le test et le développement d'algorithmes de robotique avancés. Développé initialement par l'Open Source Robotics Foundation (OSRF), Gazebo (Figure 3.6) offre les fonctionnalités suivantes :

- **Moteurs physiques robustes** : Intégration de plusieurs moteurs de simulation physique (ODE, Bullet, Simbody, DART) permettant une modélisation précise des interactions physiques.
- **Rendu graphique haute qualité** : Utilisation d' Open Graphics Rendering Engine (OGRE) pour générer un environnement visuel réaliste, crucial pour les algorithmes de vision par ordinateur.
- **Capteurs virtuels** : Simulation d'une large gamme de capteurs (caméras, LiDAR, IMU, GPS) avec possibilité d'ajouter du bruit pour une simulation plus réaliste.
- **Plugins extensibles** : Architecture modulaire permettant l'ajout de fonctionnalités personnalisées via des plugins C++.
- **Compatibilité ROS** : Intégration native avec ROS, facilitant le transfert des algorithmes entre environnements simulés et réels.
- **Outils de création de mondes** : Interface graphique pour la conception d'environnements de simulation complexes.
- **Simulation distribuée** : Capacité de répartir la charge de calcul sur plusieurs machines pour des simulations à grande échelle.



FIGURE 3.6 – Logo de Gazebo

3.3.4 Rviz

RViz est un outil de visualisation 3D pour le framework robotique ROS. Il permet d'afficher et d'interagir avec des données de capteurs, des modèles de robots et des environnements simulés en temps réel. RViz est largement utilisé dans le développement et le débogage de systèmes robotiques.

Ses principales fonctionnalités sont :

- **Visualisation de données de capteurs :**
 - Affichage de nuages de points issus de capteurs LiDAR ou de caméras de profondeur.
 - Visualisation d'images de caméras.
 - Représentation des données de capteurs inertiels (IMU).
- **Affichage de modèles de robots :**
 - Visualisation de modèles 3D de robots avec leurs articulations.
 - Mise à jour en temps réel de la pose du robot.
- **Représentation de l'environnement :**
 - Affichage de cartes 2D et 3D.
 - Visualisation d'obstacles et de l'espace navigable.
- **Planification de trajectoires :**
 - Affichage des chemins planifiés pour le robot.
 - Visualisation des espaces de travail et des configurations possibles.

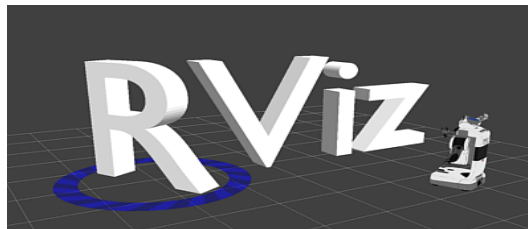


FIGURE 3.7 – Logo de Rviz

3.4 Conception de l'environnement sur Gazebo

3.4.1 Structure de l'interface d'accueil de Gazebo

La Figure 3.8 présente l'interface d'accueil de Gazebo qui est construite par :

1. **Zone de scène :** La zone principale où les objets simulés sont animés. C'est ici que l'utilisateur interagit avec l'environnement virtuel.
2. **Panneaux latéraux :**
 - **Panneau gauche :**
 - **Onglet monde :** Affiche les modèles présents dans la scène et permet de modifier leurs paramètres, comme leur position.

3.4. Conception de l'environnement sur Gazebo

- **Onglet insérer** : Permet d'ajouter de nouveaux objets à la simulation. Les utilisateurs peuvent sélectionner un modèle et l'insérer directement dans la scène.
 - **Onglet couches** : Organise les différents groupes de visualisation disponibles dans la simulation, bien que souvent vide.
 - **Panneau droit** :
 - Caché par défaut, il peut être ouvert pour interagir avec les parties mobiles d'un modèle sélectionné (joints)
3. **Barres d'outils** : Il y a deux barres d'outils
- **Barre supérieure** : Contient des options pour sélectionner, déplacer, faire pivoter et redimensionner des objets, ainsi que pour insérer des formes simples (cube, sphère, cylindre).
 - **Barre inférieure** : Affiche des données sur la simulation, comme le temps de simulation et son rapport avec le temps réel.

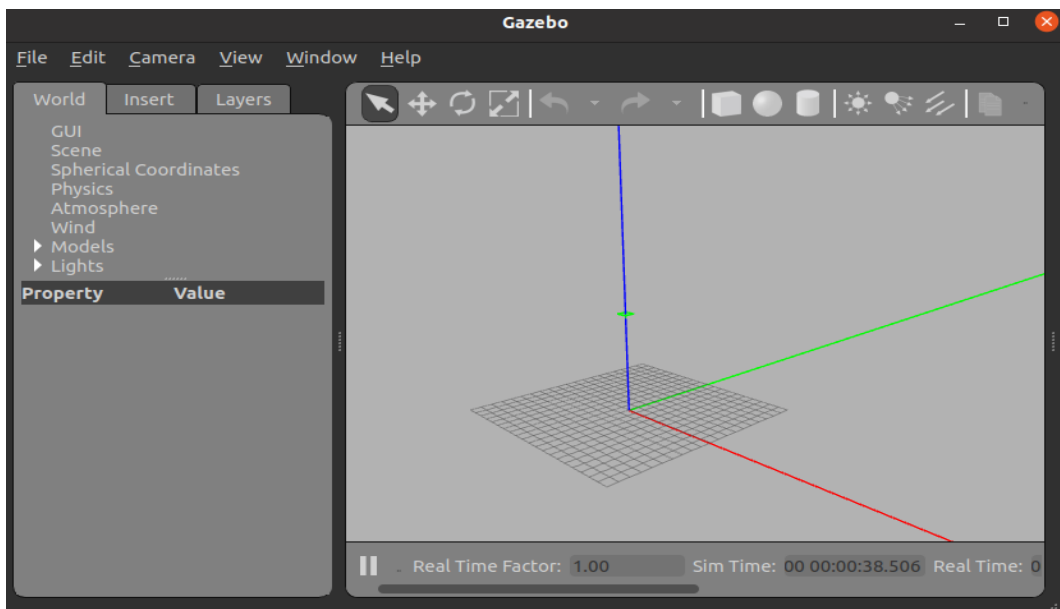


FIGURE 3.8 – Interface d'accueil de Gazebo

3.4.2 Construction d'un modèle sur Gazebo

Dans cette section, nous détaillerons la démarche méthodique suivie pour construire notre modèle sur Gazebo.

1. **Accéder à l'éditeur de bâtiment (Building Editor) :**

Dans le menu principal, sélectionnez « Edit », puis choisissez l'option « Building Editor » dans le menu déroulant, comme illustré dans la Figure 3.9. Après cette action, vous serez redirigé vers l'interface présentée dans la Figure 3.10.

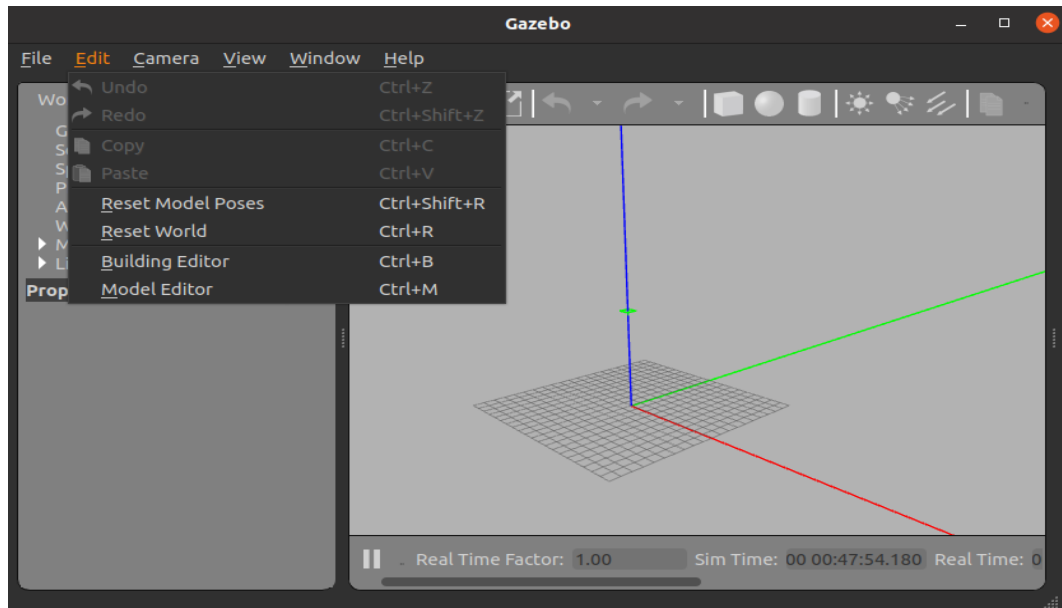


FIGURE 3.9 – Accès à l'outil Building Editor dans Gazebo

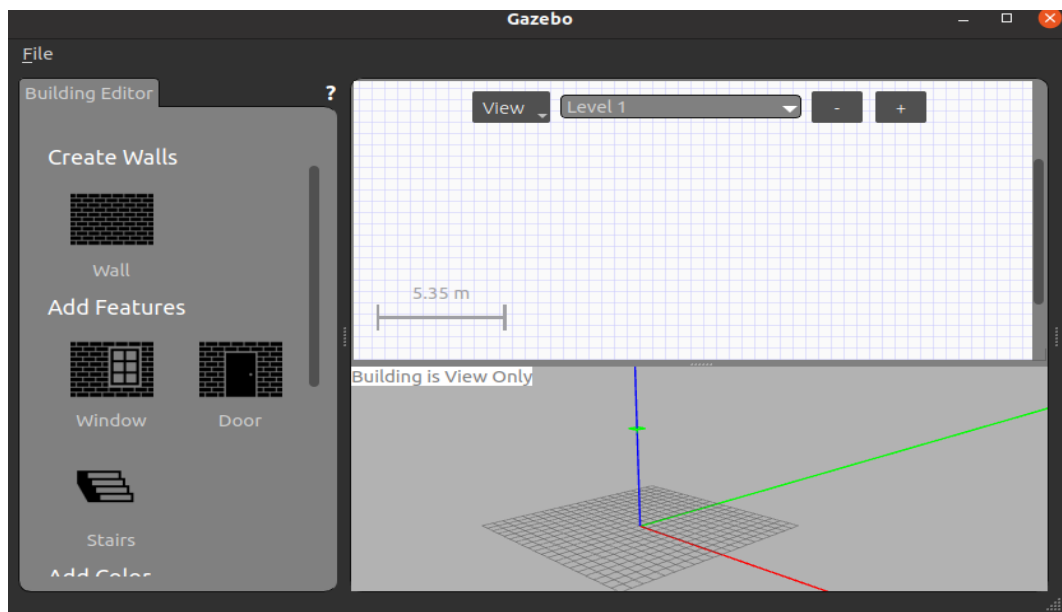


FIGURE 3.10 – Interface principale du Building Editor dans Gazebo

2. Création des murs avec l'outil « Create Walls » :

L'étape suivante consiste à utiliser le menu « Create Walls ». Ce menu est un outil essentiel pour la construction d'un modèle. Il permet de dessiner et de définir les murs et les structures de base de l'environnement virtuel. Dans ce projet, nous avons utilisé spécifiquement cet outil « Create Walls » pour créer le modèle présenté par la Figure 3.11. On peut également importer des modifications sur le modèle (changement de couleur du mur, de sa largeur, etc.) en cliquant avec le bouton droit sur un mur, comme illustré par la Figure 3.12.

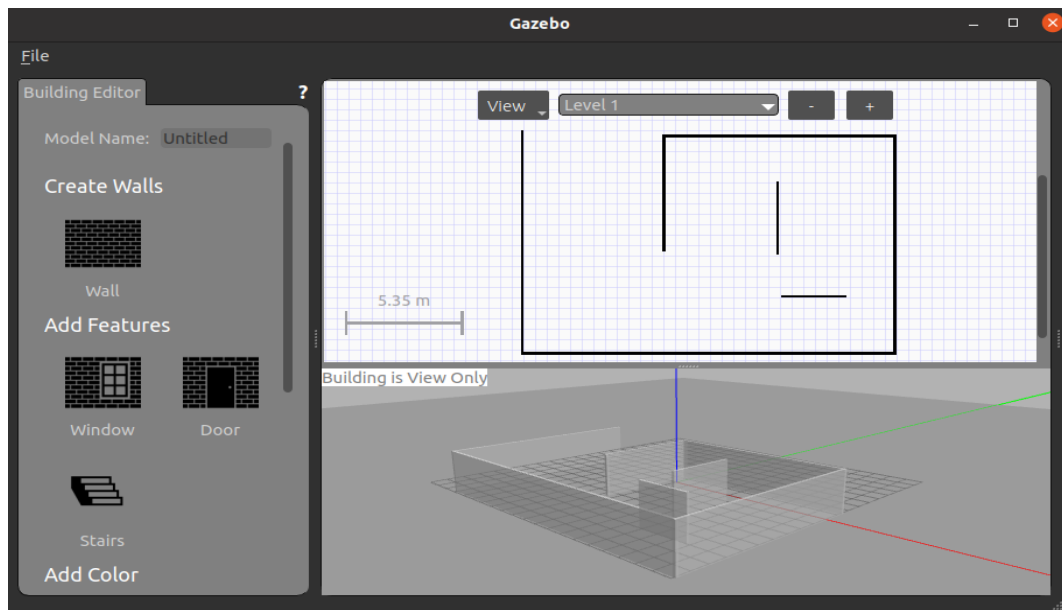


FIGURE 3.11 – Utilisation du menu « Create Walls » dans le Building Editor de Gazebo

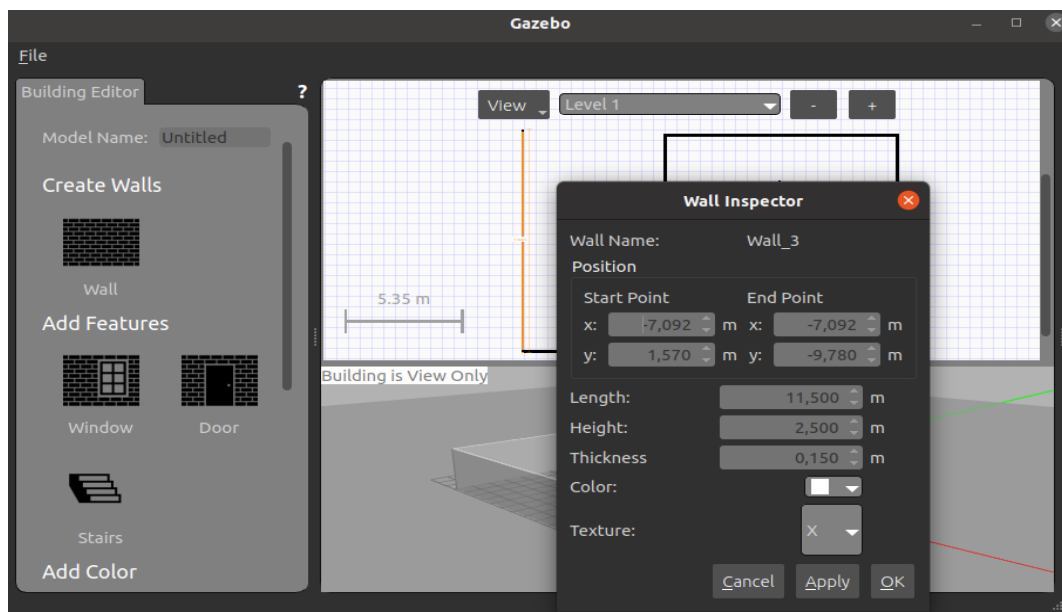


FIGURE 3.12 – Modification des propriétés du mur dans le modèle

3. Enregistrement du modèle et ajout des obstacles :

Sélectionner « Save As » dans le menu déroulant de « File », puis choisir le lieu d'enregistrement. Après cette étape, sélectionner « Exit Building Editor » dans le menu déroulant « File », ajouter des obstacles à partir de la barre d'outils de Gazebo. Le modèle final est présenté dans la Figure 3.13. Après avoir enregistré le modèle, nous obtenons deux fichiers avec les extensions .config et .sdf .

Le fichier « model.config » est utilisé pour stocker les configurations spécifiques du modèle (Figure 3.14).

3.4. Conception de l'environnement sur Gazebo

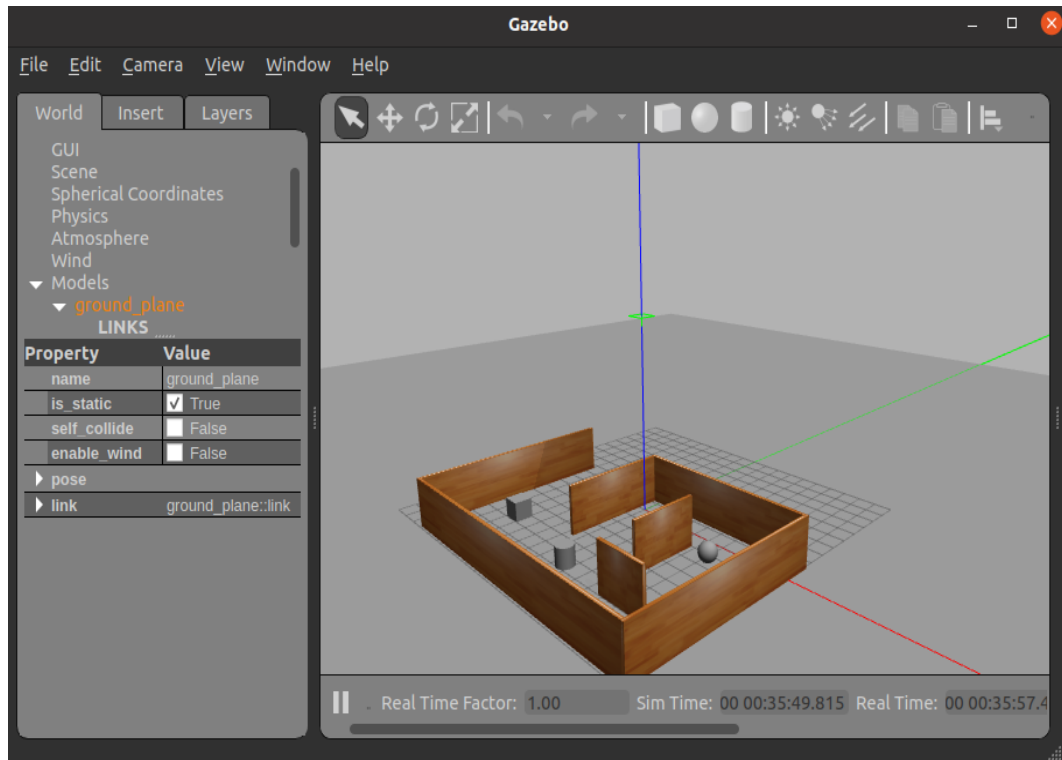


FIGURE 3.13 – Modèle final avec obstacles dans Gazebo

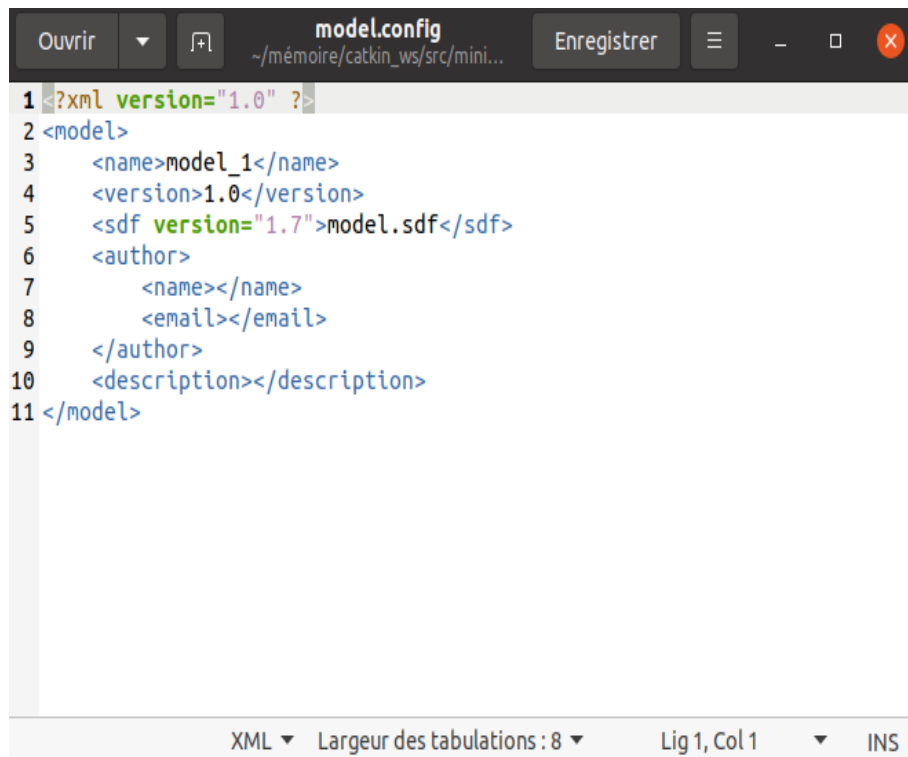


FIGURE 3.14 – Structure du fichier model.config pour la configuration de modèle

3.5. Placement du modèle du robot dans l'environnement de simulation

Le fichier « model.sdf » (Figure 3.15) contient la description du modèle (géométrie, propriétés physiques).

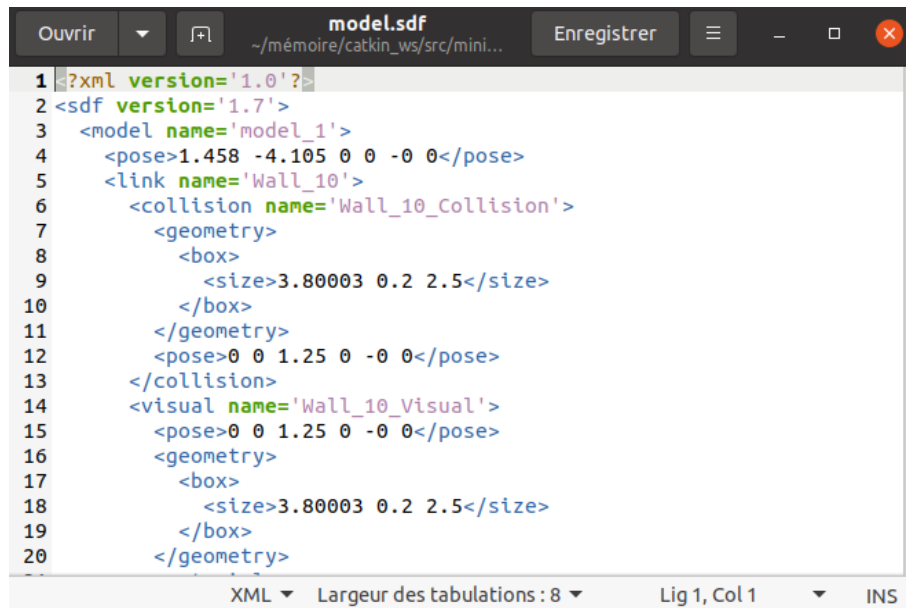


FIGURE 3.15 – Éléments du fichier model.sdf pour la description du modèle

3.5 Placement du modèle du robot dans l'environnement de simulation

3.5.1 Mise en place de l'environnement

1. Téléchargement du dossier MiniLab [?] :

Le dossier minilab contient trois sous-dossiers :



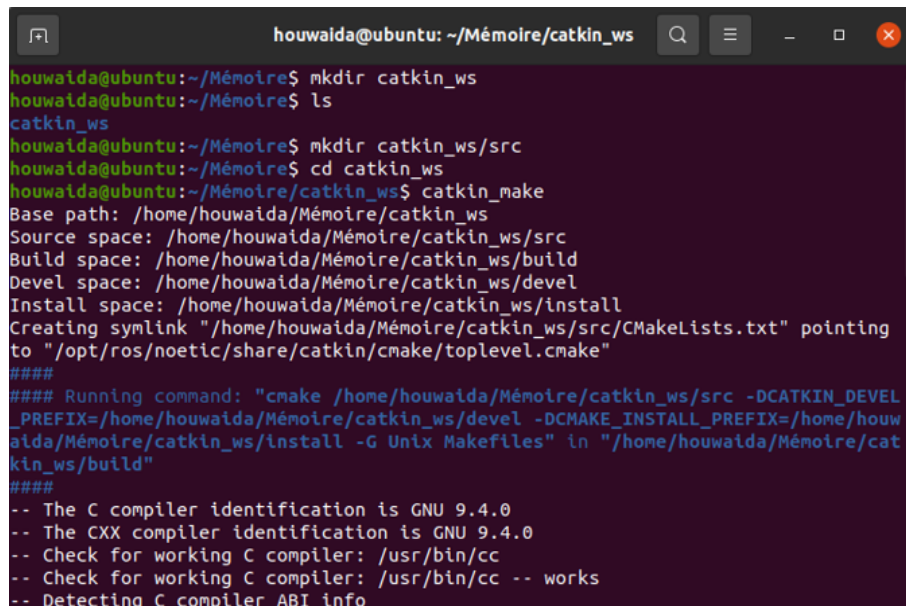
FIGURE 3.16 – Contenu du dossier MiniLab

- **Mini-Lab Matlab Gui** : Le sous-dossier « Mini-Lab Matlab Gui » contient les éléments nécessaires à la mise en place de l'interface graphique (Gui) développée sous Matlab pour contrôler et visualiser les données du MiniLab. Il y a le code source de l'application, composé de fichiers .m (scripts Matlab), .fig (définition de l'interface graphique) et le logo de Enova . Ces fichiers définissent l'agencement des différents composants de l'interface (boutons, menus déroulants, zones d'affichage, etc.) ainsi que la logique de programmation derrière chacune de ses fonctionnalités.
- **minilab hardware** : Le sous-dossier « minilab hardware » contient tous les éléments liés à la partie matérielle du MiniLab. On y trouve le dossier drivers qui englobe les pilotes nécessaires au fonctionnement des différents capteurs et actionneurs du robot, ainsi que les bibliothèques logicielles et les fichiers de configuration associés.

- **Minilab_ws** : Le sous-dossier « Minilab_ws » regroupe les éléments liés à la simulation, à la navigation et à la commande du robot. Il contient un sous-dossier « minilab_model » qui abrite les fichiers décrivant le modèle 3D du robot et de l'environnement où il se déplace. Le sous-dossier « minilab_navigation » renferme quant à lui les fichiers de configuration et les paramètres nécessaires à la navigation autonome du robot, tels que les cartes de l'environnement, les algorithmes de localisation et de planification de trajectoire. Le dossier « minilab_teleop » contient les programmes et les interfaces permettant de piloter manuellement le robot à l'aide d'un clavier.

2. Création d'un espace de travail Catkin :

Catkin Workspace : Un Catkin workspace est un répertoire spécifique utilisé dans le système de robotique ROS pour développer, construire et gérer des packages Catkin. Il sert comme une structure d'organisation pour le code lié à un projet ROS. L'ensemble des étapes à suivre pour créer un Catkin workspace sont illustrées par la Figure 3.17.



```
houwaida@ubuntu: ~/Mémoire/catkin_ws
houwaida@ubuntu:~/Mémoire$ mkdir catkin_ws
houwaida@ubuntu:~/Mémoire$ ls
catkin_ws
houwaida@ubuntu:~/Mémoire$ mkdir catkin_ws/src
houwaida@ubuntu:~/Mémoire$ cd catkin_ws
houwaida@ubuntu:~/Mémoire/catkin_ws$ catkin_make
Base path: /home/houwaida/Mémoire/catkin_ws
Source space: /home/houwaida/Mémoire/catkin_ws/src
Build space: /home/houwaida/Mémoire/catkin_ws/build
Devel space: /home/houwaida/Mémoire/catkin_ws/devel
Install space: /home/houwaida/Mémoire/catkin_ws/install
Creating symlink "/home/houwaida/Mémoire/catkin_ws/src/CMakeLists.txt" pointing
to "/opt/ros/noetic/share/catkin/cmake/toplevel.cmake"
####
### Running command: "cmake /home/houwaida/Mémoire/catkin_ws/src -DCATKIN_DEVEL
_PREFIX=/home/houwaida/Mémoire/catkin_ws/devel -DCMAKE_INSTALL_PREFIX=/home/hou
aida/Mémoire/catkin_ws/install -G Unix Makefiles" in "/home/houwaida/Mémoire/cat
kin_ws/build"
####
-- The C compiler identification is GNU 9.4.0
-- The CXX compiler identification is GNU 9.4.0
-- Check for working C compiler: /usr/bin/cc
-- Check for working C compiler: /usr/bin/cc -- works
-- Detecting C compiler ABI info
```

FIGURE 3.17 – Processus de configuration d'un Catkin workspace

La configuration de l'espace de travail Catkin se fait en plusieurs étapes. La première consiste à créer le répertoire « catkin_ws » et son sous-répertoire « src ». Une fois ces répertoires créés, il faut se placer dans « catkin_ws » à l'aide de la commande `cd catkin_ws`. La dernière étape consiste à exécuter la commande `catkin_make`.

catkin_make : est une commande utilisée pour simplifier le processus de construction des packages dans un catkin workspace. Cette commande suit la structure standard d'un catkin workspace qui comprend des répertoires tels que « src », « build » et « devel ».

devel : contient les fichiers et bibliothèques générés lors de la compilation des packages. Lorsque on exécute la commande `catkin_make`, ce répertoire est automatiquement créé et abrite divers éléments essentiels. Parmi ceux-ci, on trouve les exécutables issus de la compilation des différents packages, qui peuvent être directement exécutés pour tester les fonctionnalités développées. De plus, les bibliothèques compilées sont également stockées dans ce répertoire, permettant leur réutilisation par d'autres packages ou exécutables au

3.5. Placement du modèle du robot dans l'environnement de simulation

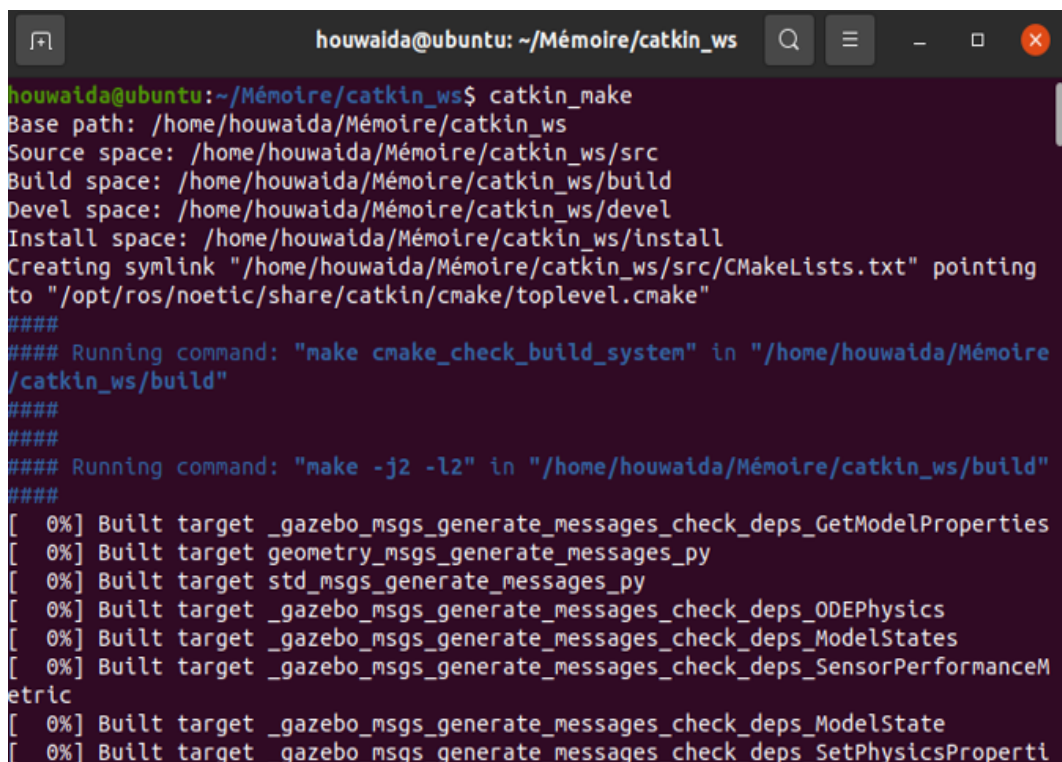
sein du workspace. Enfin, le répertoire « devel » inclut des fichiers de configuration nécessaires au bon fonctionnement des exécutables et des bibliothèques, assurant ainsi une intégration fluide et efficace des différents composants du projet.

build : contient tous les fichiers intermédiaires générés lors de la construction des packages. Lorsque vous exécutez la commande `catkin_make`, ce répertoire est automatiquement créé et utilisé pour stocker les résultats des étapes de compilation, y compris les fichiers temporaires et les artefacts nécessaires à la création des exécutables et des bibliothèques.

Chaque package compilé dispose de son propre sous-répertoire dans « build », ce qui permet une organisation claire et une gestion efficace des dépendances. En outre, le répertoire « build » facilite le débogage, car il conserve les journaux de compilation et les messages d'erreur, permettant aux développeurs d'identifier rapidement les problèmes survenant lors de la construction.

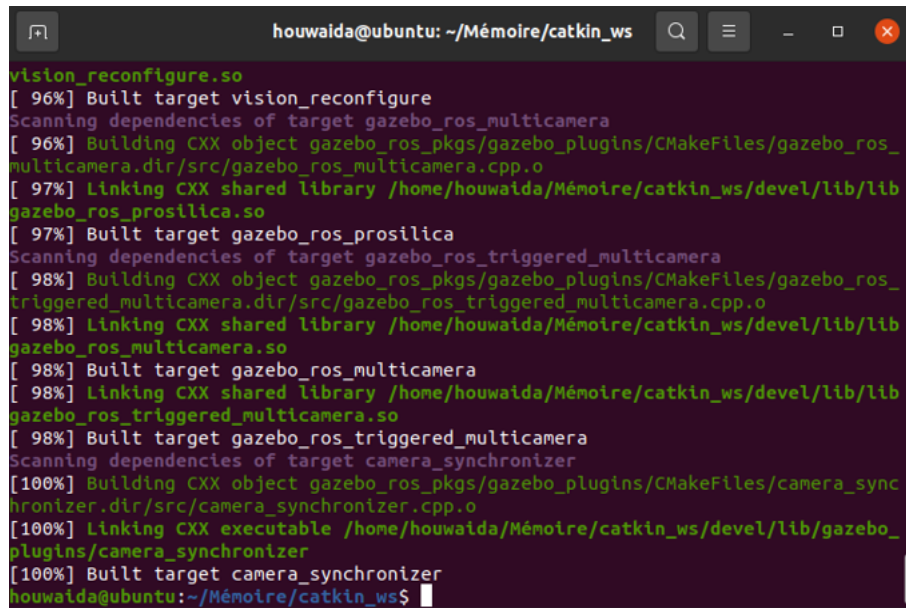
3. Déplacement du dossier « src » de « minilab_ws » vers « catkin_ws » :

Déplacez le dossier « src » qui existe dans « minilab_ws » vers « catkin_ws ». Réexécutez la commande `catkin_make` pour enregistrer les modifications que nous avons effectuées et mettre à jour les dossiers « devel » et « build » comme présenté aux Figures 3.18 et 3.19.



```
houwaida@ubuntu: ~/Mémoire/catkin_ws
houwaida@ubuntu:~/Mémoire/catkin_ws$ catkin_make
Base path: /home/houwaida/Mémoire/catkin_ws
Source space: /home/houwaida/Mémoire/catkin_ws/src
Build space: /home/houwaida/Mémoire/catkin_ws/build
Devel space: /home/houwaida/Mémoire/catkin_ws/devel
Install space: /home/houwaida/Mémoire/catkin_ws/install
Creating symlink "/home/houwaida/Mémoire/catkin_ws/src/CMakeLists.txt" pointing
to "/opt/ros/noetic/share/catkin/cmake/toplevel.cmake"
####
#### Running command: "make cmake_check_build_system" in "/home/houwaida/Mémoire
/catkin_ws/build"
####
####
#### Running command: "make -j2 -l2" in "/home/houwaida/Mémoire/catkin_ws/build"
####
[ 0%] Built target _gazebo_msgs_generate_messages_check_deps_GetModelProperties
[ 0%] Built target geometry_msgs_generate_messages_py
[ 0%] Built target std_msgs_generate_messages_py
[ 0%] Built target _gazebo_msgs_generate_messages_check_deps_ODEPhysics
[ 0%] Built target _gazebo_msgs_generate_messages_check_deps_ModelStates
[ 0%] Built target _gazebo_msgs_generate_messages_check_deps_SensorPerformanceM
etric
[ 0%] Built target _gazebo_msgs_generate_messages_check_deps_ModelState
[ 0%] Built target _gazebo_msgs_generate_messages_check_deps_SetPhysicsProperti
```

FIGURE 3.18 – Résultat de l'exécution de la commande `catkin_make`



```
houwaida@ubuntu: ~/Mémoire/catkin_ws
vision_reconfigure.so
[ 96%] Built target vision_reconfigure
Scanning dependencies of target gazebo_ros_multicamera
[ 96%] Building CXX object gazebo_ros_pkgs/gazebo_plugins/CMakeFiles/gazebo_ros_multicamera.dir/src/gazebo_ros_multicamera.cpp.o
[ 97%] Linking CXX shared library /home/houwaida/Mémoire/catkin_ws/devel/lib/libgazebo_ros_prosilica.so
[ 97%] Built target gazebo_ros_prosilica
Scanning dependencies of target gazebo_ros_triggered_multicamera
[ 98%] Building CXX object gazebo_ros_pkgs/gazebo_plugins/CMakeFiles/gazebo_ros_triggered_multicamera.dir/src/gazebo_ros_triggered_multicamera.cpp.o
[ 98%] Linking CXX shared library /home/houwaida/Mémoire/catkin_ws/devel/lib/libgazebo_ros_multicamera.so
[ 98%] Built target gazebo_ros_multicamera
[ 98%] Linking CXX shared library /home/houwaida/Mémoire/catkin_ws/devel/lib/libgazebo_ros_triggered_multicamera.so
[ 98%] Built target gazebo_ros_triggered_multicamera
Scanning dependencies of target camera_synchronizer
[100%] Building CXX object gazebo_ros_pkgs/gazebo_plugins/CMakeFiles/camera_synchronizer.dir/src/camera_synchronizer.cpp.o
[100%] Linking CXX executable /home/houwaida/Mémoire/catkin_ws/devel/lib/gazebo_plugins/camera_synchronizer
[100%] Built target camera_synchronizer
houwaida@ubuntu:~/Mémoire/catkin_ws$
```

FIGURE 3.19 – Résultat de l'exécution de la commande catkin_make (suite)

4. **Déplacement du modèle dans « minilab_model » :** Le dossier « minilab_model » est l'un des dossiers existants dans le sous-dossier « src », on va déplacer notre modèle dans le dossier minilab_model\urdf

3.6 Simulation du robot avec le modèle réalisé

Le dossier minilab_simulation\launch contient les fichiers de lancement.

Le fichier de lancement « minilab_simulation.launch » définit un argument pour le nom de la map par défaut qui est « buvette » et il fait appel à deux autres fichiers de lancement « minilab_gazebo.launch » et « minilab_state_publisher.launch ».

- **minilab_gazebo.launch :** Le fichier « minilab_gazebo.launch » est un fichier de lancement utilisé dans la simulation de robot MiniLab avec Gazebo, en association avec ROS. Il commence par définir les paramètres essentiels, tels que l'utilisation du temps de simulation et le nom de la carte à charger. Ensuite, il inclut un autre fichier de lancement « empty_world.launch » pour initialiser un environnement vide, en configurant divers aspects tels que l'interface graphique et l'état de pause. Le fichier crée également une instance du modèle du robot à partir d'un fichier URDF, en spécifiant sa position et son orientation dans l'environnement. Enfin, il charge des configurations spécifiques à la carte sélectionnée, facilitant ainsi les tests et le développement du robot dans un cadre simulé.
- **minilab_state_publisher.launch :** Le fichier « minilab_state_publisher.launch » est un fichier de lancement utilisé pour publier l'état d'un robot dans un système ROS. Il commence par définir des arguments, notamment model, qui permet de spécifier le modèle de robot, et gui, qui détermine si une interface graphique doit être utilisée (valeur par défaut : False). Le fichier définit ensuite un paramètre robot_description, qui charge la description du robot à partir d'un fichier URDF situé dans le dossier « minilab_model ». Ce paramètre est essentiel pour que les nœuds suivants puissent comprendre la structure et les articulations du robot.

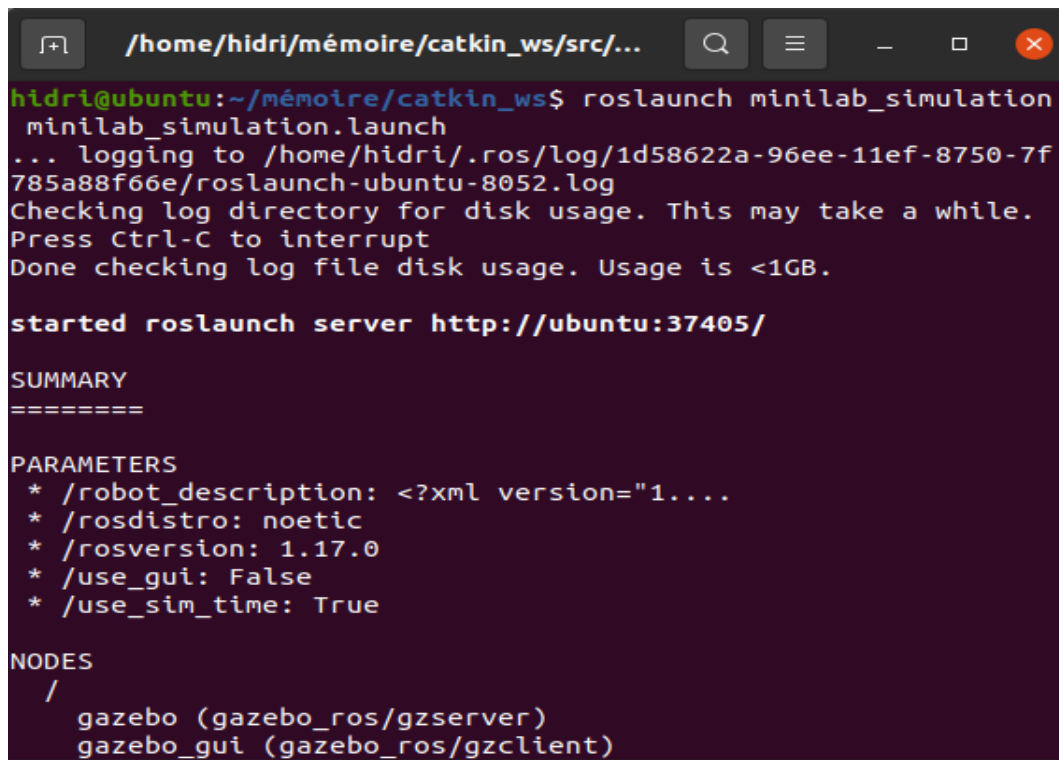
3.6. Simulation du robot avec le modèle réalisé

Deux nœuds sont ensuite lancés : le `joint_state_publisher`, qui publie l'état des articulations du robot, et le `robot_state_publisher`, qui diffuse la position et l'orientation du robot en fonction de l'état de ses articulations.

Dans le fichier « `buvette.launch` », on doit spécifier l'emplacement de notre modèle, La ligne de code inclut un nœud nommé `spawn_sdf`, qui utilise le package `gazebo_ros` pour créer une instance du modèle dans l'environnement Gazebo. Les arguments fournis définissent la position du modèle.

vous pouvez lancer le fichier de lancement à travers la commande `roslaunch minilab_simulation minilab_simulation.launch` ce qui est illustré par la Figure 3.20.

Explication de la commande : `roslaunch` est une commande utilisée pour gérer le lancement et l'arrêt des nœuds ros, il prend en paramètre le nom du package à laquelle appartient le fichier + le nom de fichier de lancement avec l'extension `.launch` dans notre cas `minilab_simulation` est le nom de package qui contient le fichier « `minilab_simulation.launch` ».



```
hidri@ubuntu:~/mémoire/catkin_ws/src/...$ roslaunch minilab_simulation
minilab_simulation.launch
... logging to /home/hidri/.ros/log/1d58622a-96ee-11ef-8750-7f
785a88f66e/roslaunch-ubuntu-8052.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://ubuntu:37405/

SUMMARY
=====

PARAMETERS
* /robot_description: <?xml version="1....
* /rostdistro: noetic
* /rosversion: 1.17.0
* /use_gui: False
* /use_sim_time: True

NODES
/
  gazebo (gazebo_ros/gzserver)
  gazebo_gui (gazebo_ros/gzclient)
```

FIGURE 3.20 – Lancement du fichier de simulation avec `roslaunch`

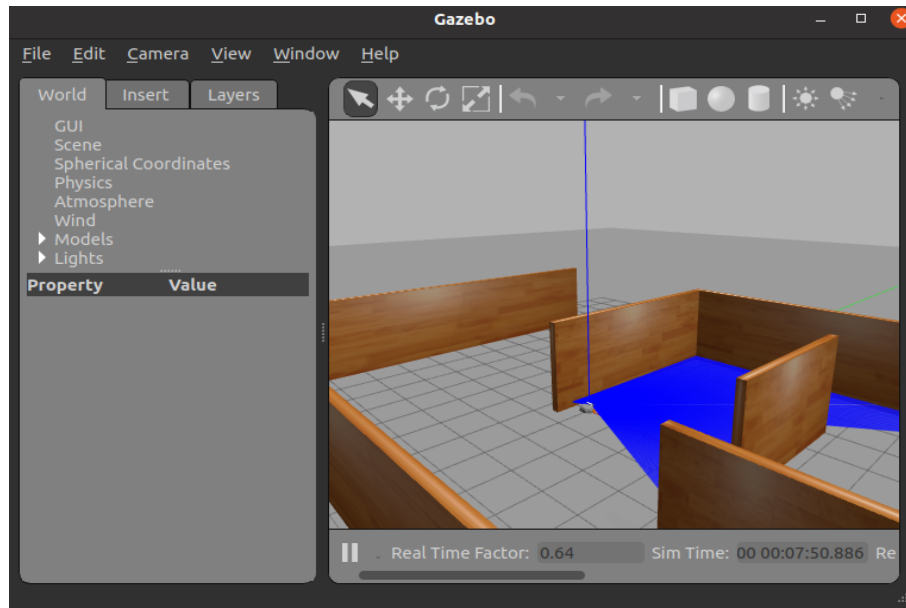


FIGURE 3.21 – Résultat de la simulation

3.7 Architecture et fonctionnement des nœuds ROS

Il est essentiel de comprendre l'architecture et le fonctionnement des différents nœuds qui composent notre projet. Cette section se concentre sur deux éléments clés de l'architecture ROS : les nœuds actifs et les nœuds topic. Nous examinerons comment ces composants interagissent pour permettre la communication et la coordination au sein du robot autonome étudié, formant ainsi l'épine dorsale de son système de contrôle et de prise de décision.

3.7.1 Nœuds actifs

Lors de l'exécution du fichier de lancement de projet avec la commande `roslaunch`, plusieurs informations s'affichent. Parmi ces informations, on trouve la liste des nœuds actifs, comme illustré par la Figure 3.22

Chaque nœud a un fonctionnement spécifique :

- **gazebo (gazebo_ros/gzserver)**
Le nœud `gzserver` est le moteur de simulation de Gazebo. Il gère l'exécution de la simulation, publie les états des objets dans l'environnement simulé, et reçoit des commandes de ROS pour contrôler les robots et les environnements.
- **gazebo_gui (gazebo_ros/gzclient)**
Le nœud `gzclient` est l'interface graphique de Gazebo. Il permet aux utilisateurs d'interagir avec la simulation à travers une interface 3D, visualiser les robots et les environnements, et contrôler les paramètres de simulation en temps réel.
- **joint_state_publisher (joint_state_publisher/joint_state_publisher)**
Le nœud `joint_state_publisher` publie l'état des articulations d'un robot. Il permet de simuler le mouvement des joints en fournissant des informations sur les positions, vitesses, et efforts des articulations à d'autres nœuds ROS.

```

NODES
/
gazebo (gazebo_ros/gzserver)
gazebo_gui (gazebo_ros/gzclient)
joint_state_publisher (joint_state_publisher/joint_state_publisher)
robot_state_publisher (robot_state_publisher/robot_state_publisher)
spawn_robot1 (gazebo_ros/spawn_model)
spawn_sdf (gazebo_ros/spawn_model)

auto-starting new master
process[master]: started with pid [8060]
ROS_MASTER_URI=http://localhost:11311

setting /run_id to 1d58622a-96ee-11ef-8750-7f785a88f66e
process[rosout-1]: started with pid [8070]
started core service [/rosout]
process[gazebo-2]: started with pid [8073]
process[gazebo_gui-3]: started with pid [8079]
process[spawn_robot1-4]: started with pid [8087]
process[spawn_sdf-5]: started with pid [8088]
process[joint_state_publisher-6]: started with pid [8089]
process[robot_state_publisher-7]: started with pid [8092]
[spawn_sdf-5] process has finished cleanly
log file: /home/hidri/.ros/log/1d58622a-96ee-11ef-8750-7f785a88f66e/spa

```

FIGURE 3.22 – Liste des nœuds actifs

- **robot_state_publisher (robot_state_publisher/robot_state_publisher)**

Le nœud robot_state_publisher prend les informations sur les articulations d'un robot et les transforme en un état global du robot. Il publie la position et l'orientation de chaque lien du robot sur des topics ROS, ce qui permet à d'autres nœuds de connaître la configuration du robot.

- **spawn_robot1 (gazebo_ros/spawn_model)**

Le nœud spawn_model permet d'ajouter des modèles de robots ou d'objets dans l'environnement de simulation de Gazebo. Il reçoit des commandes contenant les spécifications du modèle et le positionne dans la simulation.

- **spawn_sdf (gazebo_ros/spawn_model)**

Similaire à spawn_model, le nœud spawn_sdf permet de charger des modèles définis par le format SDF (Simulation Description Format) dans la simulation Gazebo. Il permet d'intégrer des objets avec des propriétés et comportements spécifiques.

3.7.2 Topics actifs

Pour obtenir une liste exhaustive des topics actifs dans le système ROS, il convient d'utiliser la commande suivante dans le terminal : **rostopic list**

La liste des topics actifs relatifs est illustrée par la Figure 3.23

- **Les topics /IR1 .. /IR5**

Les topics /IR1, /IR2, /IR3, /IR4, et /IR5 sont liés aux informations sur les obstacles détectés par le robot. Ils reçoivent des données publiées par le nœud scan, qui utilise des capteurs infrarouges pour mesurer la distance entre le robot et les obstacles environnants.

- **Le topic /cmd_vel**

Le topic /cmd_vel contient des messages de type geometry_msgs/Twist, qui spécifient la vitesse linéaire et angulaire du robot.

- **/odom**

Le topic /odom joue un rôle crucial dans la navigation des robots mobiles utilisant ROS (Robot Operating System). Il fournit des informations détaillées sur l'état cinématique du robot, essentielles pour la localisation et le contrôle du mouvement.

Le topic /odom publie les informations suivantes :

- **Position**

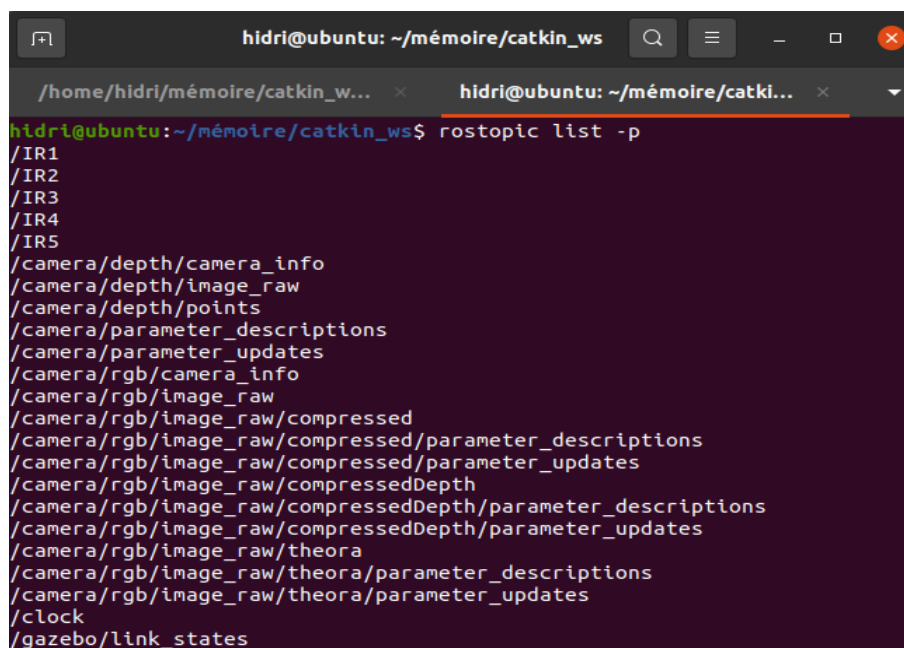
- **Position en X** : Coordonnée horizontale du robot dans le plan
- **Position en Y** : Coordonnée verticale du robot dans le plan
- **Position en Z** : Hauteur du robot (pertinent dans les environnements 3D)

- **Orientation**

Quaternion (x, y, z, w) : Représentation de l'orientation du robot sans ambiguïté, évitant les problèmes de blocage.

- **Vitesse**

- **Vitesse linéaire** : Vecteur tridimensionnel (v_x, v_y, v_z) indiquant la vitesse et la direction du déplacement du robot selon chaque axe.
- **Vitesse angulaire** : Vecteur tridimensionnel (w_x, w_y, w_z) représentant la vitesse de rotation du robot autour de chaque axe.



```
hidri@ubuntu: ~/mémoire/catkin_ws
/home/hidri/mémoire/catkin_w... x hidri@ubuntu: ~/mémoire/catki... x
hidri@ubuntu:~/mémoire/catkin_ws$ rostopic list -p
/IR1
/IR2
/IR3
/IR4
/IR5
/camera/depth/camera_info
/camera/depth/image_raw
/camera/depth/points
/camera/parameter_descriptions
/camera/parameter_updates
/camera/rgb/camera_info
/camera/rgb/image_raw
/camera/rgb/image_raw/compressed
/camera/rgb/image_raw/compressed/parameter_descriptions
/camera/rgb/image_raw/compressed/parameter_updates
/camera/rgb/image_raw/compressedDepth
/camera/rgb/image_raw/compressedDepth/parameter_descriptions
/camera/rgb/image_raw/compressedDepth/parameter_updates
/camera/rgb/image_raw/theora
/camera/rgb/image_raw/theora/parameter_descriptions
/camera/rgb/image_raw/theora/parameter_updates
/clock
/gazebo/link_states
```

FIGURE 3.23 – Liste des Topics

3.8 Conclusion

Dans ce chapitre, nous avons présenté le robot MiniLab et mis en place son environnement de simulation sous Gazebo. Nous avons détaillé les étapes de configuration de l'environnement de travail, la construction du modèle du robot et son intégration dans la simulation. L'architecture ROS développée permet désormais de contrôler et de simuler efficacement le comportement du robot dans son environnement virtuel. Cette base solide nous permettra, dans le chapitre suivant, d'implémenter et de tester différents algorithmes de planification de trajectoire pour assurer une navigation autonome du robot.

Étude et implémentation d'algorithmes de planification de trajectoire pour la navigation autonome

Sommaire

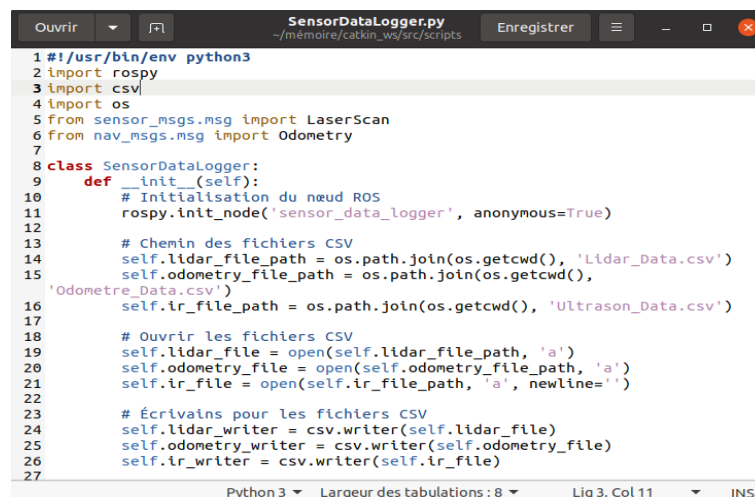
4.1	Introduction	57
4.2	Acquisition et traitement des données à partir des topics ROS	57
4.3	Analyse de l'algorithme de navigation existant	58
4.3.1	Principes fondamentaux de l'algorithme de planification de la fenêtre dynamique	59
4.3.2	Fonctionnement de l'algorithme de la fenêtre dynamique (DWA)	59
4.3.3	Avantages et limites de l'algorithme de la fenêtre dynamique	61
4.3.4	Applications du DWA	61
4.4	Implémentation de l'algorithme Q Learning	62
4.4.1	Introduction au Q Learning	62
4.4.2	Représentation de l'environnement	62
4.4.3	Fonction goal_reached	64
4.4.4	Fonction Q	65
4.4.5	Algorithme d'apprentissage	66
4.5	Comparaison	67
4.5.1	Méthodologie	67
4.5.2	Réactivité	67
4.5.3	Planification à Long Terme	68
4.5.4	Gestion des Objets Dynamiques	68
4.5.5	Complexité et Implémentation	68
4.5.6	Robustesse	68
4.6	Validation expérimentale	68
4.7	Conclusion	70

4.1 Introduction

Ce chapitre présente une étude approfondie de l'algorithme de navigation existant, le Dynamic Window Approach (DWA), ainsi que notre implémentation d'un algorithme d'apprentissage par renforcement, le Q-Learning. Tout d'abord, nous abordons l'acquisition et le traitement des données à partir des topics ROS. Ensuite, nous analysons en détail le fonctionnement du DWA, ses avantages et ses limites. Nous expliquons ensuite notre propre implémentation de l'algorithme de Q-Learning, en décrivant la représentation de l'environnement, les fonctions clés ainsi que l'algorithme d'apprentissage lui-même. Une comparaison détaillée entre les deux approches est ensuite proposée, en termes de réactivité, de planification à long terme, de gestion des objets dynamiques, de complexité et de robustesse. Enfin, nous présentons les résultats de la validation expérimentale de notre approche par Q-Learning et évoquons quelques perspectives d'amélioration.

4.2 Acquisition et traitement des données à partir des topics ROS

Le fichier SensorDataLogger.py, illustré par la Figure 4.1, est un script Python conçu pour acquérir et traiter les données provenant de divers topics. Il initialise un nœud ROS nommé `sensor_data_logger` et s'abonne à plusieurs topics, notamment `/scan` pour les données LiDAR, `/odom` pour les informations d'odométrie, et cinq topics IR pour les capteurs ultrasoniques. Chaque fois qu'un message est reçu, le script extrait les informations pertinentes, telles que la position, l'orientation et les distances entre le robot et les obstacles, et les enregistre dans des fichiers CSV distincts : `Lidar_Data.csv` pour les données LiDAR, `Odometre_Data.csv` pour les informations d'odométrie, et `Ultrason_Data.csv` pour les valeurs des capteurs IR. Les données LiDAR sont stockées avec les angles minimum et maximum ainsi que les distances mesurées, tandis que les données d'odométrie incluent des informations complètes sur la position, l'orientation et la vitesse angulaire et linéaire sur les différents axes pour le robot. Ce système d'acquisition permet une collecte structurée et efficace des données, essentielles pour l'analyse ultérieure et le développement d'algorithmes de navigation.



```

1#!/usr/bin/env python3
2import rospy
3import csv
4import os
5from sensor_msgs.msg import LaserScan
6from nav_msgs.msg import Odometry
7
8class SensorDataLogger:
9    def __init__(self):
10        # Initialisation du nœud ROS
11        rospy.init_node('sensor_data_logger', anonymous=True)
12
13        # Chemin des fichiers CSV
14        self.lidar_file_path = os.path.join(os.getcwd(), 'Lidar_Data.csv')
15        self.odometry_file_path = os.path.join(os.getcwd(),
16        'Odometre_Data.csv')
17        self.ir_file_path = os.path.join(os.getcwd(), 'Ultrason_Data.csv')
18
19        # Ouvrir les fichiers CSV
20        self.lidar_file = open(self.lidar_file_path, 'a')
21        self.odometry_file = open(self.odometry_file_path, 'a')
22        self.ir_file = open(self.ir_file_path, 'a', newline='')
23
24        # Écrivains pour les fichiers CSV
25        self.lidar_writer = csv.writer(self.lidar_file)
26        self.odometry_writer = csv.writer(self.odometry_file)
27        self.ir_writer = csv.writer(self.ir_file)
28

```

FIGURE 4.1 – Architecture du script SensorDataLogger pour l'acquisition des données

4.3 Analyse de l'algorithme de navigation existant

À partir de l'analyse des fichiers existants dans le dossier `minilab_navigation\config` (`costmap_params.yaml`, `global_costmap_params.yaml` et `move_base_params.yaml`) du robot minilab, il est évident que celui-ci utilise l'algorithme de navigation autonome appelée l'approche de la fenêtre dynamique (Dynamic Window Approach). Cet algorithme est conçu pour permettre au robot de naviguer efficacement dans son environnement tout en évitant les obstacles.

Dans cette section, nous examinerons en détails les principes fondamentaux de DWA, ses avantages, ses limites, ainsi que ses différentes applications.

- **costmap_params.yaml** : Ce fichier de configuration « `costmap_params.yaml` » définit les paramètres utilisés par le module de cartographie des coûts (Costmap) dans le système de navigation de la plateforme robotique. Les paramètres clés incluent le cadre de référence global « `global_frame : odom` » et le cadre de référence de base du robot « `robot_base_frame : base_link` ».

Le fichier configure deux plugins principaux : « `ObstacleLayer` » pour détecter les obstacles à l'aide des données laser, et « `InflationLayer` » pour calculer une zone tampon autour des obstacles afin d'éviter les collisions. Des paramètres tels que la fréquence de mise à jour, la taille et la résolution de la carte, ainsi que la définition du footprint du robot, sont également spécifiés.

- **global_costmap_params.yaml** : Le fichier `global_costmap_params.yaml` définit les paramètres de la carte de coûts pour un robot dans un environnement donné. Il spécifie que le cadre global est la carte (`map`) et que le cadre de base du robot est `base_link`. La fréquence de mise à jour est fixée à 6.0 Hz, et la carte statique est désactivée.

Trois plugins sont utilisés : le `StaticLayer` pour la carte statique, le `ObstacleLayer` pour gérer les obstacles détectés, et le `InflationLayer` pour ajuster les coûts autour des obstacles. Le plugin `ObstacleLayer` s'appuie sur des données provenant d'un capteur de type `LaserScan`, avec des paramètres spécifiques tels que la portée des obstacles et la hauteur minimale et maximale des obstacles détectables. Enfin, le `InflationLayer` applique un rayon d'inflation de 0.2 mètres, influençant la manière dont les obstacles sont pris en compte dans la navigation du robot.

- **move_base_params.yaml** : Le fichier `move_base_params.yaml` contient des paramètres essentiels pour la navigation du robot, notamment en utilisant l'algorithme Dynamic Window Approach (DWA). Ce dernier est implémenté à travers les paramètres du planificateur de trajectoire, spécifiquement sous la section `TrajectoryPlannerROS`. Les paramètres tels que `max_vel_x`, `min_vel_x`, `max_vel_th`, et `min_vel_th` définissent les limites de vitesse linéaire et angulaire, permettant au robot de naviguer de manière fluide et sécurisée.

De plus, les limites d'accélération (`acc_lim_x`, `acc_lim_y`, `acc_lim_theta`) sont configurées pour garantir une réponse dynamique adéquate aux changements de direction et de vitesse, caractéristiques fondamentales de l'approche dynamique. Le robot est également configuré pour gérer les tolérances de but (`yaw_goal_tolerance`, `xy_goal_tolerance`) et les comportements de récupération en cas d'obstacles ou de situations imprévues. Ces éléments combinés permettent au robot d'effectuer des manœuvres précises et adaptatives en utilisant DWA pour optimiser sa trajectoire vers l'objectif, tout en maintenant une navigation sûre et efficace.

4.3.1 Principes fondamentaux de l'algorithme de planification de la fenêtre dynamique

4.3.1.1 Définition

L'algorithme de planification de trajectoire locale, connu sous le nom de méthode de la fenêtre dynamique, a été proposé par Dieter Fox et al. en 1997. Cette approche est principalement utilisée par le système ROS pour les robots mobiles. La méthode de la fenêtre dynamique permet d'éviter les obstacles en temps réel [16] en opérant dans l'espace des commandes du robot, ce qui lui confère une grande réactivité face à des environnements dynamiques. Contrairement à d'autres algorithmes de planification qui nécessitent une connaissance préalable de l'environnement, cette méthode s'adapte aux changements imprévisibles, comme le mouvement ou l'apparition soudaine d'obstacles. En tenant compte des contraintes cinématiques et dynamiques du robot, la fenêtre dynamique optimise les commandes de mouvement, assurant ainsi une navigation fluide et sécurisée, même à des vitesses élevées. Cette flexibilité la rend particulièrement efficace pour les applications où la sécurité et la réactivité sont primordiales.

4.3.1.2 Objectif principal de l'algorithme

L'objectif principal du DWA est de permettre à un robot mobile de naviguer de manière autonome dans un environnement dynamique tout en évitant les obstacles et en atteignant son but. Plus spécifiquement, l'algorithme vise à

- Générer des trajectoires sûres et efficaces en temps réel.
- Prendre en compte les contraintes cinématiques et dynamiques du robot.
- Optimiser le mouvement du robot en fonction de critères multiples (sécurité, vitesse, progression vers l'objectif).
- S'adapter rapidement aux changements dans l'environnement.

4.3.1.3 Concept de la fenêtre dynamique

La fenêtre dynamique est un point central de l'algorithme, elle représente :

- Un sous-espace de l'espace des vitesses (linéaires et angulaires) du robot.
- Les vitesses que le robot peut atteindre de manière sûre dans un court intervalle de temps (typiquement la prochaine période de contrôle).
- Une prise en compte des limitations physiques du robot, telles que ses accélérations maximales.

4.3.2 Fonctionnement de l'algorithme de la fenêtre dynamique (DWA)

- **Définir l'état actuel de robot :** Le robot commence par déterminer sa position actuelle, sa vitesse et son orientation. Ces informations sont essentielles pour calculer les mouvements possibles.
- **Calculer la fenêtre dynamique :** A partir de l'état actuel, une fenêtre dynamique est construite. Cette fenêtre représente toutes les vitesses et directions que le robot peut atteindre dans un court laps de temps, en tenant compte des contraintes physiques telles que l'accélération maximale et la vitesse maximale.
- **Générer des trajectoires candidates :** À l'intérieur de cette fenêtre dynamique, plusieurs trajectoires possibles sont générées. Ces trajectoires sont généralement circulaires et doivent être atteignables dans le temps imparti, sans entrer en collision avec des obstacles.

4.3. Analyse de l'algorithme de navigation existant

- **Évaluer les trajectoires** : chaque trajectoire candidate est évaluée en fonction de critères tels que :
 - Proximité des obstacles : s'assurer que la trajectoire choisie maintient une distance sécuritaire par rapport aux obstacles.
 - Coût de la trajectoire : minimiser le temps de parcours ou l'énergie dépensée, ce qui peut être formulé comme une fonction de coût à minimiser.
- Sélectionner la meilleure trajectoire : Après évaluation, l'algorithme choisit la trajectoire qui offre le meilleur compromis entre sécurité (distance aux obstacles) et efficacité (temps en énergie).
- Appliquer le mouvement : une fois la meilleure trajectoire sélectionnée, le robot applique les commandes correspondantes pour se déplacer vers la nouvelle position
- Répéter le processus : Le processus est itératif. Une fois que le robot a avancé vers sa nouvelle position, il répète ces étapes en recalculant sa fenêtre dynamique et en générant de nouvelles trajectoires, s'adaptant ainsi continuellement aux changements dans l'environnement.

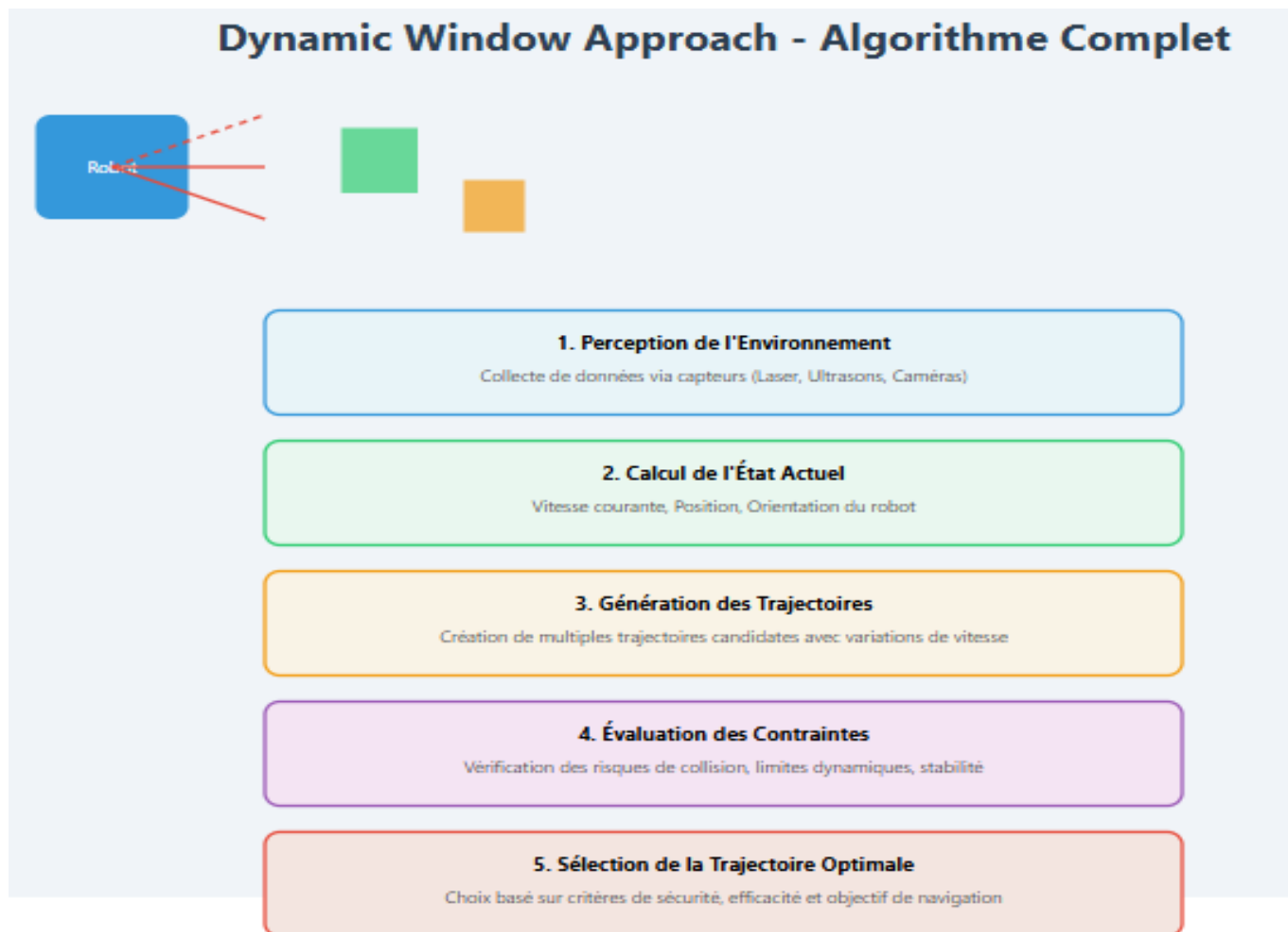


FIGURE 4.2 – Algorithme de la fenêtre dynamique (DWA)

4.3.3 Avantages et limites de l'algorithme de la fenêtre dynamique

L'algorithme DWA présente plusieurs avantages qui en font un choix privilégié pour la navigation autonome. Tout d'abord, sa réactivité élevée permet au robot de s'adapter rapidement aux changements dans l'environnement, tels que l'apparition d'obstacles. DWA évalue en temps réel les trajectoires possibles, tenant compte des dynamiques spécifiques du robot, ce qui contribue à une navigation fluide et naturelle.

De plus, sa simplicité d'implémentation le rend accessible à de nombreux projets. L'algorithme équilibre efficacement l'exploration de nouvelles trajectoires et l'exploitation des meilleures options disponibles, tout en gérant les obstacles de manière proactive, ce qui améliore la sécurité et la robustesse de la navigation. Ces caractéristiques font de DWA une solution puissante pour des environnements complexes et dynamiques.

Cependant, malgré ses nombreux avantages, le DWA présente certaines limites qui peuvent affecter sa performance en navigation autonome. L'une des principales préoccupations est sa sensibilité à la localisation ; des erreurs dans la position du robot peuvent conduire à des décisions de navigation erronées. De plus, DWA peut parfois converger vers des solutions sous-optimales, en particulier dans des environnements complexes où de nombreux obstacles sont présents. L'algorithme éprouve également des difficultés à gérer les objets dynamiques, ce qui peut accroître le risque de collisions. Dans des situations où la densité d'obstacles est élevée, DWA peut devenir inefficace, peinant à trouver des trajectoires viables. Enfin, le choix des paramètres, tels que la taille de la fenêtre dynamique, influence fortement les performances de l'algorithme, rendant son paramétrage délicat. En outre, DWA se concentre principalement sur des décisions à court terme, ce qui limite sa capacité à planifier des trajectoires sur les longues distances.

4.3.4 Applications du DWA

Le Dynamic Window Approach (DWA) est largement utilisé dans divers domaines de la robotique

- **Robots Mobiles Autonomes :** DWA est fréquemment utilisé dans les robots mobiles autonomes, tels que les robots de service, qui naviguent dans des espaces publics ou domestiques. Sa capacité à éviter les obstacles en temps réel en fait un choix idéal pour des environnements imprévisibles.
- **Véhicules Autonomes :** Dans le domaine des véhicules autonomes, DWA est utilisé pour la planification de trajectoires, permettant aux véhicules de naviguer en toute sécurité tout en évitant les obstacles et en respectant les règles de circulation.
- **Drones et Robots Aériens :** Les drones et robots aériens utilisent DWA pour la navigation dans des environnements complexes, comme les zones urbaines ou les paysages naturels, où la détection et l'évitement des obstacles sont cruciaux.
- **Simulations de Trafic :** DWA peut être appliqué dans des simulations de trafic pour modéliser le comportement des véhicules dans des environnements urbains, permettant d'étudier les interactions entre véhicules et piétons.
- **Jeux Vidéo et Simulation :** Dans le développement de jeux vidéo, DWA est utilisé pour la navigation des personnages non-joueurs (PNJ), offrant des mouvements réalistes et réactifs dans des environnements complexes.
- **Planification Urbaine et Aménagement du Territoire :** Dans la planification urbaine, DWA peut être utilisé pour modéliser les flux de circulation et évaluer l'impact de nouvelles infrastructures sur la mobilité urbaine.

4.4 Implémentation de l'algorithme Q Learning

4.4.1 Introduction au Q Learning

Le Q-learning [13] est une méthode d'apprentissage par renforcement qui permet à un agent, tel qu'un robot mobile, d'apprendre à naviguer dans un environnement en prenant des décisions optimales. Cette approche repose sur un processus itératif d'essais et d'erreurs, où l'agent interagit avec son environnement et collecte des informations sur les conséquences de ses actions. À chaque interaction, l'agent reçoit un signal de renforcement qui lui indique la qualité de l'action entreprise en fonction de l'état actuel.

En utilisant ces retours, l'agent apprend à associer des actions spécifiques à des états particuliers, dans le but de maximiser ses récompenses à long terme. Ce mécanisme d'apprentissage permet à l'agent de développer une politique optimale, qui est une stratégie définissant quelle action prendre dans chaque état. Le Q-learning est particulièrement apprécié pour sa capacité à fonctionner dans des environnements à états discrets ou continus, rendant cette méthode adaptable à des scénarios complexes de navigation.

En intégrant des techniques comme le Q-learning flou, il est possible d'améliorer encore les performances des robots en combinant les avantages de l'apprentissage par renforcement avec des systèmes d'inférence floue. Cela permet de gérer des situations où l'information est incertaine ou imprécise, tout en préservant l'efficacité du processus d'apprentissage.

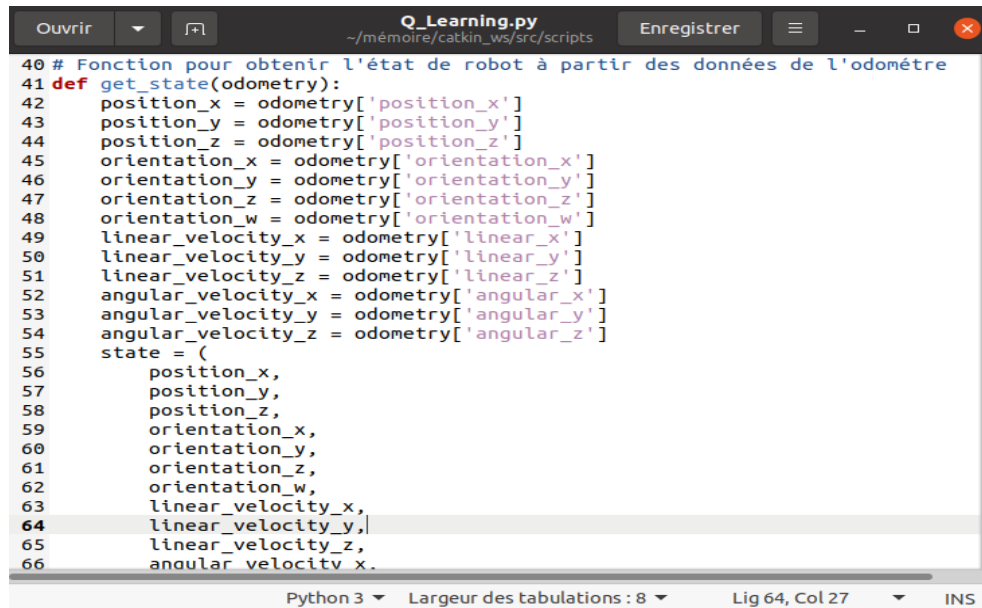
4.4.2 Représentation de l'environnement

Pour que le robot puisse naviguer efficacement, l'environnement doit être représenté sous forme d'un espace d'états. Chaque état correspond à une position spécifique dans cet environnement, en tenant compte des coordonnées x et y ainsi que de l'orientation du robot. Les obstacles présents jouent un rôle crucial, car ils influencent les transitions entre les états. Par exemple, certaines positions peuvent être inaccessibles en raison de la présence d'obstacles, ce qui limite les actions que le robot peut entreprendre, telles qu'avancer, reculer ou tourner. Ainsi, l'algorithme doit apprendre à éviter ces obstacles tout en cherchant à atteindre son objectif de manière optimale. Cela nécessite une modélisation précise des interactions entre les états, les actions possibles et les obstacles. Les fonctions ci-dessus sont utilisées pour récupérer les informations nécessaires à cette représentation :

4.4.2.1 Fonction `get_state`

Cette fonction illustrée par la Figure 4.3 permet d'extraire les données de l'odomètre à partir d'un fichier `odometre_Data.csv`. Elle récupère des informations telles que la position (x, y, z) , l'orientation (x, y, z, w) et les vitesses linéaires et angulaires du robot. Ces données sont encapsulées dans un tuple, représentant l'état actuel du robot dans l'environnement.

4.4. Implémentation de l'algorithme Q Learning

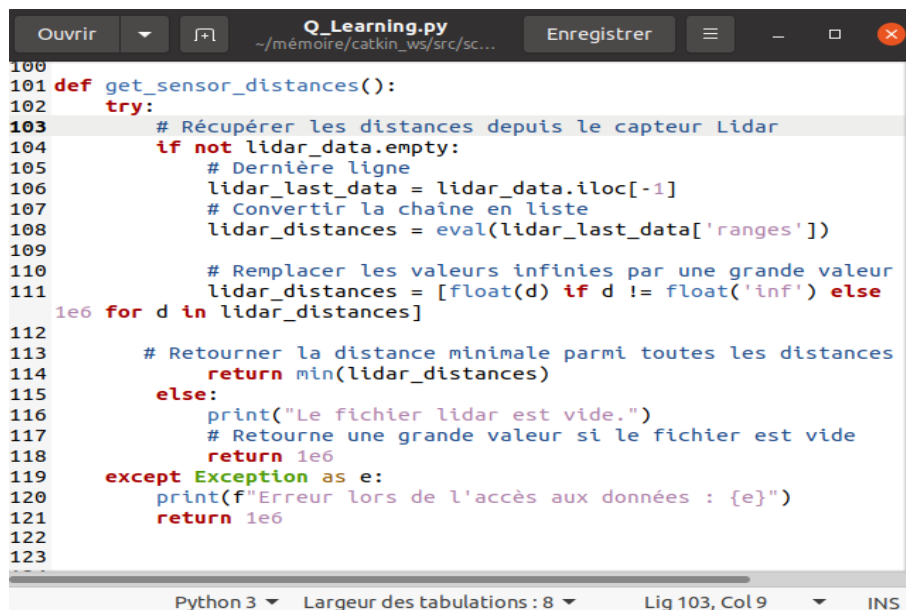


```
40 # Fonction pour obtenir l'état de robot à partir des données de l'odomètre
41 def get_state(odometry):
42     position_x = odometry['position_x']
43     position_y = odometry['position_y']
44     position_z = odometry['position_z']
45     orientation_x = odometry['orientation_x']
46     orientation_y = odometry['orientation_y']
47     orientation_z = odometry['orientation_z']
48     orientation_w = odometry['orientation_w']
49     linear_velocity_x = odometry['linear_x']
50     linear_velocity_y = odometry['linear_y']
51     linear_velocity_z = odometry['linear_z']
52     angular_velocity_x = odometry['angular_x']
53     angular_velocity_y = odometry['angular_y']
54     angular_velocity_z = odometry['angular_z']
55     state = (
56         position_x,
57         position_y,
58         position_z,
59         orientation_x,
60         orientation_y,
61         orientation_z,
62         orientation_w,
63         linear_velocity_x,
64         linear_velocity_y,
65         linear_velocity_z,
66         angular_velocity_x,
```

FIGURE 4.3 – Extraction des données de l'odomètre pour la représentation de l'état du robot

4.4.2.2 Fonction get_sensor_distances

La fonction `get_sensor_distances()` illustrée par la Figure 4.4 permet de récupérer les distances mesurées par le capteur Lidar. Elle commence par vérifier si des données sont disponibles dans le DataFrame `lidar_data`. Si c'est le cas, elle extrait la dernière ligne et convertit la chaîne de distances en liste, remplaçant les valeurs infinies par une grande valeur ($1e6$) pour éviter des erreurs. La fonction retourne ensuite la distance minimale parmi les valeurs mesurées. En cas de fichier vide ou d'erreur, elle imprime un message et renvoie également une grande valeur, garantissant ainsi une estimation fiable de la distance aux obstacles.



```
100
101 def get_sensor_distances():
102     try:
103         # Récupérer les distances depuis le capteur Lidar
104         if not lidar_data.empty:
105             # Dernière ligne
106             lidar_last_data = lidar_data.iloc[-1]
107             # Convertir la chaîne en liste
108             lidar_distances = eval(lidar_last_data['ranges'])
109
110             # Remplacer les valeurs infinies par une grande valeur
111             lidar_distances = [float(d) if d != float('inf') else
112                               1e6 for d in lidar_distances]
113
114             # Retourner la distance minimale parmi toutes les distances
115             return min(lidar_distances)
116         else:
117             print("Le fichier lidar est vide.")
118             # Retourne une grande valeur si le fichier est vide
119             return 1e6
120     except Exception as e:
121         print(f"Erreur lors de l'accès aux données : {e}")
122         return 1e6
123
```

FIGURE 4.4 – Récupération de la distance entre le robot et l'obstacle

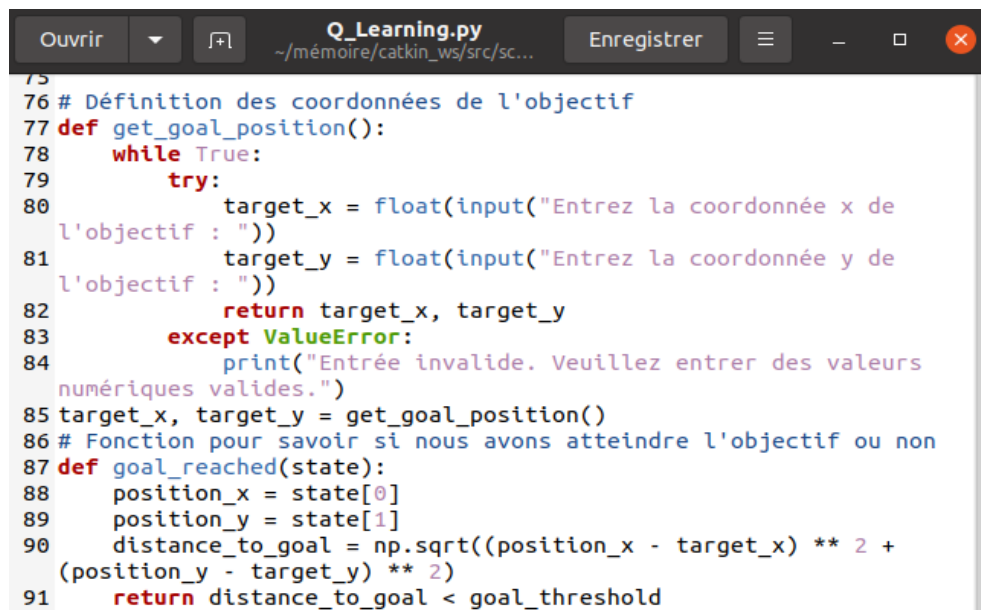
4.4.2.3 Fonction `get_goal_position`

Cette fonction définit les coordonnées de l'objectif que le robot doit atteindre. Elle demande à l'utilisateur de saisir les valeurs pour les coordonnées x et y, ce qui permet de déterminer la destination du robot dans l'environnement.

4.4.3 Fonction `goal_reached`

Cette fonction détermine si le robot a atteint l'objectif en calculant la distance entre sa position actuelle et celle de l'objectif. Si cette distance est inférieure au seuil (`goal_threshold`), elle retourne `True`, indiquant que l'objectif a été atteint.

Ces dernières fonctions sont représentées par la Figure 4.5



```
75
76 # Définition des coordonnées de l'objectif
77 def get_goal_position():
78     while True:
79         try:
80             target_x = float(input("Entrez la coordonnée x de
l'objectif : "))
81             target_y = float(input("Entrez la coordonnée y de
l'objectif : "))
82             return target_x, target_y
83         except ValueError:
84             print("Entrée invalide. Veuillez entrer des valeurs
numériques valides.")
85 target_x, target_y = get_goal_position()
86 # Fonction pour savoir si nous avons atteint l'objectif ou non
87 def goal_reached(state):
88     position_x = state[0]
89     position_y = state[1]
90     distance_to_goal = np.sqrt((position_x - target_x) ** 2 +
(position_y - target_y) ** 2)
91     return distance_to_goal < goal_threshold
```

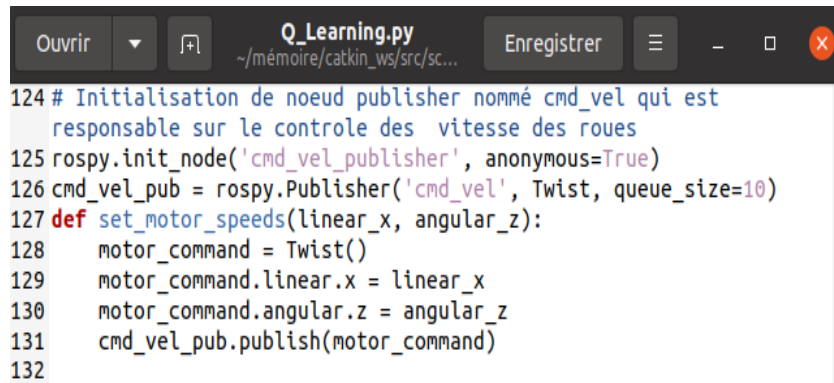
FIGURE 4.5 – Fonctions pour la définition de l'objectif et la vérification de l'atteinte de l'objectif par le robot

4.4.3.1 Initialisation du Publisher `cmd_vel`

Les deux premières lignes de la Figure 4.6 initialisent le nœud nommé `cmd_vel_publisher`, qui est responsable de la publication des commandes de vitesse pour les roues du robot. ce nœud est utilisé pour contrôler le mouvement du robot en envoyant des messages de type `Twist`.

4.4.3.2 Fonction `set_motor_speeds`

Cette fonction permet de définir les vitesses linéaires et angulaires du robot. Elle crée un message de type `Twist` pour encapsuler ces vitesses, puis publie ce message via le publisher `cmd_vel`.

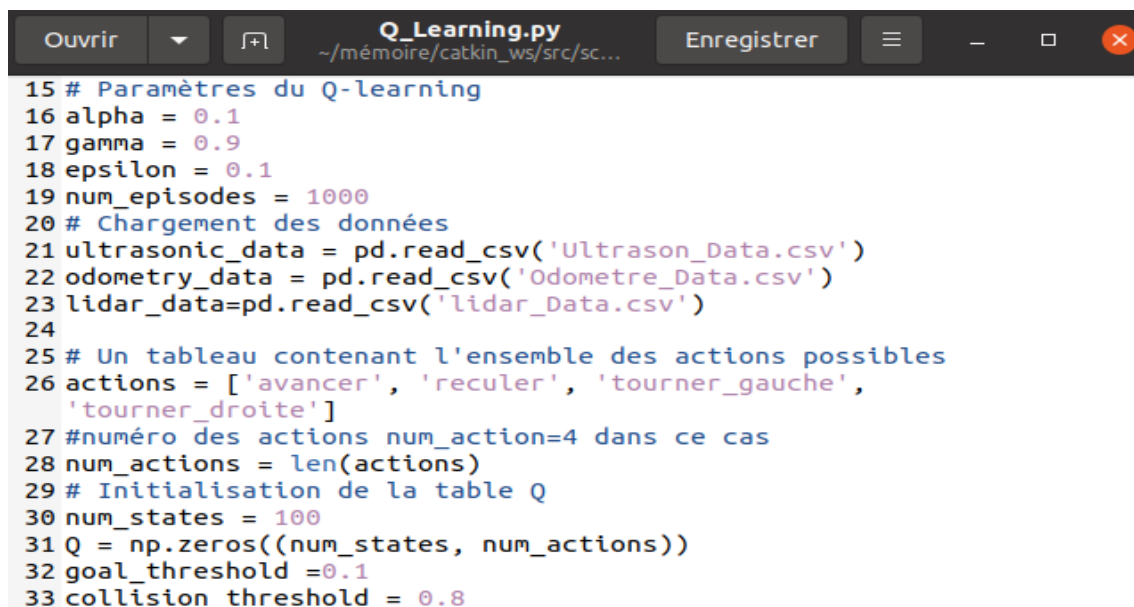


```
124 # Initialisation de noeud publisher nommé cmd_vel qui est
    responsable sur le controle des vitesses des roues
125 rospy.init_node('cmd_vel_publisher', anonymous=True)
126 cmd_vel_pub = rospy.Publisher('cmd_vel', Twist, queue_size=10)
127 def set_motor_speeds(linear_x, angular_z):
128     motor_command = Twist()
129     motor_command.linear.x = linear_x
130     motor_command.angular.z = angular_z
131     cmd_vel_pub.publish(motor_command)
132
```

FIGURE 4.6 – Contrôle des vitesses du robot

4.4.4 Fonction Q

La fonction Q, notée $Q(s, a)$, représente la valeur d'une action a dans un état s , c'est-à-dire la récompense attendue si l'agent choisit cette action. Lors de l'implémentation, nous avons initialisé la fonction Q à zéro. Pour cela, nous avons créé un tableau contenant l'ensemble des actions possibles : ['avancer', 'reculer', 'tourner_gauche', 'tourner_droite'], ce qui nous donne un total de 4 actions ($\text{num_actions} = 4$). Les paramètres du Q-learning ont été définis comme suit : un taux d'apprentissage (α) de 0.1, un facteur de discount (γ) de 0.9, et un taux d'exploration (ϵ) de 0.1. Nous avons prévu d'exécuter 1000 épisodes pour permettre au robot d'explorer son environnement. Pour gérer les états, nous avons initialisé la table Q, qui contient les actions et les états de l'environnement, à zéro, en considérant un total de 100 états ($\text{num_states} = 100$). Cette initialisation nous permettra de mettre à jour les valeurs de Q au fur et à mesure que le robot apprend des interactions avec son environnement (Figure 4.7).



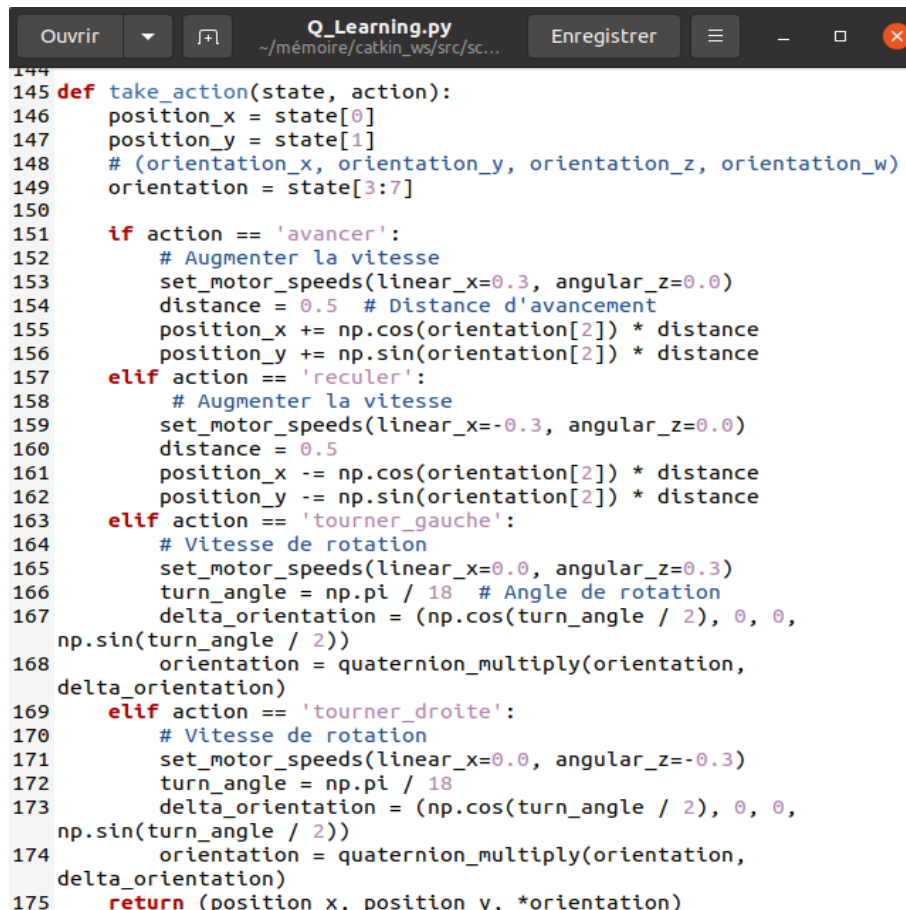
```
15 # Paramètres du Q-learning
16 alpha = 0.1
17 gamma = 0.9
18 epsilon = 0.1
19 num_episodes = 1000
20 # Chargement des données
21 ultrasonic_data = pd.read_csv('Ultrason_Data.csv')
22 odometry_data = pd.read_csv('Odometre_Data.csv')
23 lidar_data = pd.read_csv('lidar_Data.csv')
24
25 # Un tableau contenant l'ensemble des actions possibles
26 actions = ['avancer', 'reculer', 'tourner_gauche',
    'tourner_droite']
27 # numéro des actions num_action=4 dans ce cas
28 num_actions = len(actions)
29 # Initialisation de la table Q
30 num_states = 100
31 Q = np.zeros((num_states, num_actions))
32 goal_threshold = 0.1
33 collision_threshold = 0.8
```

FIGURE 4.7 – Initialisation de la fonction Q et des paramètres du Q-learning pour la navigation du robot

4.4.5 Algorithme d'apprentissage

L'algorithme Q-learning se déroule en plusieurs étapes :

- **Exploration** : Le robot explore l'environnement et choisit des actions selon une stratégie d'exploration. La fonction `take_action` (Figure 4.8) permet au robot d'exécuter une action spécifique en fonction de son état actuel. Elle prend en entrée l'état du robot (comprenant sa position et son orientation) ainsi que l'action à réaliser. Et retourne la nouvelle position et orientation du robot après l'exécution de l'action.



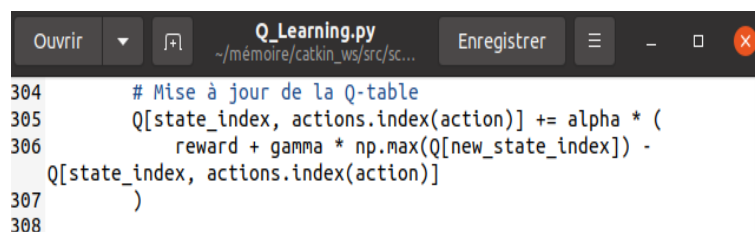
```

144
145 def take_action(state, action):
146     position_x = state[0]
147     position_y = state[1]
148     # (orientation_x, orientation_y, orientation_z, orientation_w)
149     orientation = state[3:7]
150
151     if action == 'avancer':
152         # Augmenter la vitesse
153         set_motor_speeds(linear_x=0.3, angular_z=0.0)
154         distance = 0.5 # Distance d'avancement
155         position_x += np.cos(orientation[2]) * distance
156         position_y += np.sin(orientation[2]) * distance
157     elif action == 'reculer':
158         # Augmenter la vitesse
159         set_motor_speeds(linear_x=-0.3, angular_z=0.0)
160         distance = 0.5
161         position_x -= np.cos(orientation[2]) * distance
162         position_y -= np.sin(orientation[2]) * distance
163     elif action == 'tourner_gauche':
164         # Vitesse de rotation
165         set_motor_speeds(linear_x=0.0, angular_z=0.3)
166         turn_angle = np.pi / 18 # Angle de rotation
167         delta_orientation = (np.cos(turn_angle / 2), 0, 0,
np.sin(turn_angle / 2))
168         orientation = quaternion_multiply(orientation,
delta_orientation)
169     elif action == 'tourner_droite':
170         # Vitesse de rotation
171         set_motor_speeds(linear_x=0.0, angular_z=-0.3)
172         turn_angle = np.pi / 18
173         delta_orientation = (np.cos(turn_angle / 2), 0, 0,
np.sin(turn_angle / 2))
174         orientation = quaternion_multiply(orientation,
delta_orientation)
175     return (position_x, position_y, *orientation)

```

FIGURE 4.8 – Définition de la fonction `take_action`

- **Mise à jour de la Q-table** : Après chaque action, la fonction Q est mise à jour en fonction de la récompense reçue et de la valeur maximale des actions possibles dans l'état suivant (Figure 4.9)



```

304     # Mise à jour de la Q-table
305     Q[state_index, actions.index(action)] += alpha * (
306         reward + gamma * np.max(Q[new_state_index]) -
Q[state_index, actions.index(action)]
307     )
308

```

FIGURE 4.9 – Mis à jour de la Q-table

- **Convergence** : Ce processus se répète jusqu'à ce que la fonction Q converge, c'est-à-dire que les valeurs deviennent stables et que le robot ait appris les meilleures actions à entreprendre dans chaque état.
- **Gestion des récompenses** : La gestion des récompenses est essentielle pour guider l'apprentissage du robot. Nous avons défini des récompenses positives pour les actions menant à l'objectif et des récompenses négatives pour les collisions avec des obstacles. Cette approche incite le robot à éviter les zones dangereuses tout en cherchant à atteindre son but. La fonction `evaluate_action` attribue des scores (qui peuvent être considérés comme des récompenses) en fonction de l'état du robot et de l'action qu'il choisit d'effectuer. Voici les principales caractéristiques de cette fonction en relation avec la gestion des récompenses :
 - **Pénalité pour collision** : Si l'action entraîne une collision (c'est-à-dire si la distance à un obstacle est inférieure à un seuil défini), la fonction retourne une pénalité de -10. Cela incite le robot à éviter des actions qui pourraient le mener à une collision.
 - **Récompense pour atteinte de l'objectif** : Si le robot atteint l'objectif en exécutant une action, la fonction retourne une récompense de 20. Cela encourage le robot à se diriger vers l'objectif.
 - **Distance à l'objectif** : En retournant un score basé sur la distance à l'objectif, la fonction guide le robot à minimiser cette distance. Plus le robot est proche de l'objectif, meilleur sera le score (récompense positive).

4.5 Comparaison

4.5.1 Méthodologie

Le Dynamic Window Approach (DWA) est un algorithme basé sur la planification de trajectoires en temps réel. Il évalue plusieurs vitesses et trajectoires possibles en tenant compte des contraintes dynamiques du robot et de l'environnement, prenant des décisions immédiates basées sur l'état actuel. En revanche, le Q-Learning est un algorithme d'apprentissage par renforcement qui apprend à partir des expériences en interagissant avec l'environnement. Il optimise une politique d'action pour maximiser les récompenses cumulées, avec des décisions fondées sur une fonction de valeur qui est progressivement apprise au fil du temps. Ainsi, tandis que DWA se concentre sur des décisions instantanées et réactives, le Q-Learning développe une stratégie à long terme à travers l'apprentissage.

4.5.2 Réactivité

Le Dynamic Window Approach (DWA) se distingue par sa grande réactivité aux changements d'environnement, car il évalue en temps réel les trajectoires possibles pour éviter les obstacles immédiats. En revanche, le Q-Learning peut être moins réactif dans des environnements dynamiques, car il nécessite souvent un temps d'apprentissage préalable pour affiner sa politique d'action, ce qui peut retarder les réponses aux modifications rapides de l'environnement.

4.5.3 Planification à Long Terme

Le Dynamic Window Approach (DWA) se concentre principalement sur des décisions à court terme, ce qui peut le limiter dans la planification de trajectoires sur de longues distances. En revanche, le Q-Learning permet une planification à long terme, car il optimise les actions en fonction des récompenses futures anticipées. Cette capacité à envisager les conséquences à long terme peut conduire à des stratégies plus globales et efficaces, adaptant les décisions en fonction des objectifs finaux plutôt que de se limiter aux situations immédiates.

4.5.4 Gestion des Objets Dynamiques

Le Dynamic Window Approach (DWA) peut avoir des difficultés à gérer efficacement les objets en mouvement, en raison de sa focalisation sur des décisions immédiates qui ne tiennent pas toujours compte des dynamiques futures. En revanche, le Q-Learning a l'avantage d'apprendre à anticiper le comportement des objets dynamiques grâce à l'apprentissage basé sur des expériences passées. Cette capacité améliore sa réactivité et son adaptation à des environnements changeants, permettant une navigation plus fluide et sécurisée face à des obstacles mobiles.

4.5.5 Complexité et Implémentation

Le Dynamic Window Approach (DWA) est relativement simple à implémenter et à comprendre, ce qui le rend accessible pour de nombreux projets robotiques, même pour ceux qui débutent dans le domaine. En revanche, le Q-Learning peut être plus complexe à mettre en œuvre, nécessitant une configuration appropriée des paramètres d'apprentissage et des récompenses. De plus, le processus d'apprentissage peut être long, ce qui demande une expertise supplémentaire et un investissement de temps pour optimiser les performances de l'algorithme.

4.5.6 Robustesse

Le Dynamic Window Approach (DWA) peut être sensible aux erreurs de localisation, ce qui peut affecter ses performances et compromettre la navigation précise du robot. En revanche, le Q-Learning est généralement plus robuste face aux incertitudes, car il apprend à partir d'une variété d'expériences, ce qui lui permet de s'adapter à différentes situations. Cependant, sa performance finale dépend fortement de la qualité des données d'apprentissage, ce qui souligne l'importance d'un bon échantillonnage et d'une configuration adéquate lors de l'entraînement de l'algorithme.

4.6 Validation expérimentale

Les résultats de l'exécution de l'algorithme de navigation Q-learning sont présentés dans la Figure 4.10. Cette image détaille les différentes étapes du processus de navigation du robot au sein de son environnement. On peut y observer les coordonnées initiales du robot et de son objectif. Le script affiche ensuite les actions choisies par le robot, comme avancer, tourner à droite ou à gauche, ou reculer. Les distances restantes jusqu'à l'objectif après chaque action sont également indiquées, de même que les récompenses attribuées en fonction de la progression du robot vers sa cible. Ces informations quantitatives permettent d'évaluer la capacité du système de navigation à guider le robot de manière efficace et sûre, en évitant les obstacles tout en se rapprochant de l'objectif.

```
hidri@ubuntu: ~/mémoire/catkin_ws/src/scripts
/home/hidri/mémoire/catkin_ws/src/... x hidri@ubuntu: ~/mémoire/catkin_ws/... x
hidri@ubuntu:~/mémoire/catkin_ws$ cd src/scripts
hidri@ubuntu:~/mémoire/catkin_ws/src/scripts$ rosrn scripts SensorDataLogger.py
& rosrn scripts Q_Learning.py
[1] 12380
Entrez la coordonnée x de l'objectif : 1
Entrez la coordonnée y de l'objectif : 1
État initial de robot : (-1.1762398302269435e-09, 3.4980546410878783e-08, 0.0, 5
.752178160544591e-11, -1.2935781532840228e-11, 0.7068251869520568, 0.70738826332
51629, 1.9311663574037203e-09, 4.0789576274942056e-11, 0.0, 0.0, 0.0, 0.0)
La distance initiale entre le robot et son objectif est 1.4142135384698409
-----
Action choisie: avancer
Distance restante jusqu'à l'objectif après l'action: 1.41 m
Récompense: le robot se rapproche de l'objectif.
La distance entre le robot et un obstacle est 1.6052359342575073
-----
Action choisie: avancer
Distance restante jusqu'à l'objectif après l'action: 0.96 m
Récompense: le robot se rapproche de l'objectif.
La distance entre le robot et un obstacle est 1.3177471160888672
-----
Action choisie: reculer
Distance restante jusqu'à l'objectif après l'action: 2.03 m
Pénalité: le robot s'éloigne de l'objectif.
```

FIGURE 4.10 – Résultat de l'exécution

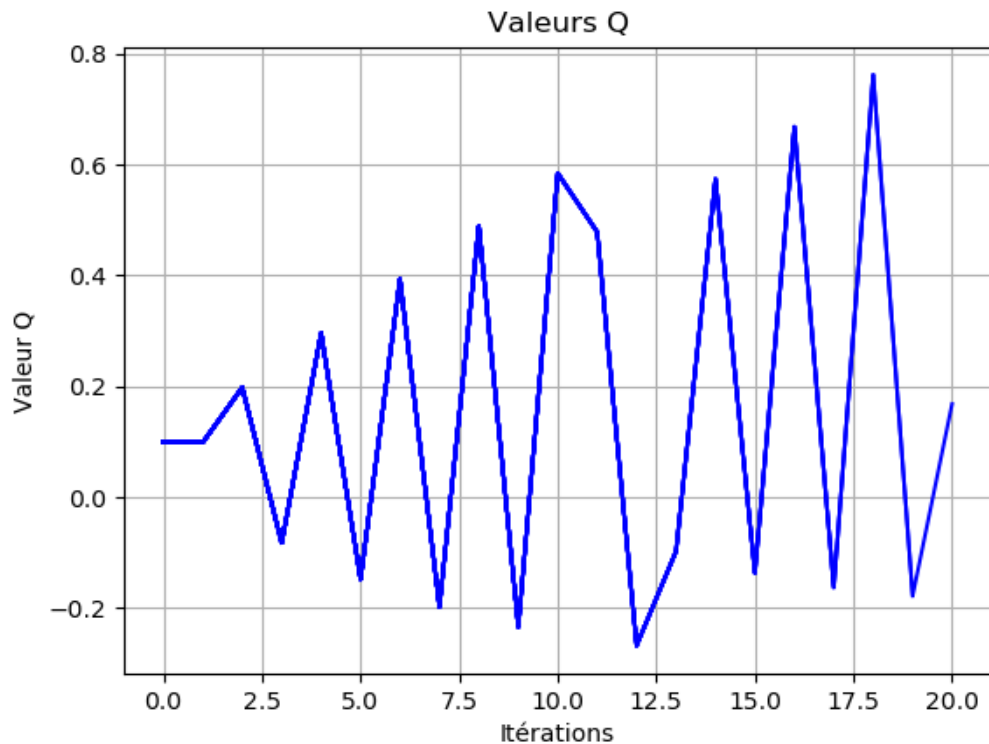


FIGURE 4.11 – Graphe de la valeur de Q

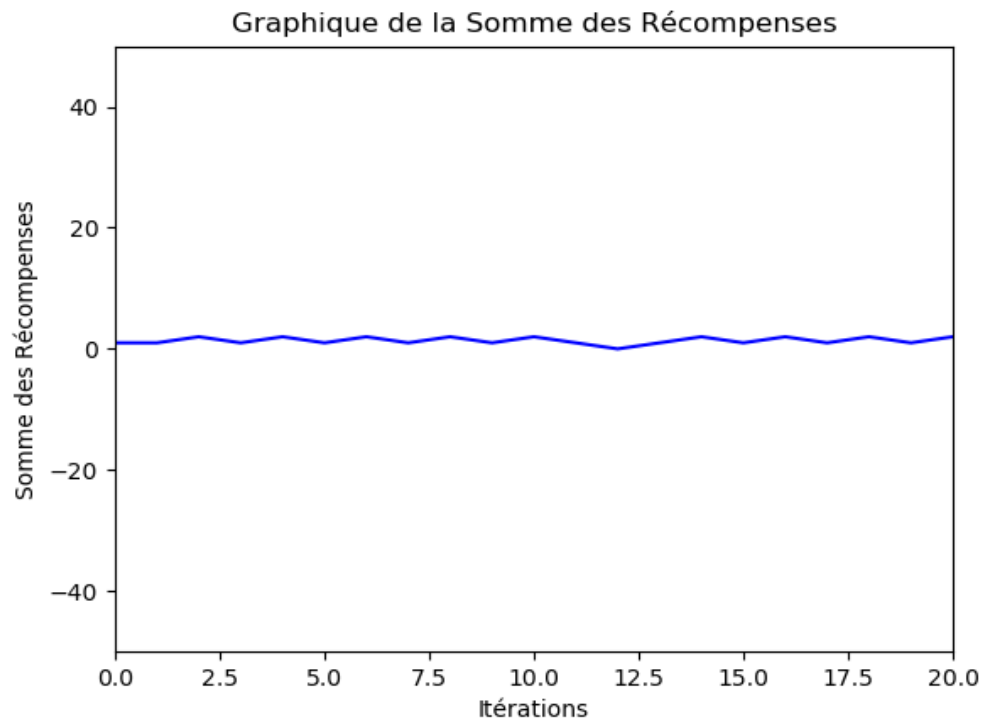


FIGURE 4.12 – Graphe de la somme des récompenses

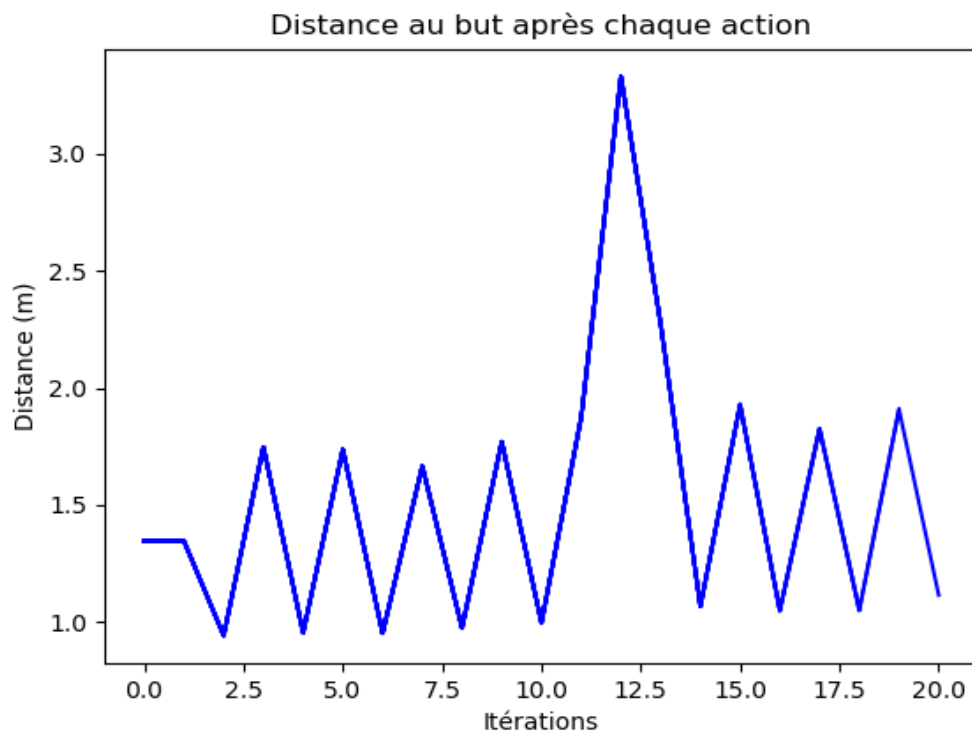


FIGURE 4.13 – Graphe de la distance au but après chaque action

4.7 Conclusion

L'étude menée dans ce chapitre a permis de mieux comprendre les forces et les faiblesses de l'algorithme de navigation DWA existant. L'implémentation d'un algorithme d'appren-

tissage par renforcement, le Q-Learning, a montré des résultats prometteurs, notamment en termes de réactivité, de planification à long terme et de gestion des obstacles dynamiques. Cependant, des efforts supplémentaires sont nécessaires pour optimiser la complexité de notre algorithme Q-Learning et améliorer davantage sa robustesse. Les perspectives envisagées, telles que l'intégration de connaissances a priori ou l'utilisation d'autres techniques d'apprentissage, ouvrent la voie à de futures améliorations de l'algorithme de navigation. Ce travail constitue une étape importante dans le développement d'un système de navigation autonome fiable et performant.

Conclusion Générale et Perspectives

Au terme de ce travail de recherche consacré à l'étude des différents algorithmes intelligents de navigation autonome et l'implémentation d'un algorithme d'apprentissage par renforcement (Q-Learning) pour la navigation autonome du robot Mini-Lab, nous pouvons affirmer que les objectifs fixés initialement ont été atteints, apportant une contribution significative à ce domaine en pleine évolution.

Notre étude s'est inscrite dans un contexte où l'automatisation et l'intelligence artificielle transforment profondément le paysage technologique, particulièrement dans le domaine de la robotique mobile autonome. Face aux défis complexes de la navigation autonome dans un environnement dynamique, nous avons étudié des algorithmes intelligents de navigation et implémenté un algorithme basé sur l'apprentissage par renforcement qui a permis de répondre aux exigences de performance et de fiabilité.

La démarche méthodologique adoptée, structurée autour de trois axes complémentaires, a permis d'aborder la problématique de manière exhaustive : L'étude bibliographique approfondie nous a fourni une base théorique solide sur les algorithmes de navigation autonome, permettant d'identifier les approches les plus prometteuses. L'analyse détaillée de l'architecture du robot Mini-Lab a ensuite guidé nos choix d'implémentation, assurant une adéquation optimale entre les solutions proposées et les contraintes matérielles.

Le cœur de notre contribution réside dans le développement et l'implémentation d'algorithmes intelligents basés sur l'apprentissage par renforcement. Cette approche, comparée à l'algorithme de la fenêtre dynamique (DWA), a démontré des performances supérieures en termes de :

- Adaptabilité aux environnements complexes
- Efficacité de la planification de trajectoire
- Réactivité face aux obstacles
- Robustesse globale du système

L'utilisation de l'environnement ROS, couplée aux outils de simulation Gazebo et Rviz, a permis de valider nos solutions dans des conditions réalistes. Les tests menés ont confirmé la pertinence de notre approche, tout en mettant en lumière des pistes d'amélioration prometteuses.

Ce travail ouvre la voie à plusieurs axes de développement futurs :

- L'optimisation des algorithmes d'apprentissage pour améliorer les performances en situations complexes
 - L'adaptation du système à des environnements plus dynamiques et imprévisibles
 - L'intégration de capacités de perception plus avancées
 - La validation des résultats sur le robot physique dans des conditions réelles
-

En définitive, cette étude a non seulement atteint ses objectifs initiaux mais a également contribué à l'avancement des connaissances dans le domaine de la robotique mobile autonome. Les solutions développées pour le Mini-Lab constituent une base solide pour de futures innovations dans ce domaine en constante évolution, où l'intelligence artificielle joue un rôle de plus en plus central dans l'amélioration des capacités des robots mobiles.

Bibliographie

- [1] A. Juton, V. Noël, R. Lali. *Introduction à l'apprentissage par renforcement*. 2022, La Revue 3EI, numéro 109, Dossier Intelligence Artificielle, Culture Sciences de l'Ingénieur 29
 - [2] A. Benmakhlouf. *Navigation Intelligente Autonome Pour Robot Mobile*. 2023, Université Kasdi Merbah Ouargla. 17
 - [3] C. Kessler. *Une première approche du SLAM (Localisation et cartographie simultanées pour un robot)*. 2023, Département d'Enseignement et de Recherche Génie Mécanique, ENS Paris-Saclay. 36
 - [4] Chen, Z., et al. (2023). *Multi-Sensor Fusion in Robotics : A Deep Learning Perspective*. Robotics and Autonomous Systems. 19
 - [5] Cadena, C., et al. (2023). *Visual-Inertial SLAM : State of the Art and Future Directions*. IEEE Robotics and Automation Magazine. 20
 - [6] C. Strauss et F. Sahin. *Autonomous navigation based on a Q-learning algorithm for a robot in a real environment*. 2008, International Conference on System of Systems Engineering (SYSOSE), 2008, pp. 1-6, Rochester, NY, USA. 35
 - [7] G. Iliev, A. Rechnitzer, and S. G. Whittington. *Localization of random copolymers and the Morita approximation*. Department of Chemistry, University of Toronto, Department of Mathematics and Statistics, University of Melbourne. 18
 - [8] I. Bukhori et I. Zool Hilmi. *Detection of kidnapped robot problem in Monte Carlo localization based on the natural displacement of the robot*. 2017, International Journal of Advanced Robotic Systems. 18
 - [9] I. GUEBLI. *Navigation et commande d'un robot mobile à 4 roues sous ROS*. 2019/2020, Université 8 Mai 1945 Guelma. 12
 - [10] M. Saravanan, P. Satheesh Kumar, et A. Sharma. *IoT Enabled Indoor Autonomous Mobile Robot using CNN and Q-Learning*. 2019, International Conference on Industry 4.0, Artificial Intelligence, and Communications Technology (IAICT), 25-27 October 2019, Chennai, India. 35
 - [11] J. Sánchez-Ibáñez., C. Pérez-del-Pulgar. and A. García-Cerezo. *Path Planning for Autonomous Mobile Robots*. 2021, Department of Systems Engineering and Automation, Universidad de Málaga. 21
 - [12] J. Sublime. *L'apprentissage non-supervisé et ses contradictions*. 2022, Bulletin de la Société Informatique de France. 29
 - [13] L. Cherroun et M. Boumehraz. *Evitement d'Obstacles pour un Robot Mobile en Utilisant L'apprentissage par Renforcement*. Département d'Automatique, Université de Biskra 62
 - [14] L. Rouvière. *Apprentissage supervisé - Machine Learning*. 2024, Université Rennes. 27
 - [15] Liu, H., et al. (2024). *PerceptionFormer : Multi-Modal Fusion with Transformers for Robot Environmental Understanding*. Conference on Robot Learning (CoRL). 20
 - [16] O. Brock, O Khatib. *High-speed Navigation Using the Global Dynamic Window Approach.*, Department of Computer Science Stanford University. 59
 - [17] P Fitzpatrick, L. Natale, et G. Metta. *YARP, a Thin Middleware for (Humanoid) Robots*. 2010, Robotics Program : Enabling Intelligence through Middleware. 15
-

- [18] P. Anggraeni, M. Mrabet, M. Defoort, et M. Djemai. *Development of a wireless communication platform for multiple-mobile robots using ROS*. 2018, International Conference on Control Engineering & Information Technology (CEIT), 25-27 October 2018, Istanbul, Turkey. [39](#)
- [19] S. Janny, L. De Matteis, et W. Shu-Quartier. *Introduction à l'apprentissage profond*. 2022, La Revue 3EI, numéro 108. [31](#)
- [20] Tahar, M., et al. (2024). *Learning-Based Qualitative Navigation for Mobile Robots*. International Journal of Robotics Research. [20](#)
- [21] *Une première mise en œuvre de ROS (Robot Operating System)*. 2012, Université Numérique d'Ingénierie et de Technologie (UNIT). [14](#)
- [22] Yang, K., et al. (2024). *Bio-Inspired Perception Systems for Autonomous Navigation*. Nature Machine Intelligence. [22](#)
- [23] Zhang, J., & Singh, S. (2023). *Advanced LiDAR Systems for Autonomous Mobile Robots*. IEEE Robotics and Automation Letters.

